

AGRI-PROFIT (A Farm Record and Decision-Support Platform)

BY

ERHABOR ISRAEL ISIMA
2021/1/83967EA

**DEPARTMENT OF AGRICULTURAL AND BIORESOURCES
ENGINEERING
FEDERAL UNIVERSITY OF TECHNOLOGY
MINNA**

AUGUST, 2026

AGRI-PROFIT (A Farm Record and Decision-Support Platform)

BY

ERHABOR ISRAEL ISIMA
2021/1/83967EA

**PROJECT SUBMITTED TO THE DEPARTMENT OF
AGRICULTURAL AND BIORESOURCES ENGINEERING,
SCHOOL OF INFRASTRUCTURE PROCESS ENGINEERING AND
TECHNOLOGY, FEDERAL UNIVERSITY OF TECHNOLOGY,
MINNA, NIGERIA IN PARTIAL FULFILMENT OF THE
REQUIREMENT FOR THE AWARD OF THE DEGREE OF
BACHELOR OF ENGINEERING IN AGRICULTURAL AND
BIORESOURCES ENGINEERING**

AUGUST, 2026

DECLARATION

I hereby declare that this project titled: “AGRI-PROFIT (A Farm Record and Decision-Support Platform)” is a collection of my original project work and it has not been presented for any other qualification anywhere. Information from other sources (published or unpublished) has been duly acknowledged.

ERHABOR, ISRAEL ISIMA

.....

.....

.....

2021/1/83967EA
Federal University of Technology,
Minna, Nigeria.

Signature and Date

CERTIFICATION

The project titled: “AGRI-PROFIT (A Farm Record and Decision-Support Platform)”
by: ERHABOR, ISRAEL ISIMA, (2021/1/83967EA), meets the regulations governing
the award of the degree of Bachelor of Engineering in Agricultural and Bioresources
Engineering of the Federal University of Technology, Minna.

.....

.....

.....

Project Supervisor

Signature and Date

.....

.....

.....

Head of Department

Signature and Date

.....

.....

.....

External Examiner

Signature and Date

DEDICATION

This project is dedicated first to God Almighty. He is my source of strength and inspiration. He gave me perseverance throughout this journey.

I dedicate this project to my Mother. Her love and sacrifices made my education possible. She supported my growth at every stage. I am forever grateful to her.

I dedicate this project to my dear friend, Olu-Arimoro Fadunsayemi Enoch. His contribution to this project was invaluable, in many ways. I thank him for believing in this work. I thank him for standing behind it in ways I cannot fully express.

I also dedicate this project to my other friends. They offered encouragement and support in their own ways. They were true companions throughout this journey. I thank them for being part of it.

ACKNOWLEDGEMENTS

I wish to express my profound gratitude to my project supervisor, Engr. Dr. Simeon M. I., for their invaluable guidance, patience, and constructive feedback throughout the course of this project.

My sincere appreciation also goes to the Head of Department, and to all the academic and non-academic staff of the Department of Agricultural and Bioresources Engineering for their administrative support and for creating an environment conducive to academic growth.

I would also like to thank my friends and family for their unwavering moral and financial support, which has been instrumental in the successful completion of my undergraduate studies. Finally, above all, I thank God for the grace and strength to see this endeavor to fruition.

ABSTRACT

Nigerian smallholder and medium-scale farms keep production and cost records on paper, so the cost of production per crop is rarely known at sale; existing farm management software assumes continuous connectivity and charges per user seat. This work designed, implemented and evaluated AGRI-PROFIT, an offline-first web platform comprising a soft-immutable paired-write ledger in which every operational record posts its financial transaction in a single commit and is corrected only by a linked contra entry, a deterministic decision-support layer computing nine quantities including per-crop gross margin, unit cost of production on two bases and retrospective break-even yield, and a post-harvest drying module applying thin-layer kinetics to the farm's own records. A design-and-build methodology was adopted, its four-strand evaluation specified in advance. Functional verification at the submitted commit returned 422 collected backend cases — 418 passed, 4 skipped, none failed — at 96% statement coverage over 1,594 statements, with 148 frontend tests across 13 files at 43.18% statement coverage, and the drying model recovered its designed parameters exactly. For a 100 kg maize harvest dried from 25% to 13% moisture and weighed out at 84 kg, unit cost moves from ₦35.00 to ₦41.67 per kilogram: post-harvest mass loss expressed in money. Under 0.4 Mbps simulated throttling the service worker reduced network transfer on a repeat visit from 139,245 bytes to 127, first paint from 5,661.2 ms to 70.6 ms and largest contentful paint to 1,612.0 ms. Role-based authorisation over three roles and twelve permissions, expiring hashed share tokens, login throttling and credential scrubbing were verified live on the running stack. The yield-forecast tier returned R^2 0.9762, MAE 0.2414 t/ha and RMSE 0.4883 t/ha, reproducible to six decimal places, on synthetic data alone; no claim of field-predictive accuracy is made. Measured resource consumption below 205 MiB gives an estimated ₦20,950 per month, or ₦105 per farm at a two-hundred-farm tenancy. The usability evaluation was not conducted and no System Usability Scale score is reported, so that objective is only partly met, and the platform has never been deployed to serve a real user.

TABLE OF CONTENTS

CONTENT.....	PAGE
CHAPTER ONE	1
1.1 Background of the Study	1
1.2 Statement of the Problem.....	2
1.3 Aim and Objectives	3
1.4 Justification of the Study	5
1.5 Scope and Limitations of the Study	6
CHAPTER TWO	9
2.1 Conceptual Review	9
2.1.1 The Evolution of Farm Record Management	9
2.1.2 Decision-Support Systems (DSS).....	10
2.1.3 The Shift from Manual to Digital Agriculture (Agriculture 4.0).....	10
2.2 Theoretical Framework.....	11
2.2.1 Technology Acceptance and Innovation Systems Theories	11
2.2.2 The Theory of Bounded Rationality	11
2.2.3 Socio-Technical Systems Perspective	12
2.3 The Nigerian Context: Precision Agriculture and Mobile Scaling.....	12
2.4 Empirical Review and Socio-Technical Barriers to Adoption	13
2.4.1 Review of Existing Systems	13
2.4.2 Socio-Technical Barriers to Adoption	13
2.5 Data-Sharing and Stakeholder Trust.....	14

2.6 Structuring the Decision-Support Prediction Model for AGRI-PROFIT	15
2.6.1 Data Layer and Feature Definition	15
2.6.2 Data Pre-processing	16
2.6.3 A Tiered Modelling Strategy	17
2.6.4 Model Training, Validation, and Evaluation Metrics	18
2.6.5 From Prediction to Decision Support	19
2.7 Post-Harvest Drying Kinetics and the Costing of Marketable Output	19
2.7.1 Thin-Layer Drying Models	20
2.7.2 The Post-Harvest Context in Africa, and the Costing Gap.....	21
2.8 Gap in Literature	22
CHAPTER THREE	24
3.1 Introduction.....	24
3.2 Research and Development Methodology	24
3.3 System Requirements Analysis	25
3.3.1 Functional Requirements	25
3.3.2 Non-Functional Requirements	26
3.4 System Architecture.....	27
3.4.1 Overall Architecture	27
3.4.2 Technology Stack	28
3.5 Data Model Design	28
3.6 Module Design.....	31
3.6.1 Offline Operational Logbook.....	31

3.6.2 Financial Ledger Engine	32
3.6.3 Reporting	33
3.6.4 Mechanisation and Equipment Tracker	34
3.6.5 Predictive Decision-Support Engine.....	34
3.6.6 Authentication and Per-Farm Data Boundary	36
3.6.7 Tokenised Stakeholder Reporting.....	36
3.6.8 Feature-Phone (USSD/SMS) Data Entry — Designed, Not Implemented ...	37
3.6.9 Post-Harvest Drying Module	38
3.6.10 Enterprise Economics Module.....	39
3.7 Security Design.....	39
3.8 Evaluation Methodology.....	40
3.8.1 Functional Verification	41
3.8.2 Performance Benchmarking	41
3.8.3 Hosting-Cost Analysis	42
3.8.4 Usability Evaluation (Heuristic and SUS)	42
3.8.5 Designed Field Evaluation (Future Work).....	43
3.9 Summary.....	43
CHAPTER FOUR.....	45
4.1 Introduction.....	45
4.2 The Implemented System	47
4.2.1 Deployed components.....	47
4.2.2 Functional inventory	48

4.3 Functional Verification: Automated Test Evidence	53
4.3.1 Suite identity and headline results	53
4.3.2 Where the coverage sits	55
4.3.3 Capability-to-proof mapping	58
4.3.4 Movement against the previously reported figures.....	60
4.4 Negative-Control Testing	63
4.5 Manually Inspected Evidence	65
4.5.1 Components verified visually only	65
4.5.2 Live system verification.....	67
4.5.3 Service-worker attribution by driven browser	67
4.6 Decision-Support Results from the Live Ledger	68
4.6.1 Per-crop gross margin and ranking.....	69
4.6.2 Unit cost of production on two bases.....	70
4.6.3 Retrospective break-even yield.....	71
4.6.4 A Withdrawn Defect Report: Per-Crop Reversal Netting	72
4.7 Enterprise Economics Results	73
4.7.1 Cost structure and classification coverage	73
4.7.2 The depreciation overlay and fixed-cost allocation	74
4.7.3 Dual break-even price and yield sensitivity	75
4.7.4 Partial budget and implementation parity	77
4.7.5 Yield baseline	77
4.8 Post-Harvest Drying Results.....	78

4.8.1 Verification by designed fixture	78
4.8.2 The seeded drying run in full	79
4.8.3 Input validation	81
4.8.4 Coupling to the financial ledger.....	81
4.9 Yield-Forecast Model Evaluation on Synthetic Data	82
4.10 Performance Results	85
4.10.1 Method and conditions.....	85
4.10.2 The principal finding	87
4.10.3 Two confounds, and a discarded pair of measurements	88
4.10.4 A defect found during measurement.....	90
4.11 Hosting-Cost Analysis	91
4.11.1 Measured footprint.....	91
4.11.2 A correction to the earlier reasoned estimate.....	92
4.11.3 Estimated monthly cost.....	92
4.11.4 Cost per farm at scale.....	93
4.12 Security, Authorisation and Deployment Verification	94
4.13 Usability Evaluation	99
4.14 Results Against the Stated Objectives	101
4.15 Limitations of the Results	106
4.16 Summary.....	109
CHAPTER FIVE	112
5.1 Summary.....	112

5.1.1 The Problem, the Approach, and the Procedures Used	112
5.1.2 Summary of Findings.....	114
5.1.3 Achievement Against the Research Objectives	116
5.2 Conclusion	124
5.2.1 Interpretation of the Major Findings.....	124
5.2.2 Comparison with the Literature	130
5.2.3 Limitations	134
5.2.4 Conclusion	137
5.3 Recommendations.....	139
5.3.1 Implications	139
5.3.2 Recommendations and Suggestions for Further Studies	141
REFERENCES	145

LIST OF TABLES

TABLE.....	PAGE
Table 3.1: Principal entities of the AGRI-PROFIT data model	30
Table 4.1: The evidential classification applied throughout Chapter Four.....	45
Table 4.2: Implemented capabilities and their reachability	48
Table 4.3: Verification results at the baseline commit 78a68c2.....	53
Table 4.4: Backend modules below full statement coverage.....	55
Table 4.5: Frontend test files at the baseline commit, with collected test counts.....	57
Table 4.6: Behaviours verified by automated test, with the evidence for each	58
Table 4.7: Reconciliation of the test figures	61
Table 4.8: Negative-control (mutation) runs	64
Table 4.9: Service-worker attribution probe	68
Table 4.10: Per-crop decision support, farm 26, at the baseline commit	69
Table 4.11: Unit cost of production, farm 26	70
Table 4.12: Retrospective break-even yield, farm 26.....	71
Table 4.13: Cost structure, farm 26	73
Table 4.14: Derived fixed-cost overlay, farm 26.....	75
Table 4.15: Break-even price, maize, farm 26.....	75
Table 4.16: Yield sensitivity matrix, maize, farm 26	76
Table 4.17: Maize drying run, farm 26, as computed by the platform	79
Table 4.18: Per-crop drying summary, farm 26.....	80
Table 4.19: Yield model as served by the platform	82

Table 4.20: Lighthouse results at the baseline commit, 29 August 2026 (medians)	85
Table 4.21: Measured container footprint.....	91
Table 4.22: Estimated monthly operating cost	92
Table 4.23: Reasoned cost per farm served	93
Table 4.24: Security and authorisation verification against the running stack	94
Table 4.25: Deployment, migration and recovery verification.....	97
Table 4.26: Usability evaluation — to be completed from retained instruments	100
Table 4.27: Usability issue log — to be completed.....	101
Table 4.28: Evidence and residual gap, by objective	101
Table 5.1: Objective-to-conclusion matrix	116

LIST OF FIGURES

FIGURE.....	PAGE
Figure 3.1: AGRI-PROFIT system architecture	28
Figure 3.2: Entity-relationship diagram of the AGRI-PROFIT data model	31
Figure 3.3: The paired write — sequence of a single record creation	33

CHAPTER ONE

1.0 INTRODUCTION

1.1 Background of the Study

The global agricultural sector faces the unprecedented challenge of increasing food production to sustain a human population projected to approach 10 billion by 2050 (Dayıoğlu & Türker, 2021). To meet these escalating demands while mitigating the adverse impacts of climate change, resource depletion, and environmental degradation, traditional farming methodologies are rapidly shifting towards “Agriculture 4.0” (Abbasi, Martinez, & Ahmad, 2022). This paradigm integrates modern information and communication technologies (ICT), the Internet of Things (IoT), artificial intelligence (AI), and data analytics to optimise agricultural systems (Abiri, Rizan, Balasundram, Shahbazi, & Abdul-Hamid, 2023). At the core of this digital transformation is the necessity for accurate, real-time data collection and on-farm recordkeeping, which serve as the foundation for precision agriculture and operational efficiency (Basir, Buckmaster, Raturi, & Zhang, 2024).

Historically, agricultural practitioners have relied on unstructured, paper-based logbooks or human memory to track operations. While these traditional methods possess inherent provenance, they are highly susceptible to data fragmentation, loss, recall bias, and significant transcription errors (Basir et al., 2024). In response, Farm Management Information Systems (FMIS) evolved from simple digitalised field books into complex platforms designed to collect, process, store, and disseminate data to support human decision-making in farm enterprises (Fountas et al., 2015; Tummers, Kassahun, & Tekinerdogan, 2019). Modern digital recordkeeping platforms streamline administrative burdens, facilitate data interoperability, and enable precise financial tracking and bioprocess monitoring. Empirical adoption studies further establish that

uptake of these systems is strongly conditioned by farmers' education, access to advisory services, and access to rural credit—factors that are unevenly distributed in developing economies (Carrer, de Souza Filho, & Batalha, 2017).

However, the distribution and adoption of these digital agricultural technologies remain highly unequal globally (Javaid, Haleem, Singh, & Suman, 2022; Mhlanga & Ndhlovu, 2023). In Sub-Saharan Africa, including Nigeria, the integration of advanced FMIS faces unique socio-economic and infrastructural barriers. The Nigerian agricultural landscape predominantly comprises smallholder farmers and medium-scale enterprises that frequently lack access to robust technological infrastructure and capital (Abioye et al., 2026). Recent market analyses by the Global System for Mobile Communications Association (GSMA) reveal that 68% of Nigerians in rural areas lacked smartphones in 2022, with rural ownership stagnating at 32% compared to 58% in urban areas (Jaiyeola, 2023). This severe digital divide is primarily exacerbated by high inflation, device-affordability constraints, and pervasive digital illiteracy (Jaiyeola, 2023; Mhlanga & Ndhlovu, 2023). Consequently, while the existing 32% smartphone penetration provides a critical leapfrog opportunity to deploy mobile-based agricultural advisory services, comprehensive digital farm record management remains heavily underutilised. Engineers and software developers must therefore contextualise global digital-farming advancements into localised, accessible platforms. Developing tools that merge farm mechanisation tracking, bioprocess monitoring, and financial management into a single decision-support architecture holds the potential to revolutionise agricultural productivity in Nigeria.

1.2 Statement of the Problem

Despite the well-documented benefits of digital agriculture, practitioners and engineers in Nigeria face severe bottlenecks in adopting existing Farm Management Information

Systems (FMIS) (Tummers et al., 2019). Current commercial FMIS platforms and Enterprise Resource Planning (ERP) systems frequently exhibit high complexity, lack standardised data formats, and impose prohibitive costs that exclude small and medium-scale farmers (Fountas et al., 2015; Giua, Materia, & Camanzi, 2021).

Furthermore, agricultural operations suffer from critical data fragmentation. Farm managers typically maintain Farm Financial Accounts (FFA) entirely separate from their operational and bioprocess records, resulting in duplicated data entry, administrative fatigue, and analytical inefficiencies (Poppe, Vrolijk, & Bosloper, 2023). Because essential metrics—such as equipment maintenance schedules, labour inputs, crop-yield data, and financial expenditures—are siloed, farmers cannot accurately correlate mechanisation inputs or biological variables with actual financial profitability (Gebresenbet et al., 2023).

Without interoperable, structured data, Nigerian agricultural enterprises cannot deploy predictive analytics or decision-support algorithms to optimise resource utilisation (van Klompenburg, Kassahun, & Catal, 2020). In rural contexts where poor internet connectivity and limited technical training exacerbate these issues, deploying resource-intensive, cloud-heavy ERP solutions proves impractical (Abbasi et al., 2022; Mhlanga & Ndhlovu, 2023). Therefore, a critical gap exists for a localised, unified digital platform—AGRI-PROFIT—that bridges financial tracking and operational farm data to generate actionable decision support tailored to the specific constraints of Nigerian agriculture.

1.3 Aim and Objectives

Aim

The primary aim of this research is to design, develop, and evaluate AGRI-PROFIT, a secure, integrated farm record management and decision-support platform that overcomes socio-technical adoption barriers to improve resource allocation, mechanisation cost tracking, post-harvest process accounting, and stakeholder data-sharing for Nigerian agricultural enterprises.

Objectives

To achieve this overarching aim, the specific, measurable objectives of the study are:

- i. To identify and evaluate the socio-technical barriers—specifically data fragmentation, interoperability constraints, and user complexity—that systematically hinder the widespread adoption of digital Farm Management Information Systems (FMIS) in the Nigerian context (Carrer et al., 2017; Tummers et al., 2019).
- ii. To formulate a conceptual system architecture for the AGRI-PROFIT platform that translates static operational and financial farm records into dynamic, actionable decision-support metrics for costing crop enterprises and for capturing mechanisation and equipment expenditure within the same ledger (Gebresenbet et al., 2023).
- iii. To design and integrate a secure data-sharing framework within the platform architecture that standardises yield and profit/loss reporting, serving as a verifiable analytical tool to build operational trust among agricultural banks, investors, and policymakers (Xiong et al., 2020; Abiri et al., 2023).

- iv. To implement and empirically evaluate the AGRI-PROFIT platform's usability, cost-effectiveness, and operational viability within the specific infrastructural constraints of Nigerian smallholder and medium-scale farms.
- v. To model the post-harvest drying of harvested produce using established thin-layer drying kinetics, and to couple the resulting moisture, water-balance and marketable-mass outcomes to the farm's financial record, so that the unit cost of production is expressed against saleable rather than harvested mass (Onwude, Hashim, Janius, Nawi, & Abdan, 2016; Baidhe, Clementson, Senyah, & Hammed, 2024).

1.4 Justification of the Study

The development of the AGRI-PROFIT platform delivers immediate economic and operational benefits to agricultural producers while significantly advancing the discipline of Agricultural and Bio-resources Engineering. For farmers and agribusiness managers, the system addresses the critical need to mitigate administrative burdens and overcome data fragmentation (Poppe et al., 2023). By synthesising operational field records with financial data, AGRI-PROFIT empowers farm managers to transition from intuition-based strategies (Husemann & Novkovic, 2012) to evidence-based precision farming. This transition allows producers to precisely calculate production costs, reduce resource wastage, and maximise gross margins at the specific crop-enterprise level (Basir et al., 2024).

Furthermore, this research specifically addresses the “human element” of digital agriculture by targeting the socio-technical barriers—such as poor software interoperability and user complexity—that cause existing systems to fail (Carrer et al., 2017; Tummers et al., 2019). By formulating a robust conceptual system architecture,

the platform addresses a major macro-economic problem for the agricultural sector: the lack of credible, standardised data required by agricultural banks and investors to approve financing or insurance (Gebresenbet et al., 2023; Xiong et al., 2020).

For Agricultural and Bio-resources Engineers, this research demonstrates the vital intersection of cyber-physical systems, data analytics, and agricultural management, often termed AgInformatics (Dayioğlu & Türker, 2021). As the Nigerian agricultural sector seeks to modernise, engineers must move beyond designing physical machinery to developing the software infrastructures required to manage those machines efficiently. Developing localised digital tools ensures that Nigeria can sustainably participate in the Agriculture 4.0 revolution, ultimately driving long-term food security and economic resilience (Mhlanga & Ndhlovu, 2023).

1.5 Scope and Limitations of the Study

Scope

This research encompasses the software-engineering lifecycle—including system architecture design, coding, and initial deployment—of the AGRI-PROFIT platform, tailored specifically for Nigerian agricultural practices. The platform primarily targets core FMIS functionalities: digital operational logbooks, financial-accounting integration, mechanisation and equipment cost tracking, post-harvest drying analysis coupled to the financial record, and decision support for crop enterprises. Livestock enterprises are outside the scope of the current implementation: the activity-category model would extend to them without structural change, but no livestock entity, category, field or metric is implemented, and no livestock capability is claimed anywhere in this work. Within the post-harvest strand the scope is confined to thin-layer drying kinetics, wet- and dry-basis moisture conversion, dry-matter and water balances,

safe-storage assessment for those crops whose thresholds are recorded, and the propagation of marketable mass into the unit-cost calculation; no other post-harvest unit operation is modelled.

Limitations

The study acknowledges specific constraints inherent to the operating environment. Given the high capital costs and limited rural network infrastructure in Nigeria, AGRI-PROFIT prioritises manual digital inputs via smartphones or computers rather than relying on fully automated, capital-intensive IoT sensor networks (Abbasi et al., 2022; Basir et al., 2024). In keeping with these realities, offline capability is implemented primarily for the data-entry (write) path, so that farmers can record operations and transactions without connectivity; a service worker additionally serves the application shell and a defined set of previously fetched read endpoints, so that recent results remain readable without a connection, while all computation is performed server-side (Mhlana & Ndhlovu, 2023). Furthermore, while immutable distributed-ledger (block chain) governance is reviewed in Section 2.5 as a mechanism for stakeholder trust, it is treated in this study as a conceptual design direction and future extension rather than part of the current implementation; within the present scope, secure data-sharing is achieved through authenticated, role-controlled, tokenised standardised reporting. The widespread deployment of the platform also faces infrastructural headwinds; notably, the 68% rural smartphone-usage gap (Jaiyeola, 2023) means the initial rollout will inherently target early-adopting smallholders and medium-scale enterprises with existing mobile access, and no feature-phone entry channel is implemented in this release. Finally, the decision-support modules employ deterministic, rule-based computation complemented by a lightweight tree-based ensemble—a Random Forest regressor—rather than advanced deep-learning algorithms, keeping the computational

requirement within modest server hosting and accessible mobile clients (van Klompenburg et al., 2020).

CHAPTER TWO

2.0 LITERATURE REVIEW

2.1 Conceptual Review

2.1.1 The Evolution of Farm Record Management

Agricultural data form the foundational basis of modern farm management. Historically, practitioners relied extensively on pen-and-paper methodologies, unstructured logbooks, and human memory to document operational inputs and crop yields. While these traditional, free-form methods possess historical provenance and flexibility, they severely restrict data mobility and are highly susceptible to fragmentation, physical loss, recall bias, and manual transcription errors. The transition toward digital recordkeeping initially introduced rudimentary spreadsheets; however, these lacked automated interoperability and were prone to manipulation.

To address these inefficiencies, Farm Management Information Systems (FMIS) emerged as centralised, planned architectures designed to collect, process, store, and disseminate agricultural data (Fountas et al., 2015). A critical evolutionary flaw, however, occurred in farm administration: the separation of Farm Financial Accounts (FFA) from operational bioprocess records. As Poppe, Vrolijk, and Bosloper (2023) observed, financial accounting and farm-management systems originated from the same historical note-taking practices but diverged into separate software ecosystems as they computerised. This separation forces farmers to duplicate data entry across disjointed systems, increasing administrative burdens and preventing the accurate correlation of agronomic inputs with true financial profitability.

2.1.2 Decision-Support Systems (DSS)

Within agricultural informatics, a Decision-Support System (DSS) does not make isolated operational decisions; rather, it processes heterogeneous metrics—such as weather patterns, soil conditions, market prices, and historical yields—through deterministic rules or statistical and machine-learning models to provide stakeholders with actionable, optimised alternatives (van Klompenburg et al., 2020). By converting complex biological and economic variables into understandable visualisations and predictive estimates, DSS platforms bridge the gap between static farm records and proactive resource management. Recent work demonstrates that such systems can be deployed even for resource-constrained smallholders when the underlying predictive models are deliberately kept inexpensive and mobile-deployable (Olisah, Smith, Smith, Lawrence, & Ojukwu, 2024).

2.1.3 The Shift from Manual to Digital Agriculture (Agriculture 4.0)

The agricultural sector is currently undergoing a disruptive transformation termed “Agriculture 4.0” or digital agriculture. This paradigm shifts farming from intuition-based practices to data-driven precision by integrating the IoT, artificial intelligence, robotics, and big-data analytics. Agriculture 4.0 operates across distinct architectural layers: sensors and IoT devices perform data sensing and connectivity, cloud computing handles large-scale storage, and AI algorithms perform predictive analysis. Because central cloud computing often struggles with latency and high bandwidth requirements in rural areas, modern precision agriculture increasingly utilises Edge Computing, which processes data locally—closer to the farm—thereby reducing latency and mitigating the vulnerabilities of poor rural internet connectivity (Abiri et al., 2023; Dayioğlu & Türker, 2021). Mhlanga and Ndhlovu (2023) do not treat edge computing; their contribution to the present argument is the finding that rural deployment in Africa

must assume degraded rather than absent infrastructure, which is the constraint AGRI-PROFIT answers at the client rather than at the network edge.

2.2 Theoretical Framework

2.2.1 Technology Acceptance and Innovation Systems Theories

Understanding why agricultural practitioners reject or adopt digital tools requires robust theoretical grounding. The literature frequently employs the Diffusion of Innovations (DOI) theory and the Unified Theory of Acceptance and Use of Technology (UTAUT) to explain that adoption depends heavily on a farmer's perception of a technology's relative advantage, usability, and complexity. Empirical FMIS-adoption research operationalises these constructs, showing that education, advisory access, and credit access materially shift the adoption decision (Carrer et al., 2017). Innovation Systems Theory further emphasises that successful digital transformation requires robust collaboration among technology providers, agricultural practitioners, advisory services, and policymakers.

2.2.2 The Theory of Bounded Rationality

Traditional agricultural economics often relies on “rational choice” theory, assuming farmers always act to maximise strict financial utility. However, agent-based modelling of agricultural communities demonstrates that farmers operate under “bounded rationality.” This theory suggests that humans make decisions with limited cognitive and computational capacities; therefore, they seek “satisficing” (satisfactory and sufficient) solutions rather than mathematically optimal ones. Consequently, an effective FMIS must reduce the cognitive load on the farmer by presenting synthesised, clear options rather than overwhelming them with raw, unprocessed big data—a design principle directly reflected in the AGRI-PROFIT decision-support layer.

2.2.3 Socio-Technical Systems Perspective

From a socio-technical perspective, digitalisation fundamentally alters organisational structures, power relations, and rural societal dynamics. Technology adoption does not occur in a vacuum; socio-economic characteristics—such as a farmer’s education, age, and existing infrastructural limitations—directly constrain the successful integration of digital platforms (Mhlanga & Ndhlovu, 2023). Thus, engineering a farm-management platform requires addressing both the technical architecture and the social realities of the target demographic.

2.3 The Nigerian Context: Precision Agriculture and Mobile Scaling

The deployment of digital agriculture in developing regions heavily relies on mobile-mediated scaling. Unlike developed nations where large-scale autonomous machinery drives Agriculture 4.0, mobile phones provide the most accessible and cost-effective leapfrog technology for Sub-Saharan Africa (Mhlanga & Ndhlovu, 2023). Empirical meta-analyses demonstrate that Mobile Phone-Mediated Scaling of Agricultural Technologies (MPSAT) increases the odds of farmers adopting new agricultural practices, particularly when mobile tools are combined with traditional extension services, which can raise adoption odds by approximately 17%. A longitudinal analysis of precision agriculture in Nigeria identifies three phases: the introductory phase (1990–2005), the experimental phase (2006–2015), and the integration and scale-up phase (2016–2023).

However, scaling digital platforms in Nigeria encounters profound infrastructural bottlenecks. Severe deficits in reliable electricity and digital connectivity in rural farming communities critically hinder the deployment of cloud-heavy agritech. To mitigate these realities, localised deployment strategies must prioritise mobile-first, low-bandwidth software integrated with off-grid or solar-powered infrastructure.

Furthermore, national integration policies such as the Food and Agriculture Organization's (FAO) Digital Village Initiative (DVI) in Nigeria champion centralised, ecosystem co-creation platforms to eliminate middlemen and specifically attract youth involvement through digital innovation.

2.4 Empirical Review and Socio-Technical Barriers to Adoption

2.4.1 Review of Existing Systems

Existing literature thoroughly documents the evolution of FMIS and Enterprise Resource Planning (ERP) systems (Fountas et al., 2015). Currently, commercial ERP software effectively handles internal payroll, invoicing, and supply-chain logistics, but such systems predominantly target large-scale, highly capitalised commercial farms (Carrer et al., 2017). In contrast, smallholder and medium-scale farmers frequently rely on simplistic, unstructured digital methods. While these rudimentary tools offer flexibility, they lack automation, restrict interoperability, and frequently succumb to data corruption.

2.4.2 Socio-Technical Barriers to Adoption

Despite the theoretical benefits of digital agriculture, empirical studies highlight several socio-technical barriers that systematically hinder widespread adoption:

- (a) **Interoperability and Data Standards:** A primary technical roadblock is the absence of standardised data-exchange protocols among heterogeneous agricultural devices and software. Because current platforms operate as isolated silos, farmers face difficulties transferring data between applications, which severely restricts scalability (Tummers et al., 2019).

- (b) **Data Fragmentation and Administrative Redundancy:** Operational field records and Farm Financial Accounts (FFA) traditionally occupy separate workflows, forcing farmers to duplicate data entry across disjointed systems and creating analytical inefficiencies (Poppe et al., 2023).
- (c) **User Complexity and Digital Illiteracy:** Complex interfaces and feature-overloaded software deter adoption. A pronounced digital divide persists; GSMA data indicate that 68% of rural Nigerians lack smartphone ownership, compounded by systemic digital illiteracy and high inflation (Jaiyeola, 2023; Mhlanga & Ndhlovu, 2023).
- (d) **Infrastructural and Economic Constraints:** High initial capital costs, unstable electricity, and limited internet connectivity in rural areas render many existing commercial FMIS platforms impractical for resource-constrained smallholders (Mhlanga & Ndhlovu, 2023).

2.5 Data-Sharing and Stakeholder Trust

A critical, often neglected aspect of farm management is the macro-economic necessity of generating verifiable data to secure financing. Historically, the use of ICT could not inherently prevent bias or manipulation in self-reported farm data, leading to a profound lack of trust from agricultural banks, investors, and insurance providers (Gebresenbet et al., 2023).

To overcome this, modern architectures increasingly integrate blockchain technology as a secure data-governance mechanism (Xiong et al., 2020). Blockchain utilises an immutable, distributed digital ledger to store yield data, mechanisation logs, and financial metrics. By locking operational and financial records into a tamper-proof framework, the system provides a constant flow of credible, unaltered data.

Consequently, this cryptographically secured transparency allows financial institutions and supply-chain partners to deploy smart contracts, automate insurance pay-outs, and accurately assess agricultural credit risk, fundamentally bridging the gap between rural producers and formal credit networks (Xiong et al., 2020). Within the scope of the present study, however, AGRI-PROFIT operationalises this trust requirement through a more immediately deployable mechanism: authenticated, access-controlled, tokenised standardised reporting. A per-farm data boundary restricts each enterprise's records to its owner, while read-only, time-limited views expose standardised profit/loss and yield summaries to invited banks or investors. Full distributed-ledger governance is identified as a future extension that can layer tamper-evidence onto this foundation once the core platform is validated.

2.6 Structuring the Decision-Support Prediction Model for AGRI-PROFIT

Building on the conceptual review of decision-support systems (Section 2.1.2), this section establishes the methodological structure that AGRI-PROFIT will follow to convert unified operational and financial records into predictive decision support. The structure is organised as a layered pipeline, deliberately progressing from lightweight deterministic logic to optional data-driven models so that the platform remains viable on accessible mobile processors and under intermittent connectivity (van Klompenburg et al., 2020; Olisah et al., 2024).

2.6.1 Data Layer and Feature Definition

The prediction pipeline begins with the unified data layer that distinguishes AGRI-PROFIT from siloed systems. Operational records (planting and harvest dates, input applications, labour hours, mechanisation usage, and observed yields) and financial records (input costs, sales revenue, and overheads) are automatically and atomically paired at the point of entry—each operation linked to its corresponding financial

transaction through a shared activity category—rather than maintained as isolated ledgers. This pairing, enforced at the write layer, is what allows operational activity and its financial consequence to be analysed together. From these records, candidate predictor variables are defined. The current release uses four: crop type, together with three agronomic conditions supplied at the point of forecast — rainfall, fertiliser applied and soil pH. A wider feature set is specified here as the intended direction of the data layer rather than as a delivered one: cultivated area, season, machine-hours, labour-hours, fuel, location and soil class are not captured by the operational log in this release and are therefore not available to the model. The prediction target in the current release is crop yield alone. Gross margin and profit or loss are not predicted at all; they are computed deterministically from recorded transactions by the Tier 1 engine described below, which is a stronger guarantee than a prediction rather than a weaker one. Recommended input allocation is identified as future work and depends on the wider feature set.

2.6.2 Data Pre-processing

Because smallholder records are typically sparse and inconsistent, a pre-processing stage is essential before any model is trained or applied. This stage handles missing values through imputation, harmonises units and date intervals, removes obvious outliers, and normalises numeric ranges. Olisah et al. (2024) demonstrate that a carefully designed pre-processing pipeline—addressing inconsistent year intervals, missing data, and variables inaccessible to smallholders—is decisive for prediction quality in Sub-Saharan farming contexts. AGRI-PROFIT adopts this as a design principle for real-farm data; in the current release, in which the forecast tier is trained on clean synthetic data, pre-processing is limited to categorical encoding and pass through

of numeric features, with the fuller pipeline described here applying as farms accumulate real records.

2.6.3 A Tiered Modelling Strategy

Rather than committing to a single algorithm, AGRI-PROFIT adopts a tiered modelling strategy that aligns model complexity with available data and device capability:

- (a) **Tier 1 – Deterministic, rule-based engine (baseline).** Transparent financial rules compute, directly from the records, per-crop gross margin and its ranking, unit cost of production on a harvested and on a marketable basis, retrospective break-even yield, and — through the enterprise-economics layer — break-even price on a cash and a total-cost basis. Input thresholds are not computed and are not claimed. This tier requires no training data and is fully explainable, satisfying the bounded-rationality requirement for clear, satisficing recommendations (Section 2.2.2). The computation itself is performed server-side on modest hosting; what survives a loss of connectivity is the readability of its most recent results, which the client holds in a service-worker cache. The tier is therefore available offline, and it does not execute on the device.
- (b) **Tier 2 – Statistical regression (interpretable prediction).** Once sufficient historical records accumulate, multiple linear or regularised regression estimates yield and profitability as functions of the defined features, quantifying the contribution of each input to the outcome.
- (c) **Tier 3 – Lightweight machine learning (optional, server-assisted).** Where data volume permits, tree-based ensembles such as Random Forest and Extreme Gradient Boosting (XGBoost) are indicated, as they consistently achieve strong accuracy on tabular agricultural data at far lower computational cost than deep

neural networks (van Klompenburg et al., 2020). The counter-case must be stated: for a Nigerian corn-yield decision-support task, Olisah et al. (2024) report that a carefully tuned deep neural network regressor outperformed the models they compared it against, and that the quality of data pre-processing rather than the model family was decisive. That work is an unpublished preprint, and it is cited in this study for its pre-processing finding and its Sub-Saharan framing rather than as authority for a model choice. The selection of a tree ensemble in the present work is therefore justified on grounds of computational cost and interpretability under the resource constraint of Section 2.2.2, and not on a claim of superior predictive accuracy.

This progression describes a general methodological ladder drawn from the literature. The AGRI-PROFIT platform realises two of its rungs as its implemented two-tier design: the deterministic rule-based tier (Tier 1) and the tree-based Random Forest tier (Tier 3), with the intermediate regression tier (Tier 2) available as an optional future refinement. This ensures the platform delivers value from day one through the deterministic tier, while providing a clear, evidence-based pathway to predictive analytics as the user's data matures.

2.6.4 Model Training, Validation, and Evaluation Metrics

For the statistical and machine-learning tiers, data are partitioned into training and testing sets using a hold-out split, guarding against over-fitting on small datasets (van Klompenburg et al., 2020). In the current implementation, continuous predictions (yield) are evaluated using the coefficient of determination (R^2) and the Mean Absolute Error (MAE); the Root Mean Square Error (RMSE) is a standard complementary measure for such tasks and is additionally reported, and Olisah et al. (2024) propose combining RMSE and MAE into a single robustness measure. For the rule-based tier,

evaluation compares system-computed margins against farmers' actual recorded outcomes to confirm arithmetic fidelity.

2.6.5 From Prediction to Decision Support

The final stage translates computed and predicted values into decision support. The deterministic outputs are presented per crop and ranked by gross margin, so that the least profitable enterprise is visible without searching for it, alongside unit cost of production and the retrospective break-even yield at which each crop would have covered its recorded costs. Two further steps are specified here as the intended direction and are not implemented in the current release: converting these figures into a ranked option set over alternative input mixes, and raising a threshold-based warning as projected cost approaches the break-even point. The reduction of a full ledger to a small number of ranked, satisficing figures is nevertheless the mechanism by which cognitive load is lowered (Section 2.2.2), and the same structured, standardised outputs feed the trusted data-sharing layer described in Section 2.5 (Xiong et al., 2020). Deployment remains mobile-first and low-bandwidth: the deterministic tier requires no training data and runs on modest server hosting, with results delivered to the mobile client, while the optional forecast tier likewise runs server-side and its outputs are synchronised when connectivity is available (Mhlanga & Ndhlovu, 2023).

2.7 Post-Harvest Drying Kinetics and the Costing of Marketable Output

The literature reviewed so far treats the farm record as an accounting and decision-support artefact. A second body of literature, largely disjoint from it, treats what happens to the harvest after it leaves the field. That body is reviewed here because the platform designed in this work implements a post-harvest drying module, and because the quantity a farmer actually sells is not the quantity harvested.

2.7.1 Thin-Layer Drying Models

Drying is the controlling unit operation in cereal and legume post-harvest handling: grain that enters storage above its safe-storage moisture content supports the storage fungi whose growth threshold is an equilibrium relative humidity of about 65%, corresponding to roughly 13% moisture content in cereal grain, and the Food and Agriculture Organization's post-harvest guidance accordingly specifies drying maize to approximately 13% before storage (Food and Agriculture Organization of the United Nations [FAO], n.d.). Drying is therefore not an optional refinement but the operation that determines whether a harvest survives to be sold.

The kinetics of that operation are conventionally described by thin-layer models, which express the dimensionless moisture ratio of a product as a function of drying time. Three are long established and remain the reference forms. The Newton, or Lewis, model, $MR = \exp(-kt)$, treats drying as a first-order process governed by a single rate constant (Lewis, 1921). The Page model, $MR = \exp(-k \cdot t^n)$, introduces an empirical exponent that accommodates the deviation from first-order behaviour observed in most biological materials (Page, 1949). The Henderson and Pabis model adds a pre-exponential coefficient derived from the first term of the analytical solution to Fick's second law (Henderson & Pabis, 1961). Onwude, Hashim, Janius, Nawi, and Abdan (2016), in a review of thin-layer modelling of fruits and vegetables, find that semi-theoretical forms of this kind dominate the applied literature precisely because they are fitted from a small number of time-and-mass observations that a practitioner can actually take, and Erbay and Icier (2010) reach the same conclusion for foods generally. Model parameters are recovered by regression on a linearising transform — for the Page model, the double logarithm $\ln(-\ln MR) = \ln k + n \cdot \ln t$ — which makes the fit reproducible and inspectable rather than opaque.

Two quantities must be kept distinct in any such analysis, and conflating them is the characteristic error of the domain. The mass of water removed is a water balance computed from the moisture contents and the dry-matter mass, and it is the intended outcome of the process; the difference between inlet and outlet mass is a mass balance that also contains any solid matter lost to spillage, handling or over-drying, and it is a process loss. Wet-basis and dry-basis expressions of moisture content are likewise not interchangeable, and the dry-matter balance — dry matter being conserved through drying — is what allows a predicted outlet mass to be computed from an initial mass, an initial moisture and a target moisture.

2.7.2 The Post-Harvest Context in Africa, and the Costing Gap

Baidhe, Clementson, Senyah, and Hammed (2024), appraising post-harvest drying and storage operations across Africa, report that inadequate and poorly controlled drying remains a principal determinant of grain quality loss on the continent, and that smallholder practice is dominated by open sun drying whose end point is judged by eye rather than measured. Their finding matters for the present work in a specific way: where the end point is unmeasured, the mass that reaches the store is unknown, and a farmer who has not measured it has no way to know the quantity against which their production costs should be divided.

This is the junction at which the two literatures reviewed in this chapter fail to meet. The drying literature models moisture and mass and stops at the store door; it does not carry its outputs into a cost record. The farm management information systems literature reviewed in Sections 2.1 and 2.4 models the cost record and stops at the field boundary; the marketable mass it divides by is the harvested mass, because that is the only mass its data model holds. Neither body of work treats post-harvest mass reduction as a financial quantity. Enterprise budgeting in agricultural economics supplies the missing

vocabulary — the classification of cost into variable, fixed and semi-variable behaviour, and the computation of unit cost of production and break-even price against a defined output quantity (Kay, Edwards, & Duffy, 2016) — but it assumes the output quantity is given. The contribution of the post-harvest module designed in Chapter Three and evaluated in Chapter Four is that junction: the thin-layer models themselves are long established and are not claimed as novel, but their coupling to the paired financial record, so that the denominator of the unit-cost calculation is the mass that can actually be sold, is not found in the reviewed literature. Objective (v) of Section 1.3 is stated in exactly those terms.

2.8 Gap in Literature

The systematic review of the literature reveals a pronounced bias in the design and deployment of existing FMIS and digital agricultural tools. Most commercially available platforms cater to the operational scale, financial capital, and robust broadband infrastructure typical of developed nations (Carrer et al., 2017; Fountas et al., 2015). Consequently, there is a severe lack of lightweight, localised frameworks tailored specifically for the socio-technical and infrastructural realities of Nigerian agriculture, which remains dominated by smallholders and medium-scale enterprises facing distinct infrastructural deficits (Mhlanga & Ndhlovu, 2023).

Furthermore, while studies acknowledge the chronic problem of data fragmentation between agricultural operations and financial accounting (Poppe et al., 2023), there is a critical scarcity of conceptual system architectures that successfully integrate these disparate data streams within a low-bandwidth, mobile-first platform and convert them into appropriately lightweight predictive decision support (van Klompenburg et al., 2020; Olisah et al., 2024). Finally, current tools focus almost exclusively on internal farm management, neglecting the macro-economic necessity of secure data-sharing

protocols—such as blockchain—to build operational trust with agricultural banks and investors (Xiong et al., 2020). A fourth gap, identified in Section 2.7, is that the thin-layer drying literature and the farm-record literature do not meet: post-harvest mass reduction is modelled as a process quantity and costed nowhere, so the unit cost of production reported by every reviewed system is computed against a mass the farmer no longer possesses (Onwude et al., 2016; Baidhe et al., 2024; Kay et al., 2016). The AGRI-PROFIT platform directly addresses these identified gaps by engineering a unified, socio-technically appropriate system architecture for the Nigerian agricultural sector in which the post-harvest process record and the financial record are the same record.

CHAPTER THREE

3.0 SYSTEM DESIGN AND METHODOLOGY

3.1 Introduction

This chapter presents the design and development methodology adopted for the AGRI-PROFIT platform and the methods used to evaluate it against the aim and objectives established in Chapter One. It describes the research and development approach, the requirements analysis, the overall system architecture, the unified data model, the design of each functional module, the security design, and the evaluation methodology. The chapter is written to describe the target architecture of the platform—that is, the complete system the design specifies—while stating explicitly, where relevant, which capabilities constitute the current release and which are deferred as future work. This distinction is maintained throughout so that the methodology remains an honest account of the system as designed and built.

The four objectives of the study are addressed by distinct but connected methods. Objective 1 (identifying socio-technical barriers) is addressed through the requirements analysis and through the inclusive design of low-barrier data-entry channels. Objective 2 (translating records into decision-support metrics) is addressed through the unified data model and the deterministic decision-support tier. Objective 3 (secure, trusted data-sharing) is addressed through the authentication and per-farm boundary design and the tokenised stakeholder-reporting mechanism. Objective 4 (evaluation of usability, cost-effectiveness, and viability) is addressed through the multi-strand evaluation methodology presented in Section 3.8.

3.2 Research and Development Methodology

The platform was developed using an iterative, incremental methodology consistent with Rapid Application Development (RAD) principles, which are well suited to

systems whose requirements are refined through successive prototypes rather than fixed entirely in advance. Development proceeded in short cycles, each producing a working increment that was verified before the next was undertaken. Version control (Git) was used throughout, with each functional increment committed as a discrete, reversible checkpoint, allowing safe experimentation and providing a traceable development history.

Development was assisted by a command-line AI coding agent operating within the project repository, guided by a curated set of engineering skills. The agent produced code increments against explicit, human-reviewed specifications; each increment was inspected, tested, and verified by the researcher before acceptance. This human-in-the-loop arrangement preserved the researcher's authorship of design decisions while accelerating implementation. Two governing documents—a domain context glossary and a product requirements document—were maintained under version control to keep terminology, scope, and implementation consistent across the codebase and the written work.

3.3 System Requirements Analysis

Requirements were derived from the problem analysis in Chapters One and Two and organised into functional and non-functional categories. The non-functional requirements are of particular importance because they encode the socio-technical constraints that this study exists to address (Objective 1).

3.3.1 Functional Requirements

The platform is required to: record farm operations and their associated costs as paired entries; refuse the deletion of a posted record and provide correction by a category-preserving contra entry linked to its original; compute profit-and-loss and gross-margin

metrics from those records; track equipment, maintenance and depreciation; produce deterministic decision-support metrics — per-crop gross margin and ranking, unit cost of production on a harvested and a marketable basis, and retrospective break-even yield — from real records; classify recorded cost by economic behaviour and derive from it a fixed-cost overlay, dual break-even prices, a yield-sensitivity matrix, an operating-expense ratio, a partial budget and an Olympic-average yield baseline; model post-harvest drying from recorded mass and moisture observations and carry its marketable output into the unit-cost calculation; provide an optional yield forecast; authenticate users, authorise each request against a named permission, and confine each user's data to their own farm; and allow a farmer to grant a bank or investor a revocable, expiring, read-only view of standardised reports.

3.3.2 Non-Functional Requirements

- (a) **Low-bandwidth, mobile-first operation:** the interface must be usable on affordable smartphones over constrained networks.
- (b) **Offline data capture:** operations must be recordable without connectivity and synchronised, without loss or duplication, once a connection returns. The requirement is scoped to the write path; offline reading is a separate and weaker guarantee, met by serving previously fetched responses for a defined set of read endpoints from a service-worker cache. No claim is made that all writes, or all reads, work offline.
- (c) **Security and data isolation:** access must be authenticated and each farm's data isolated from every other.
- (d) **Accessibility for low-specification devices and feature phones:** data entry should be possible through channels that do not presuppose a smartphone or a

reliable data connection. This requirement is stated because the barrier it answers is real and quantified in Chapter One, and it is recorded here as a requirement that the present release does not meet. No feature-phone entry channel is implemented; the design for one is given in Section 3.6.8 and it is future work, not delivered work.

- (e) **Computational feasibility:** decision-support processing must run within the resources of accessible mobile processors and modest server hosting.

3.4 System Architecture

3.4.1 Overall Architecture

AGRI-PROFIT adopts a client–server architecture comprising a mobile-first Progressive Web Application (PWA) client and a backend application-programming-interface (API) service. The PWA client is installable on a smartphone and is responsible for data entry, local queuing of records while offline, and presentation of reports and decision-support outputs. The backend exposes a thin HTTP interface over a layered structure in which business logic and database access are separated from the transport layer, promoting testability and maintainability. Data are persisted in a relational database whose schema is managed through versioned migrations.

The arrangement is shown in Figure 3.1. The three tiers correspond to the three containers of the deployment composition, and the external access path at the foot of the figure is the mechanism of Section 3.6.7 by which a lender reaches a report without holding an account.

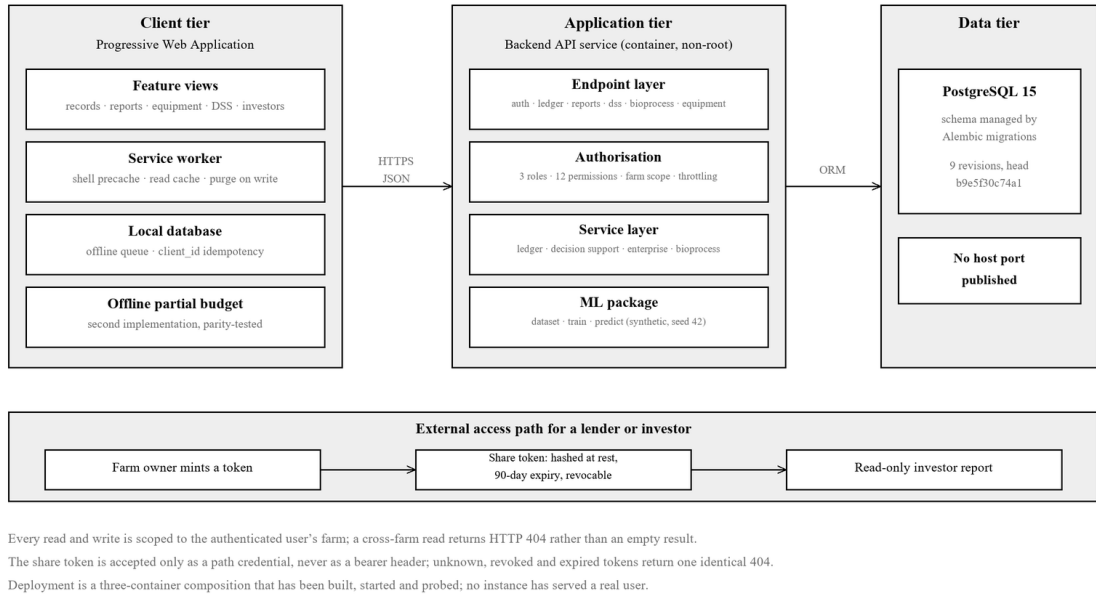


Figure 3.1: AGRI-PROFIT system architecture

3.4.2 Technology Stack

The client is implemented with a modern component-based JavaScript framework compiled to a lightweight bundle, with a local browser database used to queue records for offline-first capture and a service worker providing the precache and the read cache. The backend is implemented in Python using an asynchronous-capable web framework with an object-relational mapper and a migration tool for schema evolution; the route handlers themselves are written synchronously and are executed by the framework in a worker thread pool, which is stated plainly because the platform makes no use of cooperative concurrency and should not be described as though it did. These choices favour a small deployment footprint suitable for modest, low-cost hosting, in keeping with the cost-effectiveness objective.

3.5 Data Model Design

The core design decision of AGRI-PROFIT is the elimination of the historical separation between operational records and financial accounts. Rather than maintaining

two independent ledgers, the data model pairs each operation with its financial consequence at the point of entry. An Operational Log (a timestamped record of a physical farm activity) is automatically and atomically paired with a corresponding Financial Transaction, the two being linked through a shared Activity Category. This pairing is enforced at the write layer of the application, so that in normal operation every recorded activity carries its financial effect with it.

The financial ledger is single-entry: each Financial Transaction is classified as either an expense (debit) or revenue (credit) and tagged with an Activity Category, without an enforced double-entry debit/credit pairing. This is a deliberate simplification appropriate to the smallholder context and to the analytical purpose of the platform, and it is stated as such to avoid over-representing the accounting model. A second property of the ledger is equally deliberate and is central to Objective 3: a posted record is soft-immutable. Deletion is refused at the transport layer, and a mistake is corrected by posting a contra entry that preserves the original's Activity Category and crop and carries a reference to the record it reverses. Both rows then remain visible, the correction is netted into every derived figure, and a second reversal of the same record is rejected. The paired-record design is what allows operational activity and its financial outcome to be analysed together, and it is the structural foundation on which the decision-support tier depends.

The principal entities of the data model are summarised in Table 3.1, and their relationships are shown in Figure 3.2. Two features of that diagram carry the design argument of this section: the one-to-one edge between `OperationalLog` and `FinancialTransaction`, which is enforced at the write layer rather than by a database constraint, and the self-reference on `FinancialTransaction` through which a contra entry is linked to the record it corrects.

Table 3.1: Principal entities of the AGRI-PROFIT data model

Entity	Role in the model
Operational Log	Timestamped physical farm activity; the primary unit of operational entry.
Financial Transaction	Expense or revenue record paired with an Operational Log via a shared Activity Category.
Equipment	Machinery tracked for acquisition cost, depreciation, and maintenance.
Maintenance Log	A service or repair event recorded against a specific Equipment item.
Farm / User	Ownership boundary; scopes all records to a single enterprise (security design).
Share Token	A revocable, expiring, read-only grant of access to one farm's standardised reports; stored only as a hash, never in clear.
Reversal link (reverses_id)	A self-reference on Financial Transaction identifying the record a contra entry corrects; the mechanism by which the ledger is corrected without deletion.
Crop attribute	A free-text crop label carried on the Operational Log and inherited by its paired Financial Transaction; the grouping key for all per-crop decision support.
Client identifier (client_id)	A client-generated identifier carried by every queued record, giving the offline synchronisation path its idempotency.
Drying run attributes	Mass in and out, initial and final moisture content, drying method and optional time-and-moisture readings, recorded on an Operational Log of the post-harvest category.

Entity	Role in the model
Role and permissions	The authorisation attributes on User; each request is checked against a named permission before any farm-scope check is applied.

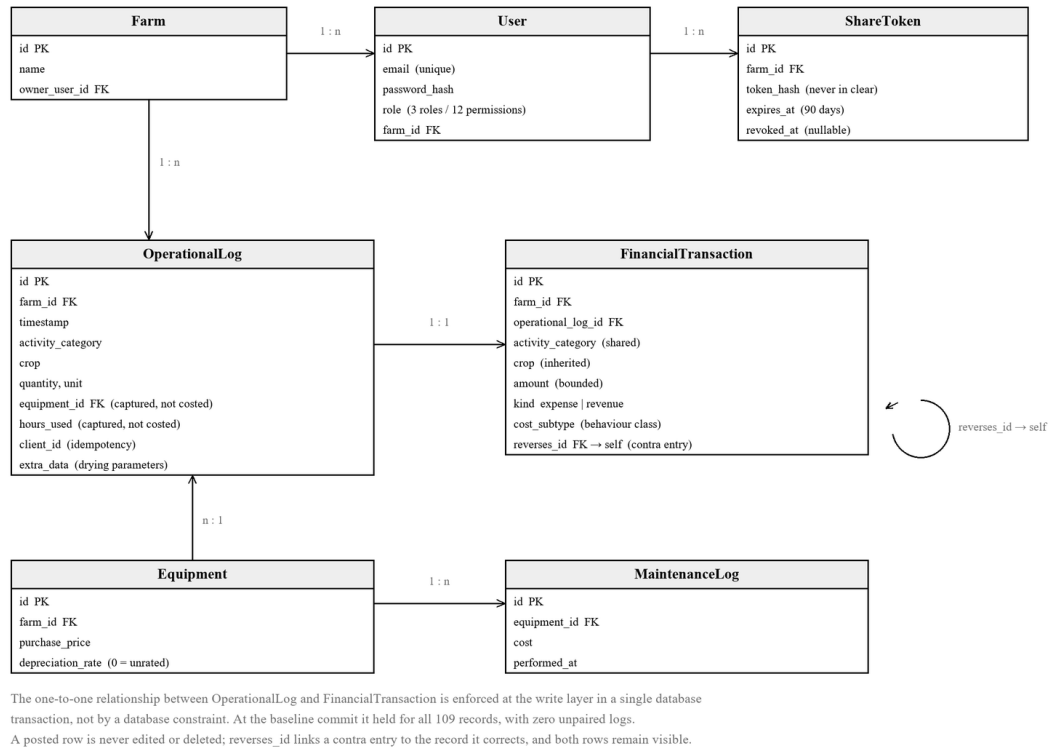


Figure 3.2: Entity-relationship diagram of the AGRI-PROFIT data model

3.6 Module Design

The platform is organised into cohesive modules, each with a defined responsibility. The design of each is described below; where a module is partially realised in the current release, the boundary between built and deferred behaviour is stated explicitly.

3.6.1 Offline Operational Logbook

This module handles data entry for operations, yields, and post-harvest drying runs. To satisfy the offline-capture requirement, records entered without connectivity are queued in a local browser database and synchronised to the backend when a connection is

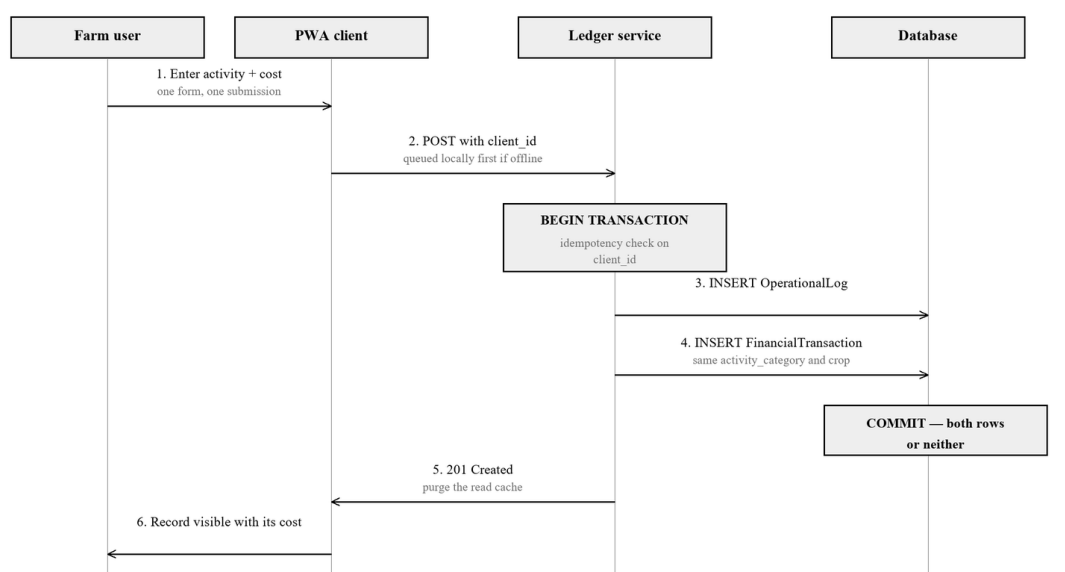
restored. Synchronisation is idempotent: each queued record carries a client-generated identifier, so that replaying the same record—for example after a connection is lost mid-entry—results in a single stored record rather than a duplicate. The queue is partitioned by identity and cleared on logout, so one user’s pending records cannot be flushed under another’s session. Offline reading is also implemented, and is described here as built rather than as planned: a service worker precaches the application shell and serves a stale-while-revalidate cache over the ledger, reporting and five named decision-support read endpoints, bounded at sixty-four entries and seven days. The cache is purged on every farm-scoped write and on any change of authenticated identity, because a caching layer that survives a correction would make the application misreport its own state. Two endpoints — the model-information and prediction routes — are deliberately excluded from the cache, since a forecast served from cache would be indistinguishable from a fresh one.

3.6.2 Financial Ledger Engine

This module is the translation layer that creates the paired Financial Transaction for each Operational Log, applying the shared Activity Category, the crop label and an optional tax label, and committing both rows in a single database transaction so that neither can exist without the other. It also enforces the correction discipline described in Section 3.5: deletion of a posted record is refused, a contra entry is created with the original’s category and crop and a link to the record it reverses, a second reversal of the same record is rejected, and a reversal request naming a record belonging to another farm is refused on the same path as any other cross-farm access. Monetary and quantity inputs are bounded at the schema edge, so that a negative, absurd or non-finite value is rejected before it can reach the ledger. The module’s correctness is central to the

platform, since all downstream reporting and decision support derive from the ledger it maintains.

Figure 3.3 traces a single record creation through the module. The property the sequence exists to make inspectable is that both rows are written inside one database transaction, so that neither can be committed without the other, and that the correction path shown beneath it never removes a row.



Correction path: a posted record is never edited or deleted. DELETE returns HTTP 405. A correction posts a contra entry carrying the original's activity category and crop and a reverses_id pointing at it; both rows remain visible and every derived figure nets the pair. A second reversal of the same record is rejected, as is a reversal naming a record belonging to another farm.

Figure 3.3: The paired write — sequence of a single record creation

3.6.3 Reporting

The reporting module aggregates the ledger into profit-and-loss statements—overall, per-category, and monthly—and produces a standardised export in comma-separated-values (CSV) format. CSV is the only export format implemented; no PDF or spreadsheet export exists, and none is claimed. This export provides the standardised-reporting basis on which the stakeholder-sharing mechanism (Section 3.6.7) is built.

3.6.4 Mechanisation and Equipment Tracker

This module records equipment acquisition cost, maintenance events and a straight-line depreciation rate, and records mechanisation usage and fuel as operational entries tagged with the mechanisation activity category, so that machinery costs are captured within the ledger alongside other activities. An equipment record can be corrected after entry through a partial-update path, so that a mistyped depreciation rate is not permanent. Depreciation is not posted to the ledger: it is computed as a derived overlay at report time and allocated to crops in proportion to direct cost, because posting a synthetic depreciation row would require either an unpaired financial entry, which violates the paired-write invariant, or a fabricated operational log describing an event that did not happen. A per-operation association between an individual activity and a specific equipment item, together with the hours that item was used, is captured and validated at the point of entry but is consumed by no service: the platform produces no machine-hour, utilisation or cost-per-hour figure, and none appears anywhere in this work. The boundary is stated as captured but not costed, and the reasoning is recorded in the project's third architecture decision record.

3.6.5 Predictive Decision-Support Engine

The decision-support engine follows a two-tier design that deliberately aligns model complexity with available data and device capability (van Klompenburg, Kassahun, & Catal, 2020; Olisah, Smith, Smith, Lawrence, & Ojukwu, 2024).

- (a) Tier 1 – Deterministic rule-based tier. Transparent rules compute, directly from the farm's own Operational Logs and Financial Transactions grouped by Activity Category and crop, nine quantities: per-crop gross margin and its ranking; unit cost of production on a harvested-mass basis; unit cost of production on a marketable-mass basis where a drying run exists; retrospective

break-even yield; cost structure classified by economic behaviour with an explicit classification-coverage figure; a derived fixed-cost overlay and its proportional allocation; break-even price on a cash and on a total-cost basis; a yield-sensitivity matrix; and an operating-expense ratio, together with a partial-budget calculator and an Olympic-average yield baseline. This tier requires no training data, runs on modest server hosting, and is fully explainable, providing clear, satisficing decision support consistent with the bounded-rationality principle discussed in Chapter Two. Its governing design rule is that a figure is withheld, with a stated reason, wherever its inputs do not support one, rather than defaulted to a plausible value.

- (b) **Tier 2 – Statistical yield forecast (optional).** A Random Forest regressor produces a yield estimate from agronomic inputs such as rainfall, fertiliser, and soil pH, reporting error measures suited to regression tasks.

An explicit disclosure governs the forecast tier: in the current release, the Random Forest model is trained on representative synthetic data rather than on any individual farm's own records, and it does not learn or adapt from farm usage. This design choice reflects the reality that a newly deployed farm has insufficient historical data to train a reliable model; retraining on real accumulated records is identified as future work. The deterministic tier, by contrast, operates entirely on the farm's real records. This distinction is stated plainly to ensure the platform's analytical claims are not overstated.

For the statistical tier, model quality is expressed using the coefficient of determination (R^2), the Mean Absolute Error (MAE) and the Root Mean Square Error (RMSE), the last being a standard complementary measure for continuous prediction tasks in agricultural yield modelling; all three are reported in Section 4.9 (van Klompenburg et

al., 2020; Olisah et al., 2024). The training pipeline is seed-deterministic and the dataset carries a recorded cryptographic fingerprint, so that the reported figures can be re-derived rather than taken on trust.

3.6.6 Authentication and Per-Farm Data Boundary

To satisfy the security requirement embedded in the aim and in Objective 3, the platform authenticates users and confines each user's records to their own farm. Authentication issues a signed token upon login, with passwords stored only in hashed form. Every read and write operation across the ledger, reporting, equipment, and decision-support paths is scoped to the authenticated user's farm, so that one enterprise's data cannot be reached by another. The farm/user boundary is introduced through a versioned database migration rather than ad-hoc schema creation, preserving the integrity of the managed schema.

3.6.7 Tokenised Stakeholder Reporting

The trust-building element of Objective 3 is realised through a tokenised, read-only sharing mechanism. A farm owner may mint an opaque token that grants a bank or investor read-only access to that farm's standardised profit-and-loss and yield summaries, without the recipient holding an account. Four properties of the design are load-bearing. The token is stored only as a hash, so a database disclosure does not yield a usable credential. It expires on a fixed ninety-day horizon as well as being revocable at will, so a link that is forgotten rather than revoked does not remain live indefinitely. It is accepted only as a path credential and not as a bearer authorisation header, so it cannot be replayed against the authenticated interface. And a token that is unknown, revoked or expired resolves to one identical not-found response, so the mechanism cannot be used as an oracle to discover which tokens ever existed. The token cannot be used to write data or to reach any other farm. Within the scope of this study, this

authenticated, access-controlled, tokenised reporting is the mechanism by which standardised data is shared with financial stakeholders; immutable distributed-ledger (blockchain) governance, reviewed in Chapter Two, is designed as a future extension that can layer cryptographic tamper-evidence onto this foundation.

3.6.8 Feature-Phone (USSD/SMS) Data Entry — Designed, Not Implemented

The socio-technical barrier at the heart of Objective 1 is that a large proportion of rural Nigerian farmers do not hold a smartphone. The design response to it is an inbound message channel through which a farm activity could be recorded from a feature phone. This section states that design, and states equally plainly that none of it is built. The intended mechanism is an inbound webhook accepting a message consisting of a sender identifier and a short structured text, a parser converting that text into an activity and an amount, a mapping from the sender's number to a registered farm so that unregistered senders are rejected, and creation of the paired Operational Log and Financial Transaction through the same ledger service the application uses, so that a phone-entered record would be indistinguishable from an app-entered one. The channel would be gateway-agnostic, so that it could be exercised against simulated inbound payloads before any carrier integration. In the release evaluated by this thesis there is no such route, no webhook, no parser, no sender-to-farm mapping, no simulated harness and no test; the navigation entries that once gestured at this channel were removed because no backend supported them. The channel is therefore recorded in Section 5.3 as future work, and no result anywhere in Chapter Four or Chapter Five rests on it. Its absence is the principal limit on the reach of the contribution and is stated as such rather than qualified.

3.6.9 Post-Harvest Drying Module

This module answers Objective (v) and carries the bioprocess engineering content of the platform. It is named the bioprocess module in the source code and its endpoints sit under a bioprocess path; the two names denote the same subsystem and the design term used throughout this thesis is the post-harvest drying module.

A drying run is entered as an Operational Log of the post-harvest category and therefore posts its cost through the same paired write as any other activity. Its own parameters are the mass and moisture content at inlet, the mass and moisture content at outlet, the drying method, and an optional series of time-and-moisture readings taken during the run. From these the module computes, by the relationships reviewed in Section 2.7: conversion between wet-basis and dry-basis moisture content; a dry-matter balance yielding the outlet mass the run should have produced; a water balance giving the mass of water actually removed, held deliberately distinct from the mass difference so that process loss and drying performance are never the same number; a mean drying rate; a safe-storage verdict against a recorded threshold for the crop, returned as an absence rather than as a negative verdict where no threshold is recorded; and, where at least three readings exist, a least-squares fit of the Newton and Page models on their linearising transforms, returning the rate constant k and, for the Page model, the exponent n .

The design decision that matters financially is which mass becomes the denominator of the unit-cost calculation. The module uses the mass actually weighed out, not the mass the dry-matter balance predicts, because costing against a predicted mass would charge the farmer's pricing decision with grain that never reached the store. Every malformed submission — outlet mass exceeding inlet mass, a final moisture not below the initial, non-positive mass, moisture outside the physical range, or readings not in strictly

increasing time order — is rejected at the schema edge, and the same rules are mirrored in the client so that an invalid run is refused before it is queued offline rather than retrying forever.

3.6.10 Enterprise Economics Module

This module extends the deterministic tier from margin accounting to cost-behaviour analysis, and supplies the vocabulary of enterprise budgeting reviewed in Section 2.7.2 (Kay, Edwards, & Duffy, 2016). Recorded cost is classified at the schema edge into variable, semi-variable and fixed behaviour; the proportion of cost that carries a subtype is reported as an explicit classification-coverage figure, and the residue is reported as unclassified rather than distributed into a plausible bucket, so that a reader can see how much of the analysis rests on classified data. On that base the module derives the fixed-cost overlay of Section 3.6.4 and allocates it in proportion to direct cost; computes a break-even price on a cash basis and a total-cost basis, the two answering different questions and the cash price provably the lower; computes a yield-sensitivity matrix, labelled conditional because every row but the baseline describes a yield that did not occur; computes an operating-expense ratio that excludes all non-cash charges; and computes an Olympic-average yield baseline, which discards the highest and lowest season before averaging and is refused, with the season count stated, below three seasons. A partial-budget calculator evaluates a proposed change by four terms and is implemented twice — once server-side and once in the client, so that it survives offline — with both implementations asserted against a single shared hand-computed fixture.

3.7 Security Design

Security is treated as a first-class requirement because the word “secure” appears in the aim and because Objective 3 depends on stakeholders trusting the platform’s outputs. The design rests on six elements. Authentication issues a signed token over credentials

stored only as a hash. Authorisation is role-based: three roles are defined over a table of twelve named permissions, and every request is checked against the permission it requires before any farm-scope check is applied, so that a refusal reports a missing permission and never doubles as an oracle confirming that a record exists. Data isolation confines every read and write on every path to the authenticated user's farm, a cross-farm read resolving to a not-found response rather than to an empty result. Failed logins are throttled on a fixed budget within a rolling window, the refusal naming no account so that it confirms no email address; the counters are held in process, which means they neither span replicas nor survive a restart, and that residual is named here rather than concealed behind the control. Credential hygiene requires that a share token never appears in any log: the reverse proxy redacts it and the application replaces it with a stable, non-reversible fingerprint that still allows requests to be correlated. Finally, external sharing uses the scoped, revocable, expiring, hash-stored, read-only tokens of Section 3.6.7, which cannot escalate to write access or cross a farm boundary. Together these ensure that data sharing is deliberate and controlled rather than open, which is the precondition for the trust that Objective 3 seeks to establish. What the design does not provide is stated with equal clarity: there is no session revocation within a token's lifetime, no refresh mechanism, no password reset, no second factor, and no independent security assessment of any of it.

3.8 Evaluation Methodology

Objective 4 requires an empirical evaluation of the platform's usability, cost-effectiveness and operational viability under Nigerian constraints. A full field deployment with a large sample of smallholder farmers represents the ideal evaluation; however, such a deployment fell outside the time and access constraints of this study and is therefore presented as designed future work (Section 3.8.5). The evaluation

specified for this study comprises four complementary strands, each producing evidence that can be reported in the results chapter. A distinction must be drawn here and held throughout: this section states the evaluation as specified, and Chapter Four reports which strands were executed. Three of the four were executed. The usability strand of Section 3.8.4 was specified and its instruments were prepared, and it was not carried out; no usability figure of any kind is reported in this work, and Objective 4 is consequently reported as partly met.

3.8.1 Functional Verification

Each functional requirement is verified against defined test cases using an automated test suite, supplemented by manual walkthroughs of the core workflows. For the deterministic decision-support tier, verification asserts that the computed unit cost of production and gross margin match values calculated by hand from seeded ledger records, confirming arithmetic fidelity. For the offline path, a test confirms that a queued record synchronises exactly once and that replaying the same client identifier yields a single stored record. The outcome of this strand is a requirement-by-requirement statement of pass or fail with the corresponding test evidence.

3.8.2 Performance Benchmarking

Performance under constrained conditions is measured using an automated auditing tool (Lighthouse) run against the production build of the application, with device emulation set to a mobile profile and network throttling applied to approximate a slow mobile connection. The reported measures are the overall performance score together with load-timing metrics—First Contentful Paint, Largest Contentful Paint, Total Blocking Time, and Speed Index—and the transferred bundle size. To reduce noise, the audit is run several times and the median reported. This strand directly evidences the low-

bandwidth claim, and any performance limitation it reveals (for example, the cost of charting libraries) is reported as a concrete finding rather than concealed.

3.8.3 Hosting-Cost Analysis

Cost-effectiveness is evaluated through a desk-based analysis of the projected monthly cost of hosting the platform on a modest cloud or virtual-private-server tier. The analysis estimates the platform's resource footprint and applies published provider pricing to derive an indicative monthly operating cost, clearly labelled as an estimate. This provides a defensible figure for the viability discussion without requiring a paid production deployment.

3.8.4 Usability Evaluation (Heuristic and SUS)

The usability strand is specified as an expert heuristic evaluation combined with a standardised usability questionnaire, a recognised and citable approach that is appropriate where a large end-user sample is not available (Nielsen & Molich, 1990; Nielsen, 1994). Three to five evaluators—comprising the project supervisor and peers—would independently perform a fixed set of core tasks (recording an operation with its cost, viewing the profit-and-loss report, generating a yield forecast, and exporting or sharing a report) and then assess the interface against Nielsen's ten usability heuristics, recording issues with severity ratings. Each evaluator would additionally complete the ten-item System Usability Scale (SUS), which yields a single score between 0 and 100 (Brooke, 1996); with a small evaluator panel the SUS score would be reported as indicative rather than statistically powered. Two procedural controls belong to the method: browser site data must be cleared between evaluators, because the offline queue task deliberately creates queued entries; and the checklist and questionnaire must be completed without the author present, because a self-report instrument administered under the author's observation is systematically inflated. The instruments for this

strand—the task script, heuristic checklist and SUS questionnaire—were prepared in full and are provided as a separate evaluation pack. The strand was not executed within the period of this study. Section 4.12 reports that absence and reports no figure against these instruments; the tables there are drafted to receive real data and are deliberately left empty.

3.8.5 Designed Field Evaluation (Future Work)

For completeness and to demonstrate methodological awareness, the ideal field evaluation is specified even though it is not executed in this study. It would recruit a representative sample of smallholder and medium-scale farmers, obtain informed consent, provide a short on boarding session, and have participants use the platform for their own record-keeping over a defined period. It would capture task-completion rates, error rates, SUS scores across a larger sample, and qualitative feedback, and would measure real device performance and data usage in the field. This design is presented as the recommended next step beyond the present work.

3.9 Summary

This chapter has set out the methodology by which AGRI-PROFIT was designed, developed and evaluated. It described an iterative, version-controlled development approach with human-reviewed, agent-assisted implementation; a requirements analysis grounded in the socio-technical constraints of Nigerian agriculture, including one requirement the release does not meet; a client–server architecture built around a mobile-first offline-capable client; and a unified data model in which each operation is atomically paired with its financial consequence and corrected only by a linked contra entry. It specified the design of each module: the offline logbook and its read cache, the ledger engine, reporting, the mechanisation and equipment tracker, a two-tier decision-support engine whose deterministic tier computes nine quantities from real records and

whose optional forecast tier is transparently trained on synthetic data, authentication and the per-farm boundary, tokenised stakeholder reporting, a designed but unimplemented feature-phone channel, the post-harvest drying module, and the enterprise-economics module. It set out a security design resting on authentication, role-based authorisation, data isolation, throttling, credential hygiene and scoped sharing, and named what that design does not provide. Finally, it presented a four-strand evaluation methodology—functional verification, performance benchmarking, hosting-cost analysis, and heuristic-plus-SUS usability evaluation—of which three were executed and one was not, together with a fully specified field evaluation designed as future work. The results of applying this methodology are presented in Chapter Four.

CHAPTER FOUR

4.0 RESULTS AND ANALYSIS

4.1 Introduction

This chapter reports what the implemented AGRI-PROFIT platform does, measured rather than asserted. It is organised around the four evaluation strands specified in Section 3.8 — functional verification (3.8.1), performance benchmarking (3.8.2), hosting-cost analysis (3.8.3) and usability evaluation (3.8.4) — together with three sections reporting the substantive analytical output of the system: the decision-support figures computed from a live ledger, the enterprise-economics figures derived from them, and the post-harvest drying analysis. Those three are reported in their own right because they are the outputs against which Objectives 2 and 3 are judged.

A reporting standard has been applied consistently throughout and is stated at the outset, because it shapes how every result in this chapter should be read. Each claim belongs to exactly one of four classes, and the class is visible wherever the claim is made:

Table 4.1: The evidential classification applied throughout Chapter Four

Class	What it means	What may establish it
Implemented	The code exists and, where stated, is reachable by a user.	Source inspection and the deployed application.
Automatically verified	A named, passing automated test asserts the behaviour.	The test suites only.
Manually inspected	Observed by the researcher, or by a live probe against the running system; no automated test exists.	Live endpoint responses, direct SQL, driven-browser probes, visual inspection.
Numerical result	A measured or computed value with a named command or artefact behind it.	Test and coverage output, live API responses, committed measurement artefacts.

Anything that fits none of these four classes is a limitation. It is reported as such in Section 4.15 and interpreted in Chapter Five, and it is not converted into a result. Where a quantity could not be obtained it is reported as absent rather than filled with a

plausible value. A second distinction cuts across the classification and governs the wording of every claim in this chapter and the next: implemented means the capability is present in the system; verified means it is supported by a named test, command or reproducible check; and validated in practice means it has been demonstrated with real users, in field conditions, or in production operation. The first two are claimed wherever the evidence supports them; the third is claimed nowhere, because no strand of this evaluation establishes it. Several results in this chapter are consequently unflattering — a low frontend coverage figure, a frontend suite that is not timing-robust under load, a usability strand that cannot be reported at all. These are reported as found, on the reasoning that a results chapter which lists only what succeeded is not a results chapter.

The reporting baseline. Every figure, table and screenshot in this chapter is produced from a single implementation state: commit 78a68c292205acdcac1a52f977fbea31b6e8495e on the main branch, dated 29 August 2026, referred to throughout as the baseline commit. This supersedes the earlier annotated tag thesis-evidence-freeze-2026-08-25 and its commit 59a6286: development did not stop at that tag, and a production-hardening branch was merged four days after it. Where a measurement was taken at an earlier commit and has not been repeated, the commit and the date are stated at the point of use and the figure is not presented as a baseline figure. The supporting evidence document is docs/EVIDENCE_FREEZE_2026-08-29.md. Demonstration figures are read from the seeded demonstration farm, farm 26, signed in as its own user; the live database holds several other farms, and no figure in this chapter is read from any of them. The commands that reproduce the whole of Section 4.3 are:

```
python -m pytest backend/tests/ -q --cov=backend/app --cov-report=term
python -m pytest backend/tests/ --collect-only -q
cd frontend && npm run test && npm run test:coverage
cd frontend && npx tsc -b --force && npm run lint && npm run build
```

Section 4.14 draws the strands together against the four objectives of Chapter One and the fifth objective added for the post-harvest strand; Section 4.15 states the limitations of the results themselves. Discussion of what the results mean for the wider problem, and the recommendations that follow from them, are reserved for Chapter Five.

4.2 The Implemented System

This section reports what exists. No verification claim is attached to anything in it; verification is the subject of Sections 4.3 to 4.5. The separation is deliberate, because “the platform does X” and “the platform is proven to do X” are different statements, and conflating them is the most common way a results chapter overstates its evidence.

4.2.1 Deployed components

AGRI-PROFIT is a working end-to-end application rather than a prototype or a mockup. It runs as a three-container Docker composition: a FastAPI backend service, a PostgreSQL 15 database whose schema is managed by Alembic migrations, and a static server for the built Progressive Web Application. The backend is organised into an endpoint layer (authentication, ledger, reports, decision support, bioprocess, equipment and investor sharing), a service layer carrying the business logic (ledger, decision support, enterprise economics, bioprocess, reports, sharing, authentication and equipment), a model and schema layer, and a machine-learning package holding dataset generation, training and prediction. The frontend is organised by feature — farm records, decision support, dashboard, authentication and investor sharing — over a library layer providing the local database, the synchronisation queue, queue ownership, authentication token handling and the read cache, with a Workbox service worker and precache.

The application serves six authenticated routes — dashboard, records, reports, equipment, decision support and investors — and one unauthenticated public route for a shared investor report reached by token.

At the baseline commit the live database held 14 farms, 13 users, 109 operational logs, 109 financial transactions, 3 equipment records, 1 maintenance log and 10 share tokens, with the schema at Alembic revision b9e5f30c74a1. A direct query for operational logs carrying no paired financial transaction returned zero rows. The one-to-one correspondence between operational logs and financial transactions is not a coincidence of the sample: it is the paired-write invariant of Section 3.5 visible in the data, every operational record having posted its financial transaction in the same commit.

4.2.2 Functional inventory

Table 4.2: Implemented capabilities and their reachability

#	Capability	Status
1	Offline-first operational logbook — local queue, automatic synchronisation on reconnect, client-generated identifier for idempotency	Implemented and reachable
2	Financial ledger engine — every operational log posts its paired transaction in one commit; per-crop revenue and expense; profit-and-loss with monthly breakdown and CSV export	Implemented and reachable
3	Ledger integrity — records are soft-immutable; deletion is refused with HTTP 405; corrections are made by category-preserving contra entry linked through <code>reverses_id</code> ; double reversal and cross-farm tampering are rejected	Implemented and reachable
4	Deterministic decision support (Tier 1) — unit cost of production on two bases, per-	Implemented and reachable

#	Capability	Status
	crop gross margin and ranking, retrospective break-even yield, both break-even prices, sensitivity matrix, operating-expense ratio, partial budget	
5	Predictive decision support (Tier 2) — Random Forest yield forecast with a confidence band derived from the spread of the forest's trees	Implemented and reachable
6	Enterprise economics — cost-behaviour classification at the schema edge, derived fixed-cost overlay with depreciation, proportional allocation of fixed cost to crops	Implemented and reachable
7	Post-harvest drying — Page and Newton model fitting, wet↔dry basis conversion, water balance, safe-storage lookup, per-crop aggregation, drying-aware unit costs	Implemented and reachable
8	Identity and per-farm data boundary — signed token authentication with hashed passwords; every query farm-scoped; a cross-farm read returns HTTP 404	Implemented and reachable
9	Tokenised stakeholder sharing — revocable read-only link; tokens stored hashed at rest	Implemented and reachable
10	Mechanisation and equipment tracker — equipment, maintenance logs, depreciation rate	Implemented and reachable
11	Olympic-average yield baseline — discards the highest and lowest season before averaging; refused below three seasons, with the season count stated in	Implemented and reachable. Rendered by the yield-baseline panel within the enterprise-economics view.

#	Capability	Status
	every branch	
12	Runtime crop list on the entry form — the selectable crops are merged at runtime from the farm's own recorded crops and the forecast model's crops	Implemented and reachable. Every crop that holds a ledger can be entered through the form.
13	Per-operation equipment attribution and machine hours	Captured but not costed. <code>equipment_id</code> and <code>hours_used</code> are validated and stored but read by no service. No cost-per-hour, utilisation or machine-rate figure exists anywhere in the platform, and none is reported in this thesis.
14	Role-based authorisation — three roles over a table of twelve named permissions; the permission check precedes the farm-scope check, so a refusal never	Implemented and verified live on the running stack.

#	Capability	Status
	doubles as an existence oracle	
15	Login throttling — a fixed failure budget within a rolling window; the refusal names no account	Implemented and verified live.
16	Share-token expiry and credential hygiene — ninety-day expiry, hashed at rest, refused as a bearer credential, and unknown, revoked and expired tokens all resolving to one identical response	Implemented and verified live.
17	Monetary and quantity input bounds enforced at the schema edge	Implemented and verified live. Negative, absurd and non-finite values are rejected.
18	Equipment correction — a partial-update path that stamps a modification time only when a value actually	Implemented and verified live.

#	Capability	Status
	changes	
19	Post-harvest drying readings capture — repeatable time-and-moisture rows on the entry form, validated client-side and plotted	Implemented and reachable. A Page fit can be obtained from a run entered through the interface.
20	Bioprocess summary endpoint (GET /bioprocess/summary)	Implemented, not consumed. The endpoint and its typed client method exist and no component calls them.

Two entries in this table bound what the thesis may claim, and they are reported here rather than omitted because each is a genuine finding about the built system. Item 13 bounds the mechanisation claim: the platform records mechanisation cost within the ledger and classifies it by behaviour, and it performs no machine-hour or utilisation analysis. The classification decision is recorded in the project's third architecture decision record. A further constraint on any such analysis is that hours_used is populated on only three of the eight seeded mechanisation logs, so a machine-hour total computed from today's data would be a total over an unknown fraction of actual machine use. Item 20 records an endpoint that is built, typed and served and that no interface component calls. Two findings carried by earlier drafts of this table have been

withdrawn: the Olympic-average yield baseline and the entry form’s crop list were both reported as implemented-but-unreachable, and both are reachable at the baseline commit.

4.3 Functional Verification: Automated Test Evidence

Section 3.8.1 specifies verification of the implemented functionality against the requirements of Chapter Three by means of an automated test suite. This section reports that strand. Every figure in it is measured at the baseline commit by the commands given in Section 4.1, and nothing in this section is drawn from any other source.

4.3.1 Suite identity and headline results

Table 4.3: Verification results at the baseline commit 78a68c2

Check	Command	Result
Backend test suite	<code>python -m pytest backend/tests/ -q</code>	422 collected, 418 passed, 4 skipped, 0 failed, 1 warning, 279.89 s
Backend statement coverage	<code>python -m pytest backend/tests/ -q --cov=backend/app --cov-report=term</code>	96% — 1,594 statements, 66 missed
The four skipped tests, executed for real	<code>MIGRATION_TEST_DATABASE_URL=... python -m pytest backend/tests/test_migrations.py -v</code>	12 passed, 0 skipped, 6.19 s, against PostgreSQL 15.18

Check	Command	Result
Frontend test suite	npm run test	13 files, 148 tests, 148 passed, 0 failed
Frontend statement coverage	npm run test:coverage	43.18% — 558 of 1,292 statements, measured over the whole of src
TypeScript compilation	npx tsc -b --force	Exit 0, no diagnostics
Static analysis	npx eslint .	Exit 0, 0 errors, 0 warnings
Production build	npm run build	Built with no errors; service worker registered in the built output

Table 4.3 states the identity of each check, the command that produces it, and its result at the baseline commit; every figure quoted in this section is taken from that table. The suite runs against the real application rather than against mocks: the backend cases exercise the FastAPI application through its test client with a real database session, and the frontend cases exercise the shipped modules, including a genuine IndexedDB implementation. The single backend warning is a deprecation notice raised inside the installed FastAPI package’s own test-client import and not by any code in this repository; it is reported rather than suppressed.

The coverage measurement is statement coverage, not branch coverage. Branch measurement is not enabled and no coverage configuration in the repository turns it on. This is stated because “96% coverage” invites the question of which kind, and the

honest answer is the weaker of the two. The backend counts are pytest collected counts rather than counts of test functions; six backend modules parameterise, so a count of test functions reads 355 and understates the suite by 67. Collected counts are quoted throughout.

4.3.2 Where the coverage sits

The headline percentage is less informative than its distribution. Of the 66 missed backend statements, the large majority lie in two modules — the synthetic-data generator and the model-training script — which are the offline command-line paths of the machine-learning tier and are not exercised by any request.

Table 4.4: Backend modules below full statement coverage

Module	Statements	Missed	Coverage
ml/train.py	53	28	47%
models/database.py	11	4	64%
ml/dataset.py	50	9	82%
api/deps.py	29	3	90%
api/endpoints/dss.py	50	5	90%
ml/predict.py	46	4	91%
core/security.py	30	2	93%
main.py	78	5	94%
services/reports_ser	68	4	94%

Module	Statements	Missed	Coverage
vice.py			
services/auth_service.py	63	1	98%
services/ledger_service.py	56	1	98%
Total missed across these modules	—	66	—

Every other backend module with executable statements is at 100%, including the decision-support service, the enterprise-economics service, the bioprocess service, the sharing service, the equipment service, the roles table, the rate limiter, the logging-safety module, the schema layer and the data models. That distribution is the substantive result: every module that serves a request is fully or almost fully covered, and the shortfall is concentrated in paths that are run manually. That is a defensible place for coverage to be thin, and it is nevertheless a real gap — it means the training pipeline is the least-verified part of the backend, which bears directly on how the model figures of Section 4.9 should be read.

The frontend figure must be quoted alongside the backend one or not at all; 96% quoted alone misrepresents the system. Measured over every file under src, frontend coverage is 43.18% of statements (558 of 1,292), 32.41% of branches (318 of 981), 36.16% of functions (149 of 412) and 43.00% of lines (492 of 1,144). The headline is low because the React page and form layer has no component tests at all — FarmRecordsPage.tsx,

FarmRecordCreateForm.tsx, DSSPredictPage.tsx, InvestorsPage.tsx, PublicInvestorReport.tsx, ReportsPage.tsx, Login.tsx, DryingCurveChart.tsx and the hooks. The tested logic modules, by contrast, are high: the drying parameter rules at 94.33%, the enterprise-economics view at 94.28%, and the break-even price panel, the cost-structure panel, the yield-baseline panel and the empty-state component at 100%. Table 4.5 lists the thirteen test files and the number of cases each contributes.

Table 4.5: Frontend test files at the baseline commit, with collected test counts

File	Tests
features/farm-records/recordBounds.test.ts	22
features/farm-records/dryingParams.test.ts	20
app/routeAccess.test.tsx	18
lib/queueOwner.test.ts	16
lib/sync.test.ts	15
features/dss/partialBudget.test.ts	15
app/navigation.test.ts	14
features/farm-records/cropOptions.test.ts	8
lib/dbUpgrade.test.ts	7
lib/cacheInvalidation.test.tsx	5
features/dashboard/dashboardAccess.test.t	5

File	Tests
SX	
features/auth/AuthProvider.test.tsx	2
lib/apiCache.test.ts	1
Total	148

4.3.3 Capability-to-proof mapping

The behaviours below are those on which the design argument of Chapter Three depends. Each is asserted by at least one named passing test. The mapping is given at the level of behaviour rather than of endpoint, because several of these behaviours are properties of the interaction between components rather than of any single one.

Table 4.6: Behaviours verified by automated test, with the evidence for each

Behaviour verified	What is asserted	Proof
Cross-farm offline-key handling	Two farms may hold the same client-generated key; neither sees the other's record and neither request fails with a server error	Three tests over the composite constraint, plus the migration that introduced it
Same-farm idempotency, sequential	Replaying a record returns the same row with HTTP 200, with exactly one log and one transaction stored	test_idempotent_log_creation; test_farm_scoped_idempotency_still_holds_after_the_constraint_change
Same-farm idempotency, concurrent	The loser of a race receives the winner's row, its flushed transaction rolled back rather than double-booked; the endpoint answers 200, not 500; a non-replay integrity error still raises; two unstubbed threads racing one key book exactly one record	test_concurrency.py, 5 tests, file-backed SQLite with two real connections; the threaded case run ten times consecutively without a flake
Offline queue identity isolation	The queue is partitioned by an owner key derived from the token's subject; every read path uses the compound owner-and-status index	queueOwner.test.ts (16) and sync.test.ts (15); both modules at 100% statements

Behaviour verified	What is asserted	Proof
Logout queue cleanup	Only the signing-out account's rows are deleted, before the token is cleared; a no-op when signed out; never blocks logout	The logout-cleanup block of <code>sync.test.ts</code> ; the by-owner purge case in <code>dbUpgrade.test.ts</code>
Local database migration	A genuine version-1 database, opened by the shipped version-2 module: all rows survive with payload intact, ownership is attributed, both indexes are queryable, and migrated rows then flush through the real synchronisation module	<code>dbUpgrade.test.ts</code> , 7 tests
Mechanisation schema behaviour	Cost subtype is validated against a closed taxonomy and is the only field that moves a derived figure; equipment identifier and hours are validated, persisted, read by nothing, and survive a read-back unchanged	Five schema and API tests
Drying model and readings	Process-loss and clean-balance fixtures, exact Page-model recovery, insufficient and out-of-range readings, wet↔dry basis round-trip, safe-storage lookup, water balance distinguished from mass difference	<code>test_bioprocess_service.py</code> (11); the service at 100% coverage
Drying input validation	Every malformed run is rejected as HTTP 422 at the schema edge, never 500	8 API tests plus 20 client-side tests mirroring the same rules
Drying arithmetic and API outputs	Detail and summary access control, cross-farm 404, exclusion of reversed runs, per-crop aggregation	The bioprocess detail and summary test blocks
Yield-baseline computation	Three seasons required; one high and one low discarded; ties discard one instance rather than all; mixed units return null with a stated reason ; the season count is reported in every branch	6 API tests and 4 service tests
Crop-selection logic	The farm's recorded crops and the forecast model's are merged, deduplicated across case and whitespace, the null crop dropped rather than displayed, one failed source falling back to the other, the	<code>cropOptions.test.ts</code> (8); the module at 100%

Behaviour verified	What is asserted	Proof
	list sorted stably	
Enterprise economics	Classification and coverage, the depreciation overlay (a zero rate counting as <i>unrated</i> , never as a zero charge), proportional allocation, both break-even prices with the cash price provably strictly lower, the sensitivity matrix, the operating-expense ratio, and the partial budget signed both ways	test_enterprise_service.py (40); the service at 100%
Offline/backend partial-budget parity	Two independent implementations agree against one shared hand-computed fixture, including that an exact zero is not returned as negative zero	partialBudget.test.ts (15) and the backend parity block
Decision-support outputs	Per-crop ranking, alphabetical tie-breaks, break-even yield and each of its null cases, reversal netting at farm level, drying-aware unit costs, every degenerate crop case	The decision-support service at 100% with its API test block
Cache invalidation on write	Log writes, queue flushes, equipment creation, maintenance logging and reversals each invalidate the cached derived reads	cacheInvalidation.test.tsx (5)

4.3.4 Movement against the previously reported figures

Figures reported earlier in this project were 190 backend tests at 93% statement coverage over 1,286 statements with 91 missed, and 89 frontend tests across nine files at 33.48% statement coverage. Those figures were measured at the superseded 25 August tag and are not the figures of the submitted software. The movement is accounted for by test modules and application code added by the production-hardening branch that merged after that tag, and the important property of the movement is that it is additive: the four backend test modules that existed at the earlier tag still collect exactly 190 cases between them, so nothing was weakened, retired or relaxed to reach the higher figure.

Table 4.7: Reconciliation of the test figures

Figure	Previously	At the freeze	Cause
Backend tests (collected)	190	422	Nine test modules added by the hardening merge: authorization, input validation, equipment correction, logging safety, rate limiting, reproducibility, share lifecycle, migrations and enterprise service
Backend statement coverage	93%	96%	New tests reach previously unexercised application code
Backend statements missed	1,286 / 91	1,594 / 66	308 statements of new application code, and 25 previously

Figure	Previously	At the freeze	Cause
			missed statements now executed
Frontend test files	9	13	navigation, routeAccess, dashboardAccess and recordBounds added
Frontend tests	89	148	The same four files
Frontend statement coverage	33.48% (378 / 1,129)	43.18% (558 / 1,292)	Routing, access and input-bound logic brought under test
Pre-existing backend modules	190	190	Unchanged — the increase is entirely additive and no existing test was weakened or removed

Table 4.7 sets each superseded figure against its baseline value and names the cause of the movement. The reconciliation is given because a test count that moves without an explanation is not evidence. One further property of the frontend suite must be reported with its count. In a full-suite run executed concurrently with the backend suite, one file — `dashboardAccess.test.tsx` — exceeded the test runner’s 5,000 ms default timeout, at 23,061 ms; run alone the same file passes in under three seconds, and the full suite passes 148 of 148 on an uncontended machine, as it did on re-measurement for this chapter. The behaviour has been observed on two separate days. It is a limitation of the test harness under machine contention and not a defect in the application, and the honest statement of the result is therefore that the suite comprises 148 tests, one of which is not timing-robust in a full-suite run under load.

4.4 Negative-Control Testing

A passing test suite demonstrates that the behaviours somebody thought to test behave as specified. It is silent about whether those tests would notice if the behaviour broke. That distinction is not academic: an assertion that a purge function was *called* passes just as happily when the purge deletes nothing, which is the exact class of defect such a test exists to catch.

Three sets of mutation experiments were therefore run, in which the implementation was deliberately broken and the suite re-run to confirm it failed, and failed for the right reason. This is the strongest single piece of verification evidence in this chapter, and it is reported in full. The runs were executed at commit 59a6286 on 25 August 2026, against frontend cache logic and fixture-loading logic that the subsequent merge did not change; they have not been repeated at the baseline commit, and the attribution is stated here rather than left to be inferred.

Table 4.8: Negative-control (mutation) runs

#	What was broken	Suite	Outcome
1	The read-cache purge replaced with a no-op, its signature and export unchanged so that every call site still resolves and still awaits	Cache-invalidation suite	5 failed, 0 passed. Every failure is on a value or on rendered text; none is on a call count
2	The local-database upgrade hook broken	Migration suite	4 of 7 tests fail
3a	The shared parity fixture renamed away	Both backend and frontend	Failure at collection and transform time , before any test body executes
3b	The parity fixture truncated to invalid JSON	Both	Decode failure at collection and transform time
3c	The fixture's declared case count raised to 9 while its cases remained 8	Both	The internal-consistency test fails in each suite: 1 failed, 39 passed (backend); 1 failed, 14 passed (frontend)

Table 4.8 records what was broken in each experiment, which suite was re-run, and whether it failed. Experiment 1 is the most informative. Each of the five tests seeds a real cache entry through a stand-in service worker, performs the write, and re-reads — so the assertion is on the value the user would see. With the purge neutered, the decision-support figure reads ₦0 where ₦3,500 is correct after a log write; ₦0 where ₦5,500 is correct after a queue flush; ₦0 where ₦12,000 is correct after a maintenance entry; a stale sentence about depreciation rates where a corrected one is expected after an equipment write; and — the discriminating case — ₦5,500 where ₦2,000 is correct after a reversal. That last figure matters because it is not zero: only a working purge combined with correctly netted contra accounting produces ₦2,000, whereas a surviving cache produces ₦5,500 and a lost record produces ₦0. The three outcomes are distinguishable, so the test can only pass for the right reason.

Experiment 3 establishes a different property: that the parity fixture cannot go missing, malformed or silently shorter without failing both suites. A parity test that skips or defaults when it cannot find its input is indistinguishable from no parity test at all, and runs 3a and 3b show that no such code path exists — the failure occurs before any test body runs. Run 3c covers the case import-time failure cannot catch, by comparing the file’s own declared case count against the number of cases present and both against a constant declared independently in each test file.

What the negative controls do not establish. They establish that these tests are load-bearing. They do not establish anything about behaviour the suites do not reach — in particular, the background-revalidation half of the service worker’s caching strategy, which is modelled in neither harness. That behaviour is reasoned from the handler’s ordering and has never been reproduced; it is reported in Section 4.15 as a candidate defect rather than a confirmed one. Nor can a parity run establish that two implementations are not wrong in the same way; it establishes only that hand-computed expectations, which neither implementation generated, disagree with neither.

4.5 Manually Inspected Evidence

This section reports evidence obtained by observation rather than by automated test. It is weaker evidence, and it is separated from Section 4.3 for that reason. It is nonetheless evidence, and reporting it separately is what allows Section 4.3 to be read as strictly as it is written.

4.5.1 Components verified visually only

The drying-curve chart component has no automated test and 0% coverage. Its visual rendering correctness is established by manual inspection alone, and no

claim of automated coverage is made for it anywhere in this thesis. The distinction is material because the drying *calculations* the chart displays are fully covered: the bioprocess service is at 100%, the client-side drying rules are covered by 20 tests, and the drying API outputs — detail, summary, validation and reversal handling — are covered by the test block named in Table 4.6. What is not covered is the drawing of the picture.

The same status applies to the wider page and form layer, which is listed here so that no reader infers coverage from the presence of a screenshot: the record creation form, the records page, the drying entry fields, the drying result panel, the login page, the decision-support prediction page, the investor pages, the application shell, and the hook that wraps the fully tested crop-option module. All are implemented, all are reachable, and none is exercised by an automated test.

No interface screenshot is presented in this chapter. The decision is deliberate and is recorded here rather than left as an omission. A screenshot is evidence only of what was on the screen when it was taken, and the states this chapter would want to show — a populated dashboard, a drying run with its computed marketable mass, the two unit-cost bases side by side, the withheld figures with their stated reasons — are all already reported here as live API responses traceable to the records that produced them, which is the stronger form of the same evidence. Where a state could not be produced by the system it is reported as absent in Section 4.15 and is not staged. The figures that this thesis does present, in Chapter Three, are design diagrams of the architecture, the data model and the paired write; they are author-drawn representations of the design and are not evidence captures, and they are labelled as such. Capturing the interface states named above against the running system, to the project's own capture run-sheet, is

recorded in Section 5.3.2 as work that would strengthen the presentation of this chapter without changing any result in it.

4.5.2 Live system verification

Every endpoint on which this chapter's figures depend was exercised against the running stack and returned HTTP 200: the decision-support breakdown, the model information endpoint, the bioprocess summary and detail endpoints, and all five enterprise-economics routes. No probe returned a non-2xx response. The authenticated probes were performed as the seeded demonstration farm's own user, and the responses are the source of every figure in Sections 4.6 to 4.9.

Two further live checks were performed and are reported because they bear on data integrity rather than on functionality. First, after the verification-artefact cleanup that preceded the freeze, the database state was confirmed by direct SQL: the transient verification farms and their logs were absent, and every seeded and demonstration record was unchanged. Second, the backend answered an unauthenticated request to a protected route with HTTP 401 and a root request with HTTP 200, with no error appearing in its log after the deletion.

4.5.3 Service-worker attribution by driven browser

A zero-byte network transfer establishes that a resource came from a cache; it does not establish which cache. Because the architectural claim under test concerns the service worker specifically — an ordinary browser HTTP cache would evidence nothing about the design — attribution was established directly by driving a real browser rather than inferred from timings. The probe was performed at commit c16924e on 17 August 2026 against service-worker configuration that the subsequent merge did not change; it has

not been repeated at the baseline commit, and no figure in this chapter other than the attribution itself depends on it.

Table 4.9: Service-worker attribution probe

Check	Result
Page controlled by a service worker	True; the controller is the built service worker
Cache storage present	One cache, the Workbox precache
Bundle resolvable from cache	Hit, HTTP 200
Full navigation with the network disconnected	HTTP 200, served from the service worker
Rendered result offline	The login screen rendered, with no console or page errors

Table 4.9 lists the checks the probe performed and their results. One qualification is recorded because it was encountered and resolved rather than assumed away. After the offline navigation the browser’s own connectivity flag reported the network as available, which taken at face value would have invalidated the test. Control requests to resources outside the precache, issued both before and after the navigation, were rejected with a disconnection error, as were requests to the backend. The network was therefore genuinely unreachable at the moment the navigation succeeded, and the connectivity flag is a known artefact of the emulation applied to a freshly created document.

This probe is the evidence for the offline claim in Section 4.10, and it is manually inspected evidence, not an automated test. **No end-to-end browser test of the offline-to-online transition, the service-worker lifecycle or the installation path exists**, and none is claimed.

4.6 Decision-Support Results from the Live Ledger

Objective 2 concerns the conversion of recorded farm activity into decision-relevant financial information. The deterministic tier is exactly that conversion: it computes from the ledger the platform actually holds, with no model, no estimation and no external

data. This section reports its output against the seeded demonstration farm at the baseline commit. Every figure is a live API response, and every one is traceable to the records that produced it.

4.6.1 Per-crop gross margin and ranking

Table 4.10: Per-crop decision support, farm 26, at the baseline commit

Rank	Crop	Revenue (₦)	Expenses (₦)	Gross margin (₦)	Yield	Marketable mass
1	Cowpea	624,950	267,700	357,250	480 kg	431 kg
2	Cassava	178,600	94,400	84,200	1,880 kg	—
3	Maize	45,000	3,500	41,500	100 kg	84 kg
4	Tomato	0	112,900	−112,900	none	—
5	Sorghum	0	119,950	−119,950	none	—
	Farm total	848,550	598,450	250,100		

Table 4.10 gives the ranked per-crop position of the demonstration farm. Four properties of this output matter to the argument, and each is pinned by a named test irrespective of the seeded amounts.

First, **the ordering is computed rather than incidental**. Crops are sorted by descending gross margin with alphabetical tie-breaking, and the test is constructed so that an alphabetical sort cannot pass it by coincidence. Note that the ranking is by margin, not by scale: cassava earns nearly four times maize’s revenue and returns roughly twice its margin, while maize returns ₦41,500 on ₦3,500 of recorded cost. Ranking and efficiency are different questions, and the platform answers the first.

Second, **a loss is displayed as a loss**. Tomato and sorghum carry recorded input costs and no sale, and both report negative margins rather than being suppressed or shown as zero. The loss-making enterprise is precisely the one a farmer most needs to see, and a system that quietly omits it would fail at the point of greatest value.

Third, the figures are the ledger, not a projection. Each is the sum of transactions carrying that crop's category, with reversed pairs netted — at farm level and, as Section 4.6.4 establishes by execution, per crop as well.

Fourth, **the yield column is unit-aware**. Where a crop's harvests were recorded in incompatible units the platform returns a null rather than an arithmetically invalid sum. Farm 26 is unit-clean, so each crop here reports a single unit; the wider database is not, which is why the behaviour exists.

4.6.2 Unit cost of production on two bases

Unit cost is the figure most directly connected to the pricing decision a farmer faces at the farm gate. The platform computes it as attributed cost divided by output, and — this being the point at which the record-keeping layer meets the process-engineering layer — it distinguishes the mass that left the field from the mass that can actually be sold.

Table 4.11: Unit cost of production, farm 26

Crop	Cost basis	Denominator	Unit cost	What it answers
Maize	Harvest (wet) mass	100 kg	₦35.00 / kg	What the crop cost to produce in the field
Maize	Marketable mass	84 kg	₦41.67 / kg	What each saleable kilogram must recover at the gate
Cowpea	Harvest (wet) mass	480 kg	₦557.71 / kg	
Cowpea	Marketable mass	431 kg	₦621.11 / kg	
Cassava	Harvest mass	1,880 kg	₦50.21 / kg	No drying run recorded, so no marketable basis exists

The 19% difference between ₦35.00 and ₦41.67 is post-harvest mass reduction expressed in money. A farmer pricing against the wet-basis figure would be pricing against a mass that no longer exists and would under-recover by ₦6.67 on every

kilogram sold. The same pattern appears on cowpea, where the two bases differ by ₦63.40 per kilogram.

Two reporting decisions inside those figures are themselves results. The marketable denominator is **the mass actually weighed out, not the mass the dry-matter balance predicts**. For the maize run those two differ: from 100 kg at 25% moisture the dry matter is 75.0 kg, which at 13% final moisture should weigh 86.21 kg, against 84 kg observed. Costing against the predicted 86.21 kg would give ₦40.60 per kilogram and would charge the farmer's pricing decision with grain that never reached the store. Costing against the observed 84 kg is the conservative and correct choice, and it is the one the platform makes. And where no drying run exists, as with cassava, the marketable figure is withheld rather than defaulted to the harvest mass — the two are not the same quantity, and reporting one as the other would be a silent error.

The API returns these values at full floating-point precision — the marketable maize figure is 41.666666666666664 — and the interface formats to two decimal places. The ₦41.67 quoted throughout this thesis is that value rounded to the kobo, and the residual difference of roughly a third of a kobo is a display artefact rather than a discrepancy.

4.6.3 Retrospective break-even yield

Break-even yield reports the output at which a crop would have covered its recorded costs **at the price actually achieved**. It is deliberately retrospective, and the interface says so by rendering it in the past tense, because the platform holds no forward price forecast and must not imply one.

Table 4.12: Retrospective break-even yield, farm 26

Crop	Recorded cost (₦)	Realised price (₦/kg)	Break-even yield	Actual yield
Maize	3,500	450.00	7.78 kg	100 kg

Crop	Recorded cost (₦)	Realised price (₦/kg)	Break-even yield	Actual yield
Cassava	94,400	95.00	993.68 kg	1,880 kg
Cowpea	267,700	1,302.00	205.61 kg	480 kg
Tomato	112,900	none realised	null	none
Sorghum	119,950	none realised	null	none

Table 4.12 sets each crop’s recorded cost and realised price against the output that would have covered them. The maize record covered its recorded cash costs within the first eight kilograms sold. The nulls are as informative as the numbers: where no sale has been recorded there is no realised price, and a break-even yield computed against a price that does not exist would be arithmetic dressed as advice. The figure is likewise withheld where no yield has been recorded, where the sale that established the price has since been reversed, and where no cost has been attributed to the crop — in that last case an arithmetically valid break-even yield of zero kilograms would communicate a false completeness to a user whose actual situation is that they have not recorded their costs. All four null cases are asserted by named tests.

4.6.4 A Withdrawn Defect Report: Per-Crop Reversal Netting

One methodological finding must travel with Tables 4.10 to 4.12, and it is reported here rather than deleted because an earlier version of this chapter carried the opposite claim.

An audit conducted on 25 August 2026 reported that the per-crop decision-support breakdown failed to net reversals, that a reversed cost continued to appear against its crop, and that a phantom “Unspecified” bucket appeared alongside it. The finding was carried into three project documents and into an earlier draft of this chapter, and it was false. It was settled by execution rather than by argument: at the baseline commit, and again at the earlier commit the audit itself described, a maize expense of ₦25,000 was reversed against a crop holding ₦35,000 of recorded cost, and the decision-support response moved that crop from ₦35,000 to ₦10,000

with its unit cost of production moving from ₦2,916.67 to ₦833.33 and with no “Unspecified” bucket appearing at any point. The investor-facing report, fetched with a live share token after the same reversal, returned the same netted figures. Five named tests pin the behaviour. The capability had in fact been introduced nine days before the audit that reported it missing, and the audit’s own summary table listed it among the verified behaviours, so the two halves of that document contradicted each other. The finding was produced by reading source code and never executing it, and that is the methodological result worth recording: a defect established by reading code is a hypothesis, and it becomes a finding only when it is reproduced. This one could not be. The passage is corrected rather than quietly removed because the erroneous text survives in a dated evidence document that a reader may encounter, and an unexplained disappearance is a worse record than a stated retraction.

4.7 Enterprise Economics Results

The enterprise-economics layer extends the deterministic tier from margin accounting to cost-behaviour analysis. All figures are live responses from farm 26 at the baseline commit.

4.7.1 Cost structure and classification coverage

Table 4.13: Cost structure, farm 26

Bucket	Maize (₦)	Whole farm (₦)
Variable cost	3,500	483,650
Semi-variable cost	0	26,500
Recorded fixed cost	0	18,000
Unclassified cost	0	70,300
Total recorded cost	3,500	598,450
Cash cost (variable + semi-variable)	3,500	510,150
Classification coverage	100%	88.25%

Bucket	Maize (₦)	Whole farm (₦)
Revenue	45,000	848,550
Cash operating cost	3,500	580,450
Operating-expense ratio	7.78%	68.40%

Table 4.13 gives the cost structure for maize and for the farm as a whole, bucket by bucket. Classification coverage is itself the substantive result. It states what proportion of recorded cost carries a cost subtype, and it is reported rather than hidden because every downstream figure inherits its incompleteness. At farm level 11.75% of recorded cost is unclassified — two mechanisation logs entered without cost parameters — and the platform reports that as unclassified rather than defaulting it into a bucket. An unclassified cost sits inside the total and outside the cash figure, so a reader can see exactly how much of the analysis rests on classified data. The maize enterprise, at 100% coverage, is the case where the figures can be read without that reservation.

The operating-expense ratio excludes the depreciation overlay of Section 4.7.2 and excludes any recorded depreciation charge, both of which are non-cash; it is a measure of how much of each naira of revenue is consumed by cash operating cost. Maize consumes 7.78 kobo in the naira; the farm as a whole consumes 68.40.

4.7.2 The depreciation overlay and fixed-cost allocation

Depreciation is computed as a derived overlay at report time and is **never posted to the ledger**. The reasoning is recorded in an architecture decision record and is worth restating because it is a design result rather than an arithmetic one: posting a synthetic depreciation row would require either an unpaired entry, which violates the paired-write invariant that the whole ledger design rests on, or a fabricated operational log describing an event that did not happen. Neither is acceptable in a ledger whose claim is that every

row records something a farmer did. The overlay is therefore computed, transparent and recomputable, in the same way as every other derived figure on the platform.

Table 4.14: Derived fixed-cost overlay, farm 26

Quantity	Value
Reporting period	7.0 days, derived from the ledger span
Period fixed cost (depreciation overlay)	₦10,068.49
Equipment records	2
Equipment without a usable depreciation rate	1
Total direct cost, all crops (the allocation base)	₦598,450
Fixed cost allocated to maize	₦58.88

Table 4.14 reports the derived overlay and the allocation computed from it. Three properties are asserted by test. A zero depreciation rate counts as unrated and is reported in the unrated count, never as a zero charge — so a partial overlay is visible as partial rather than being read as a small true fixed cost. The allocation is proportional to direct cost and is computed farm-wide before any crop filter is applied, so the same figure cannot change according to how it was requested. And where the total direct cost is zero, every share and every allocation is undefined rather than being split evenly, on the reasoning that an even split invents an allocation base that does not exist.

Two simplifications ride along and are stated rather than hidden: the method is straight-line with no salvage value and no accumulated-depreciation floor, so an asset held past its implied life keeps charging and the overlay is an upper bound. The reporting window may also be pinned explicitly rather than derived, which is a reproducibility measure: a derived span widens every time a log is entered, and a figure quoted from a widening window cannot be re-derived afterwards.

4.7.3 Dual break-even price and yield sensitivity

Table 4.15: Break-even price, maize, farm 26

Quantity	Value
----------	-------

Quantity	Value
Variable and semi-variable cost	₦3,500.00
Total recorded cost	₦3,500.00
Allocated fixed cost	₦58.88
Total cost	₦3,558.88
Marketable mass	84 kg
Break-even price to cover cash cost	₦41.67 / kg marketable
Break-even price to cover total cost	₦42.37 / kg marketable

Table 4.15 reports both, together with the quantities from which each is derived. Two prices are reported rather than one because they answer different questions: what the farmer must obtain this season to avoid a cash loss, and what must be obtained to cover the wear on the assets used as well. The cash price is provably strictly lower than the total price, which is asserted directly by test, including in the edge case of full classification coverage and no overlay. Both are withheld entirely where no marketable mass exists, because a price per kilogram of nothing is undefined.

Table 4.16: Yield sensitivity matrix, maize, farm 26

Yield scenario	Marketable mass	Break-even price, cash	Break-even price, total
75% of recorded	63.0 kg	₦55.56 / kg	₦56.49 / kg
90%	75.6 kg	₦46.30 / kg	₦47.08 / kg
100% (baseline)	84.0 kg	₦41.67 / kg	₦42.37 / kg
110%	92.4 kg	₦37.88 / kg	₦38.52 / kg
125%	105.0 kg	₦33.33 / kg	₦33.89 / kg

Table 4.16 gives the sensitivity matrix across five yield scenarios. The matrix is labelled conditional in the response and in the interface, because every row other than the baseline describes a yield that did not occur. It answers the planning question the deterministic tier is otherwise silent on: how much the required price moves if the harvest disappoints. A quarter shortfall in marketable mass raises the cash break-even price by a third.

4.7.4 Partial budget and implementation parity

The partial-budget calculator evaluates a proposed change by four terms — added revenue, reduced cost, lost revenue and added cost — and returns the signed net change. A probe with added revenue ₦120,000, reduced cost ₦15,000, no lost revenue and added cost ₦90,000 returned benefits of ₦135,000 against costs of ₦90,000 for a net change of +~~₦~~**₦45,000**. The four inputs were chosen for the probe and are not seeded data; the route is stateless and wrote nothing.

The result of interest here is not the arithmetic but the parity. The partial budget is implemented twice — once in the backend service and once in the client, so that it remains available offline — and two implementations of the same formula are an invitation to divergence. Eight cases were hand-computed, placed in a single shared fixture read verbatim by both suites, and asserted against both implementations. **They agreed on all eight on the first run; no divergence was found.** The cases include a negative result whose sign is asserted separately from its magnitude, an exact zero that must not be returned as negative zero, kobo-level precision, and a transposition trap that swaps a pair of terms. The three fixture-mutation runs reported in Section 4.4 establish that the shared fixture cannot go missing or silently shrink without failing both suites.

4.7.5 Yield baseline

The Olympic-average yield baseline discards the highest and lowest season before averaging, and requires at least three seasons. Queried for maize on farm 26 it returns **no Olympic average**, a grand average of 100.0 kg, a season count of 1, and the stated reason that an Olympic average needs at least three seasons and this crop has one. That is the result: the demonstration farm holds a single season, and the platform says so rather than averaging what it has. The season count is reported in every branch of the response, including the branches that return no average, so that a null is never read

without the count that explains it — one season and ten seasons are different reasons for the same null.

This endpoint is implemented, covered by ten tests, and reachable: the yield-baseline panel is rendered within the enterprise-economics view, so the figure and its stated reason are visible to a user. An earlier draft of this chapter reported the baseline as computed, verified, served and invisible; that finding is withdrawn, and all six enterprise-economics endpoints are reachable in the interface at the baseline commit.

4.8 Post-Harvest Drying Results

The post-harvest module implements thin-layer drying kinetics: conversion between wet-basis and dry-basis moisture content, a dry-matter balance determining marketable output, a water balance, safe-storage assessment, and the fitting of two established thin-layer models to an observed moisture series. It carries the bioprocess engineering content of the platform.

4.8.1 Verification by designed fixture

A drying model can only be shown to be correct by recovering parameters known in advance, so the module is verified by hand-calculated fixture rather than by demonstration. Eleven service-level tests pass and the service stands at 100% statement coverage. The properties established are:

- (a) Exact recovery of the Page model. The Page model, $MR = \exp(-k \cdot t^n)$, is linearised by the double logarithm $\ln(-\ln MR) = \ln k + n \cdot \ln t$ and its parameters recovered by least-squares regression on that transform. A moisture-ratio series generated from an initial moisture content of 30.0% wet basis with known parameters $k = 0.30$ and $n = 0.75$ is recovered to those values within a relative tolerance of 1×10^{-4} , which establishes that the linearisation,

the regression and the back-transformation from $\ln k$ to k are each correct — the whole of the fitting path.

- (b) **The water balance is not a mass difference.** A dedicated fixture asserts that the mass of water removed and the process loss are distinct quantities and fails if they are ever made equal. The distinction is the specific error the module exists to prevent: water leaving the grain is the intended outcome of the process, whereas solid matter leaving the system is a loss.
- (c) **A clean-balance control.** A second fixture is constructed so that observed outlet mass equals the mass the dry-matter balance predicts, giving zero process loss, which isolates the balance itself from the loss accounting layered on it.
- (d) **Insufficient and out-of-range readings.** A fit is refused rather than attempted below three readings, and readings outside the physical moisture band are dropped.
- (e) **Safe storage is null, never false, for an unknown crop.** An unrecognised crop has no threshold, and the platform reports the absence of a threshold rather than a negative verdict.
- (f) **Basis conversion round-trips.** Wet-basis to dry-basis and back returns the original value.

4.8.2 The seeded drying run in full

Table 4.17: Maize drying run, farm 26, as computed by the platform

Quantity	Value
Method	Solar dryer

Quantity	Value
Inlet mass	100.0 kg at 25.0% moisture, wet basis
Outlet mass observed	84.0 kg at 13.0% moisture, wet basis
Drying time	10.0 h
Dry matter	75.0 kg
Expected outlet mass from the dry-matter balance	86.21 kg
Process loss	2.21 kg (2.56%) — below the 5% data-quality threshold
Water removed (water balance)	14.08 kg
Mean drying rate	1.408 kg/h
Specific drying rate	0.01877 kg water / kg dry matter / h
Moisture, dry basis	33.33% → 14.94%
Moisture ratio, final	0.4483
Newton (Lewis) rate constant, k	0.08023 h ⁻¹
Page model	n = 0.7956, k = 0.1316, linear r ² = 0.9981 over 3 readings, 0 dropped
Safe-storage verdict	Safe, against the 13.0% wet-basis maize threshold

Table 4.18: Per-crop drying summary, farm 26

Crop	Runs	Mass in	Marketable mass	Water removed	Mean drying rate	Mean Newton k (solar dryer)	Safe-storage share
Cowpea	1	480.0 kg	431.0 kg	36.84 kg	2.631 kg/h	0.03745	1.0
Maize	1	100.0 kg	84.0 kg	14.08 kg	1.408 kg/h	0.08023	1.0

Table 4.17 reports the seeded maize run in full, quantity by quantity, and Table 4.18 aggregates both seeded runs by crop. The two runs are internally consistent in a way that is worth noting: the maize run, which removes a larger proportion of its initial moisture, returns a Newton rate constant roughly twice the cowpea run's under the same drying method, and the cowpea run's higher absolute drying rate reflects its larger batch rather than a faster process. Both are reported per crop rather than pooled, because a mean rate constant across crops with different initial moistures would not describe anything physical.

One point about the Page fit is recorded because an earlier draft of this chapter left it open. The fit requires intermediate time-and-moisture readings, and the readings on the

seeded runs were supplied by the seed script. At the baseline commit the drying entry form does capture such readings: the drying fields component provides repeatable optional reading rows, the client-side drying rules validate them before the record is queued, and the drying-curve component plots them. A run entered through the interface with three or more readings therefore reaches the backend with a series on which the Page model is fitted, and a run entered without them correctly receives no Page fit rather than a fabricated one.

4.8.3 Input validation

Every malformed drying submission is rejected **at the schema edge with HTTP 422, and never with a server error**. Eight API tests cover outlet mass exceeding inlet mass, a final moisture not below the initial, non-positive mass, moisture outside the physical range, readings not in strictly increasing time order, and readings outside the moisture band. Twenty client-side tests mirror the same rules in the browser, so an invalid run is refused before it is queued as well as after it is sent. This matters for the offline path in particular: a record that the API will always reject, but that the client accepts and queues, would retry forever.

4.8.4 Coupling to the financial ledger

The engineering significance of this module is not the kinetics in isolation — the models are long established — but that its output enters the financial record. A drying run is recorded as an operational activity like any other, so it posts its cost to the ledger through the same paired write as a fertiliser application, and its computed marketable output becomes the denominator of the unit-cost figure in Table 4.11. The chain from a moisture measurement to a price per marketable kilogram — 100 kg at 25% moisture, dried to 13%, weighed out at 84 kg, costed at ₦41.67 rather than ₦35.00 — is computed by the platform without manual intervention at any step. Access control and reversal

handling travel with it: a drying run belonging to another farm returns HTTP 404, and a reversed run is excluded from the summary.

4.9 Yield-Forecast Model Evaluation on Synthetic Data

The predictive tier is a Random Forest regressor serving a yield estimate from agronomic inputs. Its reported quality is:

Table 4.19: Yield model as served by the platform

Property	Value
Model	Pipeline (one-hot encoder + Random Forest regressor)
Estimators	200
Training samples	6,000, synthetically generated
Features	Rainfall, fertiliser used, soil pH, crop
Crops	Maize, rice, sorghum, soybean, cassava
Target	Yield, t/ha
Coefficient of determination (R^2)	0.9762
Mean absolute error (MAE)	0.2414 t/ha
Feature importances	Crop 0.3445; rainfall 0.2845; soil pH 0.2479; fertiliser 0.1231
Trained	1 July 2026
Root mean square error (RMSE)	0.4883 t/ha (0.488334)
Real farmer records used in training	None. Zero.
Dataset fingerprint	sha256 a461b884afc9fafda953ff90cc2f75eb0d860 8f2e58ad9d206fc215d7b6ba5bf
Seeds	Generation seed 42; split test_size 0.2, split_random_state 42; model random_state 42

Property	Value
Reproducibility	Re-derived by regenerating the dataset and refitting, not read from the sidecar; agreement to six decimal places; 18 reproducibility tests pass; agreement recorded across Python 3.14.5 and 3.11.15

These figures are an in-distribution result on synthetic data and nothing more. The training data was generated from a parameterised agronomic relationship and the model was then fitted to that generated data, so an R^2 of 0.9762 measures how well the Random Forest recovers a relationship that was constructed to be recoverable. It is evidence that the machine-learning pipeline is correctly implemented — that features are encoded, the model fitted, and predictions served end to end — and it is **not evidence that the model predicts Nigerian smallholder yields**. No claim of field-predictive accuracy is made anywhere in this thesis.

Three further reservations are recorded because they bound the claim further.

First, the model is not validated against any real observation. Nothing establishes the synthetic data’s correspondence to Nigerian conditions or the fitted model’s accuracy outside its own distribution. What the suite does establish is that a model exists, that its inputs are validated, that a forecast is returned, and that the whole pipeline is seed-deterministic. Accuracy in the field is a different question and this work does not answer it. The two least-covered backend modules remain the training and data-generation paths, and they account for the large majority of the 66 missed statements reported in Table 4.4.

Second — and this reverses a claim carried by an earlier draft — the artefact is reproducible from the repository. The generation seed, the split seed and the estimator seed are all fixed; the generated dataset carries the recorded SHA-256 fingerprint given in Table 4.19; and the three reported metrics were re-derived for this chapter by regenerating the dataset and refitting the pipeline from source, rather than by reading the sidecar file the training run wrote. The re-derived values agree with the recorded values to six decimal places. Eighteen tests in the reproducibility module assert the property, and agreement was additionally recorded across two Python versions, 3.14.5 and 3.11.15, with differing patch versions of the dataframe library. The earlier claim that regenerating the model would produce different figures was wrong, and it understated the rigour of the project’s own pipeline.

Third, the root-mean-square error is computed and reported: 0.4883 t/ha, measured 0.488334 over the held-out split. Chapter Three states that RMSE is a standard complementary measure for continuous prediction tasks in agricultural yield modelling, and all three measures — R^2 0.9762, MAE 0.2414 t/ha and RMSE 0.4883 t/ha — are reported together. The earlier record of a missing RMSE is closed.

What the platform does do with these figures is report them honestly in the interface. Three display states are distinguished: metrics unavailable while loading; an explicit statement that the model has not been trained, where that is the case; and the figures with their disclosure where a trained model exists. The untrained case is deliberately not rendered as R^2 0.0000, because a zero reads to a user as a very poor model rather than as no model. All three states are asserted by test at the endpoint level, though the rendered states themselves are manually inspected only.

4.10 Performance Results

Section 3.8.2 specifies a Lighthouse audit of the production build under mobile emulation and network throttling.

4.10.1 Method and conditions

Lighthouse 13.4.1 was run in headless Chrome against the production build served by vite preview at <http://localhost:4173/> — the build in which the service worker is registered, as distinct from the development server, which registers none. Device emulation was a mobile profile at 412×823 with a device pixel ratio of 1.75, and throttling was Lighthouse’s simulated (Lantern) method throughout with a constant 4× CPU slowdown, so that differences between conditions are attributable to network characteristics alone. The slow-3G condition applies 400 ms latency at 400 Kbps and the slow-4G condition 150 ms at 1,638.4 Kbps. Five cold runs were performed per network condition and three user-flow iterations, each flow iteration containing a cold and a warm navigation. Medians are reported and the individual runs are retained. The whole set was measured at the baseline commit on 29 August 2026 and supersedes an earlier set measured on 17 August at commit c16924e. One property of the method must travel with every figure below: this is simulated throttling against a local server on a single developer machine, which is a model of a degraded network and not a measurement of a real one.

Table 4.20: Lighthouse results at the baseline commit, 29 August 2026 (medians)

Condition	n	Score	FCP	LCP	Speed Index	TBT	Transfer
Slow 4G, 1,638.	5	99	1,709.9 ms	1,859.9 ms	1,709.9 ms	9.0 ms	139,245 B / 8 req

Condi on	n	Score	FCP	LCP	Speed Index	TBT	Transfer
4 Kbps — cold							
Slow 3G, 400 Kbps — cold	5	65	5,661.2 ms	5,661.2 ms	5,661.2 ms	10.5 ms	139,245 B / 8 req
Slow 3G, 400 Kbps — warm	3	100	70.6 ms	1,612.0 ms	—	0 ms	127 B / 7 req
Flow harnes s, cold navigat ion	3	—	≈5,661 ms	—	—	—	139,245 B / 8 req

Cumulative Layout Shift was 0 in every run and is omitted from the table. Request count falls from eight to seven on the warm navigation. The host benchmark index across this set ranged from 1,406 to 1,996; the 17 August set ran between 425 and 1,654, and no score or blocking-time figure may therefore be compared across the two dates.

The final row is an internal control rather than a fourth condition. The user-flow harness performs a cold navigation before its warm one, and that cold navigation independently reproduces the command-line cold median to within about three milliseconds — which establishes that the two harnesses are comparable and that the warm measurement may legitimately be set against the command-line cold result.

4.10.2 The principal finding

Between the first and second navigation of the same flow, on an identical 0.4 Mbps simulated link, first paint fell from 5,661.2 ms to 70.6 ms — a factor of eighty — and network transfer fell from 139,245 bytes over eight requests to 127 bytes over seven. The service-worker precache serves the application shell from local storage, and this is proven rather than inferred by the driven-browser probe of Section 4.5.3: a full navigation with the network disconnected returned HTTP 200 and rendered the login screen. The 127 residual bytes are not noise: they are the manifest icon, which is absent from the precache and is the measured defect reported in Section 4.10.4. The application therefore transfers almost nothing on a repeat visit, and the earlier claim that it transfers nothing at all is corrected by measurement.

The finding must be stated with equal precision about what did not improve. Largest contentful paint in the warm run was 1,612.0 ms — a factor of three and a half, not eighty — and it is largest contentful paint rather than first paint that both

weighs most heavily in the composite score and corresponds most closely to the moment a user considers the page usable. Reporting the warm result as “0.07 seconds” would be true of first paint and materially misleading about the experience. The honest summary is that the service worker all but eliminates network transfer and all but eliminates time to first paint, while time to main content remains bounded by client-side rendering and improves by a factor of about three and a half. That warm figure is within two milliseconds of the one measured on the earlier build, 1,613.8 ms, which is itself informative: reducing the transferred bundle does not move a metric that was never network-bound.

The cold results locate a threshold. Between 1,638.4 Kbps and 400 Kbps first paint rises by 3.95 s, and the arithmetic is checkable: a 139,245-byte payload is 1,113,960 bits, requiring 2.79 s of transfer at 400 Kbps against 0.68 s at 1,638.4 Kbps, with the remainder accounted for by additional round-trip latency across eight requests. CPU throttling was identical in both conditions, confirming a network effect rather than a compute effect. The application therefore performs well from a degraded 3G connection upwards and degrades materially only at throughput characteristic of 2G or EDGE-class service.

4.10.3 Two confounds, and a discarded pair of measurements

Three qualifications are reported in this section rather than deferred to the limitations, because each changes what the numbers above may be used for.

First, the host machine’s speed varies between measurement sets. Lighthouse records a benchmark index of available compute with every run: it ranged from 1,406 to 1,996 across the baseline set and from 425 to 1,654 across the 17 August set. First contentful paint, largest contentful paint and speed index are modelled

analytically under simulated throttling and are largely insulated from host speed; total blocking time and the composite performance score are not. No score difference and no blocking-time difference between the two measurement dates may therefore be attributed to the code, and none is attributed to it here — which specifically rules out reading the fall in cold slow-3G blocking time from 121.0 ms to 10.5 ms as a code effect. The comparison that survives the confound is the warm-against-cold contrast, measured within a single browser session on a single machine.

Second, the movement against the earlier build is now measured rather than reasoned. The 17 August set was run at commit c16924e, which precedes both the enterprise-economics work and the route-level code-splitting present at the baseline. Comparing only the network-bound metrics and the byte counts, which are the figures the confound above leaves comparable: cold transfer fell from 243,724 B to 139,245 B, a reduction of 104,479 bytes or 43%; cold slow-3G first paint fell from 7,674.4 ms to 5,661.2 ms, 26%; cold slow-4G first paint fell from 2,317.7 ms to 1,709.9 ms, also 26%; warm first paint fell from 116.4 ms to 70.6 ms; and warm largest contentful paint was unchanged at 1,612.0 ms against 1,613.8 ms, because it was never network-bound. An earlier draft of this chapter recorded the direction of this difference as likely favourable and noted that a likely direction is not a measurement. It has now been measured.

The re-run called for by an earlier draft of this section has been performed, and Table 4.20 is its result. The precache figure quoted in earlier project documents — 21 entries totalling 859.67 KiB — was measured at commit 59a6286 and has not been re-measured at the baseline; it is therefore not quoted as a baseline figure anywhere in this chapter.

Third, two measurement configurations were executed, found invalid, and discarded. They are reported because in each case the error surfaced through an internal inconsistency in the data rather than through inspection of the configuration, which is the more instructive point. An intermediate network condition specified at 1,600 Kbps was invalid because the default mobile profile already throttles to 1,638.4 Kbps, making the intended reduction negligible, and because the latency parameter used belongs to a different throttling method and was disregarded entirely; it was identified as invalid because the supposedly slower setting returned a marginally *faster* median first paint, which no genuine bandwidth reduction can produce. A first attempt at the warm condition used repeated command-line invocations on the assumption that a registered service worker would persist between them; it does not, because each invocation launches a fresh browser profile, so every supposedly warm run was in fact cold. This was caught by the transfer field showing a full bundle transfer with no service-worker attribution, and confirmed by inspecting the profile directory, which contained no service worker at all. Had those timings been reported they would have shown the service worker to be ineffective, which is the opposite of the truth.

4.10.4 A defect found during measurement

The application's manifest icon, `pwa-192x192.png`, is absent from the service-worker precache and is therefore unavailable offline. At the baseline the evidence is a direct measurement rather than a reading of the precache listing: a warm navigation transfers 127 bytes over seven requests, and those bytes are that icon. The impact is cosmetic — the shell renders correctly without it and an installed application would fall back to a default — and it is recorded as a genuine finding about the build rather than an artefact of the measurement.

4.11 Hosting-Cost Analysis

Objective 4 requires an assessment of cost-effectiveness. **A production deployment was outside the scope of this study, so operating cost is estimated rather than incurred.** The estimate is anchored on the platform’s measured resource consumption rather than on assumption, and is labelled an estimate wherever it appears.

4.11.1 Measured footprint

Table 4.21: Measured container footprint

Component	Measured memory	Note
FastAPI backend	145.5–161.5 MiB	At rest, $\approx 0.2\%$ CPU. Includes the Python runtime, the framework, the ORM, numpy, scikit-learn and the fitted model
PostgreSQL 15	33.4–41.3 MiB	At rest, $\approx 0.0\%$ CPU, against the seeded dataset
Measured subtotal	178.9–202.8 MiB	The two containers exercised during sampling
Static file server	Not measured	Not running during sampling; a Node process serving a built bundle would add on the order of 50–80 MiB, a lightweight static server closer to 15 MiB
Host OS and container runtime	Not measured	Conventionally 300–400 MiB

Table 4.21 reports the measured footprint of each container. Ranges rather than single figures are given because memory varied by approximately 11% between two instances of the same image. Startup was sampled by repeated single-shot polling at a mean interval of 2.37 s, and the widest gap of 4.11 s falls across the steepest part of the memory climb, so the 145.5 MiB startup peak is a lower bound on the true maximum rather than the maximum itself. Two components of the footprint were not measured at all, so the total is part-measured and part-assumed, and is described as such.

4.11.2 A correction to the earlier reasoned estimate

An earlier version of this analysis reasoned the footprint from the known composition of each container and concluded that the machine-learning tier dominated memory, that the train-on-boot step set the sizing constraint, and that a 4 GB instance was required. **Measurement does not support that argument.** The entire backend process occupies 145 to 162 MiB, and whatever share the machine-learning tier accounts for is far too small to influence the choice of instance. The reasoned estimate was wrong by a factor of approximately three, and wrong in kind rather than merely in degree. A 2 GB instance is sufficient.

What measurement does show is that the train-on-boot step costs **time** rather than memory: 8.57 s from restart to accepting traffic, of which the memory climb occupies roughly the first eight seconds. On a single-instance deployment with no redundancy that is 8.6 s of unavailability on every restart or redeployment. The architectural case for externalising the model artefact therefore stands, but it rests on availability and the ability to run replicas, not on hosting cost. This correction is reported rather than quietly replaced, because a superseded estimate and the reason it was superseded are part of an honest method.

4.11.3 Estimated monthly cost

Table 4.22: Estimated monthly operating cost

Line item	USD / month	NGN / month
Instance (2 GB RAM, 50 GB SSD)	\$12.00	₦16,327
Automated snapshot backups (≈20% of instance)	\$2.40	₦3,265
Domain name (amortised)	\$1.00	₦1,361
TLS certificate	\$0.00	₦0
Estimated total	\$15.40	≈ ₦20,950

Table 4.22 sets out the estimate line by line. Conversions use the Central Bank of Nigeria rate of ₦1,360.58 to the US dollar as at 12 August 2026; the parallel-market rate on 13 August 2026 was ₦1,425, so the naira figures are indicative rather than exact. The reference instance is a 2 GB RAM, 50 GB SSD commodity tier. Prices are provider list prices retrieved on 12 August 2026, exclusive of tax and of any committed-use discount; no provider was contacted and no quotation was obtained, and the figures are therefore an estimate anchored on a retrieval date rather than a quoted price. Any reader re-deriving this estimate should re-check the provider pricing pages against that date.

Storage and bandwidth are immaterial at any plausible pilot scale. A paired record occupies on the order of 1 KB including index overhead, so a farm logging five activities a day generates roughly 1.8 MB a year, and the 50 GB volume included with an entry-level instance would hold tens of thousands of farm-years. The application bundle crosses the network once per device because it is served from the precache thereafter, and API responses are measured in kilobytes.

4.11.4 Cost per farm at scale

Table 4.23: Reasoned cost per farm served

Farms per instance	NGN / farm / month	Assumption
50	₦419	Early pilot
100	₦210	Small cooperative
200	₦105	Conservative single-instance capacity
500	₦42	Upper bound before single-instance limits bind

Table 4.23 gives the reasoned cost per farm at four tenancy sizes. Because cost is dominated by a fixed monthly instance charge rather than by per-farm consumption, cost per farm falls almost linearly with the number of farms served. That is the

structural finding of this strand, and it bears directly on the problem stated in Chapter One: the platform’s cost profile is the opposite of per-seat licensing.

These tenancy figures are reasoned, not measured. They assume the low write rate characteristic of manual data entry, and **no concurrent-load, stress or soak testing was performed.** The single-instance architecture and the platform’s unresolved query-efficiency issue on the ledger listing would bind well before hardware capacity did. The estimate also excludes staff time, a managed database service, observability, secret management, any data-residency requirement, messaging aggregator fees, and the farmer’s own data cost — the first of which would dominate total cost of ownership in any real deployment.

4.12 Security, Authorisation and Deployment Verification

Objective 3 is answered by mechanisms whose evidence has not so far been reported in this chapter. This section reports it. Every result below was obtained by probing the running production composition at the baseline commit, not by reading source; each is therefore manually inspected evidence of a live system, supported in most cases by named automated tests within the 422 collected in Section 4.3. Two hundred and one of those 422 cases belong to the authorisation, input-validation, rate-limit, logging-safety, share-lifecycle and equipment-correction modules that this section reports.

Table 4.24: Security and authorisation verification against the running stack

Property verified	Probe	Result
Role-based authorisation	Live matrix over 3 roles and 12 named permissions on the running stack	Each role admitted exactly its permitted routes; the 403 body names only the

		missing permission
Privilege ceiling	POST /dss/train issued as the farm owner	403 — no role may retrain the model
Per-farm isolation	Cross-farm read on the enterprise-economics and bioprocess detail routes	404, not an empty result, so the boundary is not an existence oracle
Login throttling	Repeated failed logins against one account	10 failures returned 401; the 11th returned 429 with Retry-After: 896. Budget is 10 failures per 900 s; the refusal names no account
Share-token expiry	Token lifetime inspected against its creation time	90-day expiry, verified to the second
Share-token storage	Database inspection	Stored only as a hash; no clear-text token at rest
Share-token misuse	Token presented as an Authorization bearer credential	401 — the token is accepted only as a path credential
Share-token revocation	Revoked, unknown and expired tokens each requested	All three returned an identical 404

Unauthenticated report	Public investor report fetched with a live token	200, returning the netted figures
Credential hygiene	Raw token searched for across all three request logs	Absent from every log; replaced by a stable 12-hex non-reversible fingerprint that still correlates requests
Monetary input bounds	Amounts of $-1,000,000$, $2,000,000,000$, Infinity and NaN submitted	422 on each; a valid amount returned 201
Equipment correction	PATCH on an equipment record, and a no-op PATCH	200 with a modification time stamped; the no-op left the modification time unchanged, so a non-correction is not recorded. An out-of-range depreciation rate returned 422
Ledger immutability	DELETE on a posted operational log	405 — deletion is refused outright

Table 4.24 lists each property, the probe that exercised it, and the result. Three properties in that table are design results rather than test results, and are worth stating separately. The permission check precedes the farm-scope check, so a user who lacks a

permission receives the same refusal whether or not the record they named exists; a refusal is therefore never an oracle confirming existence. Unknown, revoked and expired share tokens all resolve to one identical not-found response, for the same reason. And the login refusal names no account, so a failed attempt confirms no email address. Each of these is a decision to give the attacker less information than the honest user loses by it.

Two residuals are recorded rather than concealed behind the controls. The rate-limit counters are held in process: they do not span replicas and they do not survive a restart, and both were observed directly. And there is no session revocation within a token's lifetime, no refresh mechanism, no password reset and no second factor. The boundary these results establish is that the mechanisms behave as designed under the probes applied to them by their author. No independent security assessment, penetration test or threat model was conducted, and that is the single most important thing this thesis does not know about its own security posture.

Table 4.25: Deployment, migration and recovery verification

Check	Result
Production composition	3 services started and reported healthy
Live probes	7 of 7 passed
Container privilege	Both application containers run non-root (backend uid 1000, frontend uid 101)
Database exposure	The database container publishes no host port

Configuration	The production composition renders from <code>.env.example</code> alone; no signing-key literal is committed and no environment file is tracked; the optional TLS edge profile renders
Migration chain	9 revisions, single head <code>b9e5f30c74a1</code> , every revision defining both directions
Migration against PostgreSQL	12 of 12 passed against PostgreSQL 15.18, including the schema-versus-models drift check and a rollback-and-reapply
Backup and restore	Executed with the project's own scripts: a 28,164-byte dump, 8 tables with data, every row count matched on restore, <code>alembic_version</code> <code>b9e5f30c74a1</code> , 0 unpaired operational logs, the live database untouched
TLS for a real domain	Not performed. Automatic certificate issuance has never run against a real hostname
Observability	None. No metrics, no tracing, no alerting; containers log to standard output only

Continuous integration	No run result is recorded for the baseline commit
------------------------	---

Table 4.25 records the deployment, migration and recovery checks and their outcomes.

The distinction that these results establish, and that must be carried into Chapter Five, is between a deployment path that has been demonstrated and a deployment that has been performed. This work has the first and not the second. The production composition builds, starts and answers its probes; the migration chain runs and rolls back against a real PostgreSQL server; a backup was taken and restored with every row count matching and the paired-write invariant intact. No instance serves a real user, no certificate has been issued for a real hostname, and there is no observability of any kind — no metrics, no tracing, no alerting, the containers logging to standard output only. Nor is any continuous-integration result recorded for the baseline commit: the workflow is defined and the substance of its jobs was executed locally, which is what Section 4.3 reports, and no claim that the pipeline passed is made anywhere in this work.

4.13 Usability Evaluation

Section 3.8.4 specifies a usability evaluation combining observed task performance, heuristic inspection against Nielsen’s ten heuristics, and the System Usability Scale, with a panel of three to five evaluators.

The instrument for this strand exists and the results do not. The project holds a complete facilitator run-sheet and a three-part evaluation pack — a task script, a heuristic checklist and the ten-item SUS questionnaire — and the frozen evidence base contains no completed instrument of any kind: no scored SUS forms, no filled heuristic checklists, no session notes, no participant records and no scans or photographs. This

section is therefore drafted to receive real data and reports no figure that no retained instrument supports.

Table 4.26: Usability evaluation — to be completed from retained instruments

Measure	Evaluator 1	Evaluator 2	Evaluator 3	Panel
Tasks completed	—	—	—	—
Mean ease rating (1–5)	—	—	—	—
Session duration	—	—	—	—
Mean heuristic compliance (1–5)	—	—	—	—
SUS score (0–100)	—	—	—	—

Author’s note — what must exist before each cell is filled. SUS is scored by the standard procedure: odd-numbered items score (response – 1), even-numbered items score (5 – response), the ten adjusted scores are summed to a total out of 40, and the total is multiplied by 2.5 to give a score from 0 to 100. It is not a percentage. Report the mean **with the individual scores beside it** and label it indicative rather than statistically representative, since SUS is understood to be unstable below roughly a dozen respondents and the specified panel is three to five. Merge the issue logs, deduplicate, and keep the highest severity any evaluator assigned to each issue. Photograph or scan every completed sheet: these are the raw instruments an examiner may ask to see. State the panel at its true size — a panel smaller than the method specifies is a limitation, and a panel reported larger than it was is misconduct.

Table 4.26 is the instrument prepared to receive the panel’s scores and Table 4.27 the issue log that would accompany them; both are deliberately empty, and neither has been populated. Two procedural points are recorded now because they belong to the method rather than to the results. First, browser site data must be cleared between evaluators. This is a validity control rather than a courtesy: the offline queue task deliberately creates queued entries, and the identity partitioning of that queue — verified in Section 4.3.3 — protects against cross-account flushing within the application, but a shared browser profile would still carry one evaluator’s session state into the next. Second, the heuristic checklist and the SUS questionnaire must be completed without the author present, because a self-report instrument administered under the author’s observation is systematically inflated.

Table 4.27: Usability issue log — to be completed

#	Finding	Source	Severity	Disposition
1	—			

Because this strand is outstanding, Objective 4 is reported as partially met in Section 4.14, and no claim of a usability finding, a task-completion measurement, a SUS score or a comparison against paper-based practice appears anywhere in this thesis.

4.14 Results Against the Stated Objectives

Table 4.28: Evidence and residual gap, by objective

Objective	What the evidence shows	Evidence class	Residual gap
1. Identify and evaluate the socio-technical barriers hindering FMIS adoption in Nigeria	The barriers identified in Chapters One and Two — fragmentation, interoperability, complexity, connectivity and cost — were carried into the non-functional requirements of Section 3.3.2 and answered by specific implemented	Implemented; numerical (Sections 4.2, 4.10, 4.11)	Partially achieved. The barriers are established from the literature and answered by design; no empirical barrier study with Nigerian farmers was conducted, and none is claimed. The feature-phone channel that would address the rural device gap is not implemented in any

Objective	What the evidence shows	Evidence class	Residual gap
	mechanisms: the paired write against fragmentation (109 logs, 109 transactions, 0 unpaired), the offline queue and service worker against connectivity, and the fixed-instance cost profile against cost		form
2. Convert operational and financial records into actionable decision-support metrics	Nine deterministic quantities computed from a live ledger and each pinned by named tests; the decision-support and enterprise-economics services at 100% statement coverage; all six enterprise endpoints	Automatically verified; numerical (Sections 4.6, 4.7)	Achieved as arithmetic. Demonstrated on one seeded farm rather than a season of real records, so the agronomic relevance of the figures is unevaluated. The forecast tier is

Objective	What the evidence shows	Evidence class	Residual gap
	reachable in the interface; two independent partial-budget implementations agreeing on eight hand-computed cases		trained on synthetic data
3. Design and integrate a secure data-sharing framework to build stakeholder trust	Signed-token authentication over hashed credentials; role-based authorisation over 3 roles and 12 permissions with the permission check preceding the scope check; per-farm isolation returning 404; share tokens revocable, expiring at 90 days, hashed at rest, refused as	Automatically verified and probed live (Sections 4.3.3, 4.12)	Achieved as implementation and internal verification. Not validated in practice: no independent security assessment, penetration test or threat model. No session revocation, refresh, password reset or second factor. The ledger is audit-trailed, not cryptographically

Objective	What the evidence shows	Evidence class	Residual gap
	bearer credentials, and resolving unknown, revoked and expired to one identical response; raw tokens absent from all three logs; login throttling; monetary input bounds; deletion refused with correction by linked contra entry		tamper-evident
4. Evaluate usability, cost-effectiveness and operational viability under Nigerian constraints	Three of four specified strands executed: functional verification (422 collected backend cases, 418 passed, 4 skipped, 0 failed, at 96% statement coverage, with 148 frontend tests at	Numerical; manually inspected (Sections 4.3–4.5, 4.10–4.12)	Partially achieved. The usability strand was specified and not executed (Section 4.13), so no SUS score, heuristic rating or task-completion measurement exists. Performance is

Objective	What the evidence shows	Evidence class	Residual gap
	43.18%, plus three sets of negative controls and a migration chain run against real PostgreSQL); performance benchmarking across two network conditions with cold and warm cache and driven-browser attribution; and a hosting-cost analysis anchored on a measured footprint		simulated throttling against a local server; tenancy figures are reasoned, not load-tested; hosting cost is estimated, not incurred; and the deployment path has been exercised but never run for a real user
5. Model post-harvest drying and couple its output to the financial record	The Page and Newton models recovered from a designed fixture to within 1×10^{-4} ; water removed held distinct from	Automatically verified; numerical (Section 4.8)	Achieved as a demonstrated computation on seeded runs. No drying trial was conducted with real produce, no dryer

Objective	What the evidence shows	Evidence class	Residual gap
	process loss by a test that fails if they are ever equated; safe storage returned as an absence rather than a negative verdict for an unrecognised crop; and a maize run of 100 kg at 25% moisture dried to 13% and weighed out at 84 kg moving unit cost of production from ₦35.00 to ₦41.67 per marketable kilogram, computed end to end without manual intervention		was instrumented, and the thin-layer models themselves are long established and are not claimed as a contribution — their coupling to the financial record is

4.15 Limitations of the Results

Table 4.28 states, for each objective, the evidence obtained, its evidential class and the residual gap. The following bound what the results above may be taken to show. They

are stated here so that Chapter Five interprets them rather than rediscovering them, and they are classified so that four different kinds of shortfall are not read as one. An implementation limitation is something the system does not do. An evaluation limitation is something the system may well do that this study did not measure. Future work is a reasonable extension. Out of scope is a boundary deliberately drawn in Chapter One. The distinction matters: the absence of a usability study does not make the interface unusable, the absence of load testing does not show the platform cannot scale, a synthetic evaluation does not make the model worthless, and the absence of a security assessment does not mean authentication and authorisation do not exist.

Application defects — implementation limitations. Two are known, and three carried by earlier drafts are withdrawn or closed. The two that stand: a mistaken reversal cannot itself be rolled back, only compensated by a further unlinked entry, which leaves an audit trail that is truthful but not self-explanatory to a lay reader; and a stale-revalidation window in the service worker is reasoned from the handler's ordering and has never been reproduced, provoked or observed, so it is recorded as a hypothesis rather than as a defect. Withdrawn is the report that per-crop decision support fails to net reversals: it nets them, as Section 4.6.4 establishes by execution at two commits. Closed are the unbounded monetary fields, which are now rejected at the schema edge with HTTP 422, and the missing equipment edit path, which now exists and is verified. Two further implementation limitations are added: per-operation equipment attribution is captured but costed nowhere, so no machine-hour, utilisation or cost-per-hour figure exists anywhere in this work; and the bioprocess summary endpoint has a typed client method and no caller.

Verification gaps — evaluation limitations. The yield model is not validated against any real observation, as set out in Section 4.9; what is established there is

that the pipeline is correct and reproducible, not that its predictions are accurate in the field. There are no component tests for the page and form layer, including the drying-curve chart. The local-database migration is proven against an in-memory IndexedDB implementation rather than a real browser, so browser-specific storage eviction, quota behaviour and vendor bugs are out of its reach. Backend tests run on SQLite while production runs PostgreSQL; only the migration chain was executed against a real PostgreSQL server. Concurrency is measured on SQLite and inferred for PostgreSQL. There is no end-to-end browser test of the offline-to-online transition, the service-worker lifecycle or the installation path, and the one driven-browser probe was taken at an earlier commit. The frontend suite is not timing-robust under machine contention, as Section 4.3.4 reports. There has been no independent security review, no penetration test, no threat model, and no load, stress or soak testing. And no continuous-integration run result is recorded for the baseline commit.

Data-state limitations. Every figure derives from one seeded demonstration farm. The live database holds eleven unidentified exploratory farms from development, deliberately left in place rather than deleted on an assumption, and two farms share the name “Demo Farm” — only farm 26 carries the seeded data, which is why no figure in this chapter is taken from a farm-list view. Machine hours are populated on three of eight mechanisation logs. The yield model is trained on synthetic data and no real farmer record has ever entered it, so every forecast inherits that. Two components of the hosting footprint — the static server and the container runtime — were not sampled at all, so the total is part-measured and part-assumed, and the 145.5 MiB startup figure is a lower bound on the peak

rather than the peak, because the widest sampling gap falls across the steepest part of the memory climb.

Documentation gaps. Two citations that were outstanding in the implementation are now resolved and are attributed in Chapter Two: the cost-behaviour taxonomy to the enterprise-budgeting literature of agricultural economics (Kay, Edwards, & Duffy, 2016), and the safe-storage moisture ceilings to post-harvest handling guidance (FAO, n.d.; Baidhe et al., 2024).

Validation gaps. There has been no field trial and no real farmer data. There has been no user evaluation, no task-completion measurement and no comparison against the paper-based practice this work is positioned against. The decision-support outputs are verified as arithmetic and unvalidated as agronomic advice — whether following them improves an outcome is untested and, on synthetic training data and one seeded farm, untestable here. And nothing here is validated in practice: the production path of Section 4.12 has been built, started, probed, backed up and restored, and no instance has ever served a real user on a real domain on a departmental host. That is a smaller and more accurate claim than the flat statement that the system has never been deployed, and it is the one the evidence supports.

4.16 Summary

The platform’s record-keeping core is verified at the baseline commit by a backend suite of 422 collected cases, of which 418 pass, 4 skip and none fail, at 96% statement coverage over 1,594 statements with 66 missed; the four skips are PostgreSQL-only migration tests, which were separately executed 12 of 12 against PostgreSQL 15.18. The frontend suite comprises 148 tests across 13 files at 43.18% statement coverage

over the whole of src, one of those tests not being timing-robust in a full-suite run under machine contention. The two coverage figures must be read together. The type check, the lint and the production build are clean. The paired write, the soft-immutability constraint and the linked contra entry behave as designed; offline queue identity partitioning, logout cleanup and the local-database migration are each asserted by named tests; and same-farm idempotency holds sequentially and under a genuine two-thread race. Coverage is concentrated where it matters — every module that serves a request is fully or almost fully covered — and its distribution is reported rather than averaged away. Three sets of negative-control runs, executed at an earlier commit against logic the merge did not change, establish that these tests are load-bearing.

The deterministic decision-support layer computes per-crop margin and ranking, unit cost on both a harvest and a marketable basis, retrospective break-even yield, both break-even prices, a sensitivity matrix, an operating-expense ratio, an Olympic-average yield baseline and a partial budget from a live ledger, netting reversed pairs at farm level and per crop alike, and withholds each figure where its inputs are insufficient rather than substituting a plausible default. All six enterprise endpoints are reachable in the interface. Two independent implementations of the partial budget agree on eight hand-computed cases. The post-harvest module recovers known thin-layer parameters — $k = 0.30$ and $n = 0.75$ — to within one part in ten thousand, keeps water removed distinct from matter lost, and connects a moisture measurement to a price per marketable kilogram in a single computed chain, moving maize from ₦35.00 to ₦41.67 per kilogram, a 19% difference that is post-harvest mass reduction expressed in money.

Under simulated throttling re-measured at the baseline, the service worker reduces network transfer on a repeat visit from 139,245 bytes over eight requests to 127 bytes over seven — the residue being the manifest icon, a measured defect — and reduces

first paint by a factor of about eighty, from 5,661.2 ms to 70.6 ms, though time to main content improves by a factor of about three and a half, and that is the figure a user experiences. Measured resource consumption places the two exercised containers at 178.9 to 202.8 MiB, supporting an estimated \$15.40, approximately ₦20,950, per month and a reasoned cost per farm falling to ₦105 at a two-hundred-farm tenancy — a cost structure that scales in the opposite direction to the per-seat licensing identified in Chapter One as excluding this user.

The security and tenancy boundary is exercised as well as asserted: three roles over twelve named permissions refuse a worker the farm's financial picture and refuse a manager the power to disclose it; share links expire at ninety days, are stored only as hashes, and revoke to a verdict indistinguishable from that for a token that never existed; the raw token reaches none of the three logs that record its request; failed logins are throttled; the migration chain runs and rolls back against real PostgreSQL; and a backup was taken and restored with every row count matching and the paired-write invariant intact. Against all of that, the yield forecast is an in-distribution result on synthetic data — R^2 0.9762, MAE 0.2414 t/ha, RMSE 0.4883 t/ha, reproducible to six decimal places and evidence about the pipeline rather than about Nigerian yields; the platform has a production path that has been demonstrated but has never served a real user; no independent security assessment exists; and the usability strand was specified and not executed, leaving Objective 4 partly met. Chapter Five interprets these results against the problem set out in Chapter One, states the platform's limitations in full, and sets out what would be required to take it from a verified prototype to a deployed system.

CHAPTER FIVE

5.0 SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

5.1.1 The Problem, the Approach, and the Procedures Used

This study began from a documented separation. Nigerian smallholder and medium-scale farms keep their production records and their cost records apart — commonly on paper, commonly in different books — so that the cost of producing a crop is rarely known at the moment the crop is sold. The farm management information systems that would close that gap were designed for a different farm: they assume continuous connectivity, they charge per user seat, and they present a complexity that the literature reviewed in Chapter Two identifies as a barrier in its own right. The specific consequence, and the one this work exists to address, is that a farmer who cannot join an operational record to its financial consequence cannot know what any enterprise on the farm actually earns.

The approach was to remove the separation rather than to manage it. Where an integrative design synchronises two records, AGRI-PROFIT posts one: every operational log commits its paired financial transaction in the same database transaction, and a posted record is corrected not by editing it but by a linked contra entry that preserves the original's category and crop. On that record the platform computes a deterministic decision-support tier — nine quantities derived from the farm's own ledger with no model and no external data — and, separately and optionally, a forecast tier. A post-harvest drying module applies established thin-layer kinetics to the farm's own drying runs and carries their marketable output into the unit-cost calculation, so that what a farmer costs against is the mass that can actually be sold. The client is an

offline-first progressive web application; sharing with a lender is a revocable, expiring, read-only token.

The procedures used were those specified in Chapter Three and reported in Chapter Four. Development was iterative and version-controlled, with human-reviewed, agent-assisted implementation. The evaluation was specified in advance as four strands: functional verification by automated test suite with hand-computed fixtures and negative controls; performance benchmarking by Lighthouse under mobile emulation and simulated network throttling; a hosting-cost analysis anchored on a measured container footprint; and a heuristic-plus-System-Usability-Scale evaluation with a small expert panel. Three of the four were executed. The fourth, usability, was specified, its instruments were prepared in full, and it was not carried out. A fifth body of evidence, not among the four strands as originally specified, was obtained by probing the running production composition directly and is reported in Section 4.12. All evidence in this work is measured at a single implementation state: commit 78a68c292205acdcac1a52f977fbea31b6e8495e on the main branch, dated 29 August 2026, referred to as the baseline commit. Where a measurement was taken at an earlier commit it is attributed at the point of use.

Two commitments carried over from Chapter Four govern this chapter as well. Nothing is interpreted that was not first evidenced: where an interpretation rests on a result, the result is the one reported in Chapter Four and not a stronger version of it. And where the evidence does not reach a conclusion the reader might expect — most consequentially on whether the platform would help a real farmer — that is stated as a limit on the work rather than bridged by argument.

5.1.2 Summary of Findings

The evaluation returned findings in seven areas.

Structural integrity of the record. At the baseline commit the live database held 109 operational logs and 109 financial transactions, and a direct query for operational logs carrying no paired transaction returned zero rows. The one-to-one correspondence is not a property of the sample but the paired-write invariant made visible in the data, and it is the single most quotable structural result of the work.

Functional verification. The backend suite collects 422 tests, of which 418 pass, 4 are skipped — PostgreSQL-only migration tests, separately executed 12 of 12 against PostgreSQL 15.18 — and none fail, at 96% statement coverage over 1,594 statements with 66 missed. The frontend suite comprises 148 tests in 13 files at 43.18% statement coverage over 558 of 1,292 statements, measured across the whole of the source tree including page and form components that carry no tests at all. The two coverage figures must be read together; 96% quoted alone misrepresents the system. One frontend test is not timing-robust in a full-suite run under machine contention, which is a limitation of the harness and not a defect in the application. Three sets of negative-control runs establish that the tests fail when the behaviour they guard is broken.

Decision support. The deterministic tier computes nine quantities from the farm's own ledger with no model and no external data, and all six enterprise-economics endpoints are reachable in the interface. The decision-support and enterprise-economics services both stand at 100% statement coverage. The platform withholds a figure wherever its inputs do not support one, and 88.25% of recorded cost at farm level carries a cost subtype, the residual 11.75% being reported as unclassified rather than distributed into a plausible bucket.

Post-harvest coupling. A maize harvest of 100 kg at 25% moisture, dried to 13% and weighed out at 84 kg against a dry-matter prediction of 86.21 kg, moved the unit cost of production from ₦35.00 to ₦41.67 per marketable kilogram — a 19% difference, being post-harvest mass reduction expressed in money. The Page and Newton models were recovered from a designed fixture at $k = 0.30$ and $n = 0.75$ to within one part in ten thousand.

Forecasting. A 200-estimator Random Forest over 6,000 synthetic samples returned R^2 0.9762, MAE 0.2414 t/ha and RMSE 0.4883 t/ha, reproducible to six decimal places across two Python versions from a dataset carrying a recorded SHA-256 fingerprint. That figure measures the recovery of a constructed relationship. It is evidence that the machine-learning pipeline is correctly implemented end to end; it is not evidence about Nigerian yields, and the model learns from no farm's records.

Operation under constraint. Under simulated slow-3G throttling a cold load transfers 139,245 bytes over eight requests and reaches first paint at 5,661.2 ms; a warm load transfers 127 bytes over seven requests and reaches first paint at 70.6 ms. Two exercised containers consume 178.9 to 202.8 MiB, supporting an estimated \$15.40, approximately ₦20,950, per month and a reasoned ₦105 per farm per month at a two-hundred-farm tenancy.

Security and the deployment path. Role-based authorisation over three roles and twelve named permissions was exercised live, with the permission check preceding the farm-scope check so that a refusal is never an existence oracle; login throttling returned 429 on the eleventh failure within the window without naming an account; share tokens were verified to expire at ninety days, to be stored only as hashes, to be refused as bearer credentials, and to resolve unknown, revoked and expired alike to one identical

response; the raw token reached none of the three logs recording its request; monetary bounds rejected four out-of-range inputs; and a backup was taken and restored with every row count matching. The production composition was built, started and probed with seven of seven checks passing. No instance has served a real user.

The usability strand of the specified evaluation was not executed. No System Usability Scale score, heuristic-severity rating, task-completion measurement or evaluator finding is reported anywhere in this work.

5.1.3 Achievement Against the Research Objectives

Table 5.1 states the verdict on each objective and the evidence that supports it. The verdicts are used strictly: achieved, partially achieved, and demonstrated as prototype, the last denoting a capability that is implemented and verified but whose value in use has not been established. No objective is forced to achieved, and an honest partial achievement is preferred to an overstated one.

Table 5.1: Objective-to-conclusion matrix

Objective	Chapter 3 design	Chapter 4 evidence	Verdict
(i) Identify and evaluate the socio-technical barriers hindering FMIS adoption in Nigeria	§3.3.2 non-functional requirements (a)–(e); §3.6.1 offline logbook; §3.6.8 feature-phone channel, designed	§4.2 functional inventory; census 109 / 109 / 0 unpaired; offline-queue and idempotency tests within the 422;	Partially achieved. Achieved as identification and design response; not achieved as empirical evaluation. The

	and not implemented	§4.10 performance; §4.11.4 cost per farm	largest barrier — the 68% rural device gap — is answered by a mechanism that was designed and not built
(ii) Translate operational and financial records into actionable decision-support metrics	§3.5 paired write and contra entry; §3.6.2 ledger engine; §3.6.5 two-tier engine, nine Tier-1 quantities; §3.6.10 enterprise economics	§4.6 per-crop margin, dual unit cost, break-even yield; §4.7 cost structure, overlay, dual break-even price, sensitivity, partial budget, yield baseline; both services at 100% coverage	Achieved as arithmetic, on one seeded farm. The agronomic value of the figures is unevaluated
(iii) Design and integrate a secure data-sharing framework for stakeholder trust	§3.6.6 authentication and farm boundary; §3.6.7 share tokens; §3.7 security design —	§4.3.3 capability-to-proof mapping; §4.12 live RBAC matrix, throttling, token lifecycle, log scrubbing, input	Achieved as implementation and internal verification; not validated in practice. No

	authentication, RBAC, isolation, throttling, credential hygiene, scoped sharing	bounds, immutability, backup and restore	independent assessment; no session revocation, refresh, reset or second factor
(iv) Evaluate usability, cost-effectiveness and operational viability under Nigerian constraints	§3.8 four specified strands; §3.8.4 usability, specified and not executed; §3.8.5 designed field evaluation	§4.3–§4.5 verification and negative controls; §4.10 performance; §4.11 hosting cost; §4.12 deployment path; §4.13 usability, reporting no figure	Partially achieved. Three of four strands executed. No usability evidence of any kind exists
(v) Model post-harvest drying and couple its output to the financial record	§2.7 literature basis; §3.6.9 post-harvest drying module; §3.3.1 functional requirement	§4.8 fixture recovery at $k = 0.30$ and $n = 0.75$, the seeded maize run, input validation, and the coupling to §4.6.2 unit cost	Demonstrated as prototype. Computed end to end and verified by designed fixture; no real drying trial, no instrumented dryer, two seeded runs

Objective (i) — the socio-technical barriers. The barriers were identified from the literature and carried into the design as binding requirements rather than as background. Four became non-functional requirements in Section 3.3.2 and can now be traced to specific implemented mechanisms. Fragmentation between operational and financial records (Poppe, Vrolijk, & Bosloper, 2023) is answered structurally by the paired write: 109 operational logs and 109 financial transactions with none unpaired, a correspondence that is the invariant made visible rather than a property of the sample. Intermittent connectivity (Mhlanga & Ndhlovu, 2023) is answered by the offline queue, whose identity partitioning, logout cleanup, three-strike retry and schema migration are each asserted by named tests, and by a service worker shown by a driven-browser probe at commit c16924e on 17 August 2026 to serve the whole application shell with the network disconnected, against service-worker configuration the subsequent merge did not change. Cost as an exclusion mechanism is answered by an architecture whose cost profile is a fixed monthly instance charge rather than a per-seat licence, giving a reasoned ₦105 per farm per month at a two-hundred-farm tenancy. User complexity is answered by the deliberate reduction of raw ledger data to a small number of ranked, satisficing figures, in the sense of the bounded-rationality argument of Section 2.2.2. The objective is met as identification and design response and not as empirical evaluation: no study of Nigerian farmers was conducted. That distinction matters most for the barrier that is quantitatively largest. The rural smartphone gap reported at 68% (Jaiyeola, 2023) is the headline socio-technical constraint of Chapter One, and the mechanism designed to address it — the feature-phone entry channel of Section 3.6.8 — is not implemented in any form in the submitted build: there is no route, no webhook, no parser, no sender-to-farm mapping, no test and no simulated harness. The platform therefore addresses the barriers facing farmers who already hold a smartphone

and an intermittent connection, which is the 32% rather than the 68%. That is a real limit on the reach of the contribution and it is stated plainly rather than folded into a claim about inclusion.

Objective (ii) — converting records into decision-support metrics. This is the objective on which the evidence is strongest. The deterministic tier computes, from the farm's own ledger and with no model and no external data, per-crop gross margin and ranking, unit cost of production on both a harvest and a marketable basis, retrospective break-even yield, cost structure by behaviour with an explicit classification-coverage figure, a derived depreciation overlay and its proportional allocation, two break-even prices, a five-row yield sensitivity matrix, an operating-expense ratio, a partial budget and an Olympic-average yield baseline. The decision-support service and the enterprise-economics service both stand at 100% statement coverage; every behaviour on which the design argument depends is asserted by a named test; and the partial budget — implemented twice, so that it survives offline — agreed with an independent hand-computed fixture in both implementations across eight cases including a sign trap, an exact-zero trap and a transposition trap. One qualification bounds the achievement and two that earlier drafts recorded are withdrawn. The qualification that stands is that the figures are demonstrated on one seeded farm rather than over a season of real records: the arithmetic is verified, the agronomic relevance is not. Withdrawn are the claims that the per-crop breakdown fails to net reversals — it nets them, as Section 4.6.4 establishes by execution at two separate commits — and that the Olympic-average baseline has no interface surface, which ceased to be true when its panel was mounted in the enterprise-economics view. Both were carried in good faith from an earlier audit and neither survives contact with the running system.

Objective (iii) — a secure data-sharing framework for stakeholder trust. The mechanism is built and its boundary properties are verified, and the evidence is materially larger than earlier drafts of this chapter reported. Authentication issues a signed token over hashed credentials. Authorisation is role-based over three roles and twelve named permissions, with the permission check applied before the farm-scope check, so that a refusal reports a missing permission and never doubles as an oracle confirming that a record exists; the model is refused retraining even to the farm’s owner. Every read and write path is scoped to the authenticated user’s farm, a cross-farm read returning HTTP 404 rather than an empty result. Failed logins are throttled at ten within a nine-hundred-second window, the eleventh returning 429 with a Retry-After header and naming no account, so the refusal confirms no email address. Share tokens are revocable, expire at ninety days, are stored only as hashes, are refused when presented as bearer credentials, and resolve unknown, revoked and expired alike to one identical 404. The raw token appears in none of the three logs that record its request, replaced by a stable twelve-character non-reversible fingerprint that still allows requests to be correlated. Monetary inputs are bounded at the schema edge. And the ledger refuses deletion outright, requiring a category-preserving contra entry linked to its original, with double reversal and cross-farm tampering both rejected. That last property is the substantive answer to the question an examiner or a lender would actually ask — how does a bank know the farmer did not simply edit the numbers? The answer is that the numbers cannot be edited, only offset by a further recorded entry that remains visible beside the original. It must be stated with precision, however, that this is audit-trail discipline enforced at the application layer, not cryptographic tamper-evidence. There is no hash chaining, no signing and no append-only log; a party with database access is not constrained by it. Chapter Two reviewed distributed-ledger governance as the mechanism that would

provide that stronger guarantee (Xiong et al., 2020), and Chapter One correctly scoped it as a future direction. Three residual gaps qualify the objective, and one previously recorded gap is withdrawn. The boundary is proven by tests and by live probing of the running stack, not by an independent security assessment; no penetration test, review or threat model was conducted, and that is the single most important thing this thesis does not know about its own security posture. Identity management is incomplete in ways that matter to a real deployment: a leaked token stays valid until it expires, and there is no revocation within a lifetime, no refresh, no password reset and no second factor. The rate-limit counters are held in process, so they neither span replicas nor survive a restart, both of which were observed. Withdrawn is the claim that the investor-facing report inherits a per-crop reversal defect: there is no such defect, and the report was fetched with a live share token after a reversal and returned the netted figures.

Objective (iv) — usability, cost-effectiveness and operational viability. It is partially achieved. Three of the four specified strands were executed. Functional verification returned 422 collected backend cases, of which 418 passed, 4 skipped and none failed, at 96% statement coverage over 1,594 statements with 66 missed, together with 148 frontend tests across 13 files at 43.18% statement coverage — the two coverage figures being quoted together because 96% alone would misrepresent the system — strengthened by three sets of negative-control runs establishing that those tests fail when the behaviour they guard is broken, and by a migration chain executed against a real PostgreSQL server. Performance benchmarking, re-measured at the baseline commit, established that the application is bandwidth-bound below roughly 1 Mbps and that a repeat visit transfers 127 bytes over seven requests rather than nothing at all — the residue being the manifest icon, a measured defect reported in Section 4.10.4 — with service-worker attribution proven by driven browser rather than inferred. The hosting-

cost analysis, anchored on a measured footprint of 178.9 to 202.8 MiB across the two exercised containers, produced an estimated \$15.40, approximately ₦20,950, per month at the Central Bank of Nigeria rate of ₦1,360.58 to the dollar as at 12 August 2026. Two qualifications travel with that total: it is part-measured and part-assumed, because the static server and the container runtime were not sampled at all, and the 145.5 MiB startup figure is a lower bound on the peak rather than the peak, because the widest sampling gap falls across the steepest part of the memory climb. The usability strand is outstanding. The instrument exists in full and no completed instrument does, so Chapter Four reports no usability figure. This is the single largest gap in the evaluation, and it is consequential rather than cosmetic: the platform's central claim is that it reduces cognitive load for a user operating under bounded rationality, and that claim is precisely the one no strand of the completed evaluation touches. A platform that computes correct figures which a user cannot find, or cannot interpret when found, has not met Objective (ii) in the sense that matters to a farmer, and nothing in the executed evaluation would detect that condition. Each executed strand also carries its own limit. The performance figures are simulated throttling against a local server on a single machine — a model of a degraded network, not a measurement of one. The cost figure is an estimate, not an incurred charge; the per-farm tenancy figures are reasoned and no load testing was performed. And functional verification, however extensive, demonstrates only that the behaviours somebody thought to test behave as specified.

Objective (v) — post-harvest drying coupled to the financial record. The objective is achieved as a demonstrated computation and is bounded as a prototype. The module recovers the Page and Newton parameters of a designed fixture exactly, keeps water removed distinct from matter lost by a test that fails if the two are ever equated, refuses a fit below three readings, drops readings outside the physical moisture band, and

returns an absence rather than a negative verdict where no safe-storage threshold is recorded for a crop. It carries a moisture measurement through to a price per marketable kilogram without manual intervention at any step, and the readings that the Page fit requires can now be entered through the interface. What has not been done is equally definite: no drying trial was conducted with real produce, no dryer was instrumented, the two seeded runs are the only runs, and the thin-layer models themselves are long established and are not claimed as a contribution. The contribution is their coupling to the paired financial record, and that coupling is demonstrated rather than validated in use.

5.2 Conclusion

5.2.1 Interpretation of the Major Findings

The paired write is the structural contribution, and it is visible in the data. The literature identifies the separation of farm financial accounts from operational records as a historical accident with continuing costs: duplicated entry, administrative fatigue, and the impossibility of correlating agronomic inputs with financial outcome (Pope et al., 2023; Fountas et al., 2015). Most responses to it are integrative — two systems and a synchronisation path between them. AGRI-PROFIT’s response is different in kind: there is only one write, and it posts both records in a single commit, so the two cannot drift because they were never separate. The finding that makes this more than an assertion is arithmetic. At the baseline commit the database held exactly as many financial transactions as operational logs — 109 of each, with zero unpaired — at Alembic revision b9e5f30c74a1. There is no reconciliation step in the platform because there is nothing to reconcile, and the correlation the literature describes as unavailable is not computed but inherent: every cost already carries the activity that incurred it and the crop it was incurred against. The cost of this design is equally real and is stated in

Chapter Three: the ledger is single-entry, so it is an analytical instrument rather than a bookkeeping system in the accounting sense. That is the right trade for the target user, whose alternative is not double-entry software but paper, and it should not be presented as anything else.

The drying-to-price chain is the engineering contribution. The result most specific to agricultural and bioresources engineering is the computed chain running from a moisture measurement to a price. A maize harvest weighed in at 100 kg and 25% moisture, dried to 13%, weighed out at 84 kg, and the unit cost of production moved from ₦35.00 to ₦41.67 per kilogram. The 19% difference is post-harvest mass reduction expressed in money, and a farmer pricing on the wet-basis figure would under-recover ₦6.67 on every kilogram sold. Three properties make this a result rather than an illustration. First, the chain is computed end to end without manual intervention: the drying run is an operational log like any other, so it posts its cost through the same paired write, and its computed marketable output becomes the denominator of the unit-cost figure. Second, the denominator is the mass actually weighed out rather than the mass the dry-matter balance predicts — 84 kg against 86.21 kg — which is the conservative choice and charges the pricing decision only with grain that reached the store. Third, the distinction the module is most likely to get wrong is guarded by a test that fails if it is ever collapsed: water removed is a water balance, not a mass difference, and conflating them would make process loss and drying performance the same number. One refinement must travel with the ₦41.67 figure now that the demonstration farm owns a depreciating asset. That figure is the break-even price on a cash basis, at ₦41.666666666666664; on a total-cost basis, once maize takes its ₦58.88 share of the farm-wide depreciation overlay, the figure is ₦42.37. The two answer different questions — what must be obtained this season to avoid a cash loss, and what must be

obtained to cover the wear on the assets used as well — and neither should be quoted as though it were the other. Break-even yield, in kilograms and retrospective, is a third quantity again and is never to be conflated with either price. This junction is the answer to the question of what makes the platform an agricultural engineering artefact rather than a bookkeeping application. The thin-layer models themselves are long established and are not the contribution; their coupling to the financial record is.

Withholding a figure is a design principle, and the null cases are its evidence. A recurring pattern across Chapter Four is that the platform refuses to answer where the inputs do not support an answer. Break-even yield returns null in four distinct cases. Unit cost on a marketable basis is withheld where no drying run exists rather than defaulting to the harvest mass. Mixed harvest units return a null with a stated reason rather than an arithmetically invalid sum. An unrecognised crop returns no safe-storage threshold rather than an unsafe verdict. A zero depreciation rate counts as unrated rather than as a zero charge. An Olympic average over fewer than three seasons is refused, and the season count is reported in every branch so that a null is never read without the count explaining it. The untrained model reports itself as untrained rather than as R^2 0.0000. This is more than defensive coding. Under bounded rationality the cost of a wrong figure is higher than the cost of a missing one, because a farmer who receives a plausible number has no way to discover it was unfounded, whereas a farmer who receives a stated absence knows exactly what to do about it — record the missing data. The design decision that best illustrates this is the fourth null case on break-even yield: a crop with no attributed cost has an arithmetically valid break-even yield of zero kilograms, and reporting it would communicate a false completeness to a user whose real situation is that they have not recorded their costs. The same discipline is what makes the classification-coverage figure worth reporting. At farm level 88.25% of

recorded cost carries a cost subtype, and rather than distributing the remaining 11.75% into a plausible bucket the platform reports it as unclassified, so a reader can see how much of the analysis rests on classified data.

The service worker is the operative form of the low-bandwidth claim — and a liability. The bundle-size version of the argument is the weaker one. A 139,245-byte payload over eight requests takes 2.79 s to transfer at 400 Kbps and 0.68 s at 1,638.4 Kbps, which is why the cold performance score falls from 99 at slow-4G to 65 at slow-3G. Route-level code-splitting removed 104,479 bytes from the first load between the 17 August build and the baseline, a reduction of 43%. The stronger version, and the one the measurements support, is that the application remains usable when the network is intermittent or absent. A returning user reaches first paint in 70.6 ms and main content in 1,612.0 ms on a link whose cold first paint is 5,661.2 ms, and reaches the application at all with no connection whatsoever. That is a different claim from “the bundle is small”, and it maps onto the actual rural condition described by Mhlanga and Ndhlovu (2023) — unreliable connectivity rather than uniformly slow connectivity. The finding must be held together with its opposite, which Chapter Four also reported. A caching layer that improves first paint by a factor of about eighty is a layer that can make the application misrepresent its own state: a cached shell can make a stopped backend look like a running one, and a cached read can serve pre-reversal figures after a correction. The platform’s response is an explicit purge on every farm-scoped write and on any change of authenticated identity, and the negative-control run of Section 4.4 is what establishes that the purge does real work — with it neutered, all five tests fail on the value a user would see. A stale-revalidation window nevertheless remains reasoned but unreproduced. Both halves of that sentence belong in an honest evaluation. Two further honesties belong here. The comparison across measurement dates is bounded: the host

benchmark index differed between the two sets, so no score or blocking-time difference may be attributed to the code, and only the network-bound metrics and the byte counts compare. And the warm load does not transfer nothing: it transfers 127 bytes over seven requests, which is the manifest icon missing from the precache — a small, real, measured defect that is worth more to this chapter than the rounder claim it replaces.

The cost structure is the finding that scales. The cost analysis produced one result of structural rather than arithmetic interest. Because the platform's cost is dominated by a fixed monthly instance charge and not by per-farm consumption — storage at roughly 1 KB per paired record and a bundle that crosses the network once per device being immaterial at any pilot scale — cost per farm falls almost linearly with the number of farms served: ₦419 at fifty farms, ₦210 at one hundred, ₦105 at two hundred and ₦42 at five hundred. That is the exact structural opposite of the per-seat licensing which Chapter One identifies as excluding smallholders from existing systems. Under per-seat pricing the marginal farmer costs the same as the first; here the marginal farmer is nearly free. The implication is that the natural deployment unit for this platform is a cooperative, an aggregator or an extension service rather than an individual farm — which also happens to be the unit at which the advisory support that conditions adoption (Carrer et al., 2017) is already organised. The finding is bounded honestly in Chapter Four and the bound should be repeated here: the tenancy figures are reasoned, not measured. No concurrent-load testing was performed, and the single-instance architecture together with an unresolved query-efficiency issue on the ledger listing would bind well before hardware capacity did. The configuration priced is also one in which every restart costs 8.57 s of unavailability, driven by the train-on-boot step — an availability cost, not a memory cost.

The negative controls are the reason the verification figures should be believed. Test counts and coverage percentages are the most commonly quoted and least informative figures in a software evaluation. A suite can be large, green and worthless. The three sets of mutation runs reported in Section 4.4 are what distinguishes this evaluation from one that reports 422 and 96% and asks to be trusted: with the cache purge neutered, five tests fail — each on a value or on rendered text, none on a call count; with the local-database upgrade hook broken, four of seven fail; and the shared parity fixture cannot go missing, malformed or silently shorter without failing both suites at collection time. The most instructive detail is the discriminating case in the first experiment. After a reversal, a working purge with correct contra accounting yields ₦2,000, a surviving cache yields ₦5,500, and a lost record yields ₦0. Because the three outcomes are distinguishable, the test can only pass for the right reason. That is what a load-bearing test looks like, and it is the property a coverage percentage cannot express. The attribution must travel with the claim: these runs were executed at commit 59a6286 against frontend cache logic and fixture-loading logic that the subsequent merge did not change, and they have not been repeated at the baseline.

A recorded defect that was not one. The most instructive methodological finding of this work was not planned. An audit conducted on 25 August 2026 reported that per-crop decision support failed to net reversals and produced a phantom unclassified cost bucket. The finding was read out of source code and was never executed. It was false; it survived into three project documents and into an earlier draft of two chapters of this thesis; and it was overturned only by running the path at two separate commits, where reversing a ₦25,000 maize expense moved the crop from ₦35,000 to ₦10,000 and its unit cost from ₦2,916.67 to ₦833.33 with no phantom bucket at any point. The same audit's own summary table had listed the capability among the verified behaviours, so

the document contradicted itself and neither half was reconciled against the running system. Two of its other findings — that the Olympic-average yield baseline and two crops on the entry form were unreachable — failed in the same way. The methodological point is a genuine result of this evaluation and belongs in it as one: a defect read out of code without being reproduced is not a defect, it is a hypothesis, and an evaluation that does not distinguish the two will report its own reading as evidence. This is the reason the negative-control experiments of Section 4.4 matter beyond the specific behaviours they guard, and it is the reason every correctness claim in Chapter Four is tied to a command that was run rather than to a file that was read.

5.2.2 Comparison with the Literature

The comparison below is restricted to works reviewed in Chapter Two. Where the platform diverges from what the literature prescribes the divergence is argued rather than concealed.

On fragmentation. Poppe et al. (2023) describe farm financial accounts and farm-management records as having originated in the same note-taking practice and diverged as they computerised. Chapter Four’s finding of one financial transaction per operational log is the platform’s answer to that divergence, and it differs from the integrative approach implied by most FMIS designs (Fountas et al., 2015) in kind rather than in degree: the records are not synchronised because they are not separate.

On interoperability. Tummers, Kassahun, and Tekinerdogan (2019) identify the absence of standardised data-exchange protocols as a primary technical roadblock, and this work does not solve it. AGRI-PROFIT exports a standardised profit-and-loss statement as CSV and exposes a tokenised read-only report, which addresses the sharing problem for a human recipient; it implements no agricultural data-exchange standard and would not

interoperate with another FMIS. The contribution here is to the fragmentation problem, not the interoperability one, and conflating the two would overstate it.

On adoption conditioning. Carrer, de Souza Filho, and Batalha (2017) find that FMIS uptake is conditioned by education, advisory access and rural credit, and Giua, Materia, and Camanzi (2021) reach a compatible conclusion across a wider literature. Two of those three channels are addressed structurally rather than incidentally by findings reported in Chapter Four. The tokenised stakeholder report exists to make a farm's records legible to a lender, which speaks to the credit channel; and the per-instance cost structure makes cooperative or extension-level deployment economically natural, which speaks to the advisory channel. The third — education — is precisely the channel a usability evaluation would test, and precisely the strand that is outstanding.

On connectivity and the device gap. Mhlanga and Ndhlovu (2023) argue that rural deployment in Africa must assume degraded rather than absent infrastructure. AGRI-PROFIT is a partial instance of that argument and should be described as such: the write path and the application shell are genuinely local — a record can be captured, queued and displayed with no connection — while all computation, including the deterministic decision-support tier, executes on the server. The evidence for the local half is the driven-browser probe and the offline queue tests; the server-side half follows from the client–server architecture of Section 3.4.1. Any characterisation of the deterministic tier as running on-device would be incorrect, and the edge-computing literature reviewed in Section 2.1.3 describes a placement this platform does not adopt. Jaiyeola (2023) reports 68% of rural Nigerians without smartphones, and as Section 5.1.3 concedes, the channel that would reach them is not built. Abioye et al. (2026), studying willingness to adopt digital tools among Nigerian farmers directly, find smartphone access and the cost

of the tool among the binding determinants — the two constraints this platform respectively fails and answers.

On the modelling ladder, van Klompenburg, Kassahun, and Catal (2020) find tree-based ensembles to perform strongly on tabular agricultural data at far lower computational cost than deep networks, and the implemented forecast tier follows that finding with a 200-estimator Random Forest. Chapter One scoped the statistical tier as lightweight modelling, and the implementation went one step up the modelling ladder to an ensemble of regression trees; that is a defensible design decision rather than a drift, and it is taken on the grounds of cost and interpretability that van Klompenburg et al. establish. The counter-case is recorded rather than suppressed: Olisah et al. (2024), working on Nigerian corn-yield prediction for smallholder decision support, report that a carefully tuned deep neural network regressor outperformed the models they compared it against and that the quality of pre-processing was decisive. Their work is an unpublished preprint and is cited here for its pre-processing finding rather than as authority for a model choice; the implemented tier deliberately does not yet exercise that finding, because it is trained on clean synthetic data for which the pre-processing pipeline they describe has nothing to do. That is the correct reading of the R^2 of 0.9762 reported in Chapter Four: it is the recovery of a constructed relationship, and the moment the platform trains on real farm records the pre-processing problem those authors identify becomes the central one. All three regression measures are reported — R^2 0.9762, MAE 0.2414 t/ha and RMSE 0.4883 t/ha — and all three are reproducible to six decimal places from the recorded seed, so what is unproven is the model's relevance and not the integrity of the procedure that produced it.

On post-harvest drying. The thin-layer models the platform fits are those established by Lewis (1921) and Page (1949) and reviewed by Onwude et al. (2016); nothing in the

kinetics is new, and no claim to the contrary is made. What the reviewed literature does not do is carry those outputs into a cost record. Baidhe, Clementson, Senyah, and Hammed (2024) report that inadequate and unmeasured drying remains a principal determinant of grain quality loss across Africa, and that smallholder practice judges the end point by eye; the enterprise-budgeting literature (Kay, Edwards, & Duffy, 2016) supplies the cost-behaviour taxonomy and the unit-cost formulation but assumes the output quantity is given. This work joins the two, and the ₦35.00-to-~~₦~~41.67 result is what the junction produces. That is the contribution claimed for Objective (v), stated at its true size: a demonstrated computation on seeded runs, not a validated finding about Nigerian maize.

On trust and the ledger. Xiong, Dalhaus, Wang, and Huang (2020) present distributed-ledger technology as the mechanism by which self-reported farm data becomes credible to banks and insurers, and Gebresenbet et al. (2023) frame the underlying problem as the absence of verifiable data. AGRI-PROFIT implements the weaker, immediately deployable mechanism reviewed in Section 2.5 — authenticated, role-controlled, tokenised standardised reporting over an audit-trailed ledger. The comparison should be stated exactly: what the platform provides is a record that cannot be silently edited through the application and a report a farmer can grant, revoke and let expire; what it does not provide is cryptographic verifiability against a party with database access. The distributed-ledger layer remains a future extension, as Chapter One scoped it.

On usability method. Nielsen and Molich (1990), Nielsen (1994) and Brooke (1996) supply the instruments specified in Section 3.8.4, and the choice of an expert heuristic panel with a standardised questionnaire remains the appropriate method where a large end-user sample is unavailable. This work reports no result against those instruments, and the comparison cannot be made.

5.2.3 Limitations

The limitations below are those that most constrain what this work may claim. They restate, in interpretive form, the results limitations of Section 4.15, and they are separated by kind because four different sorts of shortfall are otherwise read as one. An implementation limitation is something the system does not do. An evaluation limitation is something the system may well do that this study did not measure — the absence of a usability study does not make the interface unusable, the absence of load testing does not show the platform cannot scale, a synthetic evaluation does not make the model worthless, and the absence of a security assessment does not mean authentication and authorisation do not exist. Future work is a reasonable extension. Out of scope is a boundary deliberately drawn in Chapter One.

Scope and deployment — implementation. The platform has not been deployed to serve real users, and no certificate has been issued for a real departmental domain. What exists, and was exercised for this evaluation, is a production deployment path: a composition that builds and starts with every service healthy, a frontend proxying the API with its path intact, schema migration applied on startup, liveness and readiness probes answering from the application rather than from the page shell, unprivileged application containers, a database publishing no host port, environment guards that refuse to start on a default signing key, an optional automatic-TLS edge that renders, and backup and restore scripts that were run end to end against a real PostgreSQL server with every restored row count matching its source. The distinction that matters is between a deployment path that has been demonstrated and a deployment that has been performed; this work has the first and not the second. Production observability — metrics, tracing and alerting — is genuinely absent, and it is the one element of the four an earlier draft of this chapter listed that survives scrutiny. The hosting cost is an

estimate anchored on a measured footprint, not an incurred charge, and no continuous-integration result is recorded for the baseline commit.

Data — evaluation. Every figure in Chapter Four derives from a single seeded demonstration farm. There has been no field trial and no real farmer data. The seeded records were constructed to exercise the platform’s analytical paths, so they are not a sample of Nigerian farm practice, and no inference about typical costs, margins or yields should be drawn from them.

The forecast tier — implementation and evaluation. The yield model is trained on synthetic data and is not validated against any real observation; it learns from no farm’s records and does not adapt from use. Its reported quality measures the recovery of a constructed relationship. It is, however, reproducible: the pipeline is seed-deterministic, the dataset carries a recorded fingerprint, and the three metrics were re-derived to six decimal places at the baseline by regenerating and refitting rather than by reading the sidecar. What is unproven is the model’s relevance, not the integrity of the procedure. The claim that the artefact was not reproducible from the repository, carried by an earlier draft, was wrong and understated the project’s own rigour. No claim of field-predictive accuracy is made, and the model should not be relied upon for a planting or input decision in its present state.

The decision-support outputs as advice — evaluation. The arithmetic of the deterministic tier is verified to an unusual degree. Whether acting on it improves an outcome is untested, and on synthetic training data and one seeded farm it is untestable here. The platform computes what a farm’s records imply; it does not know whether what they imply is good agronomy.

Correctness defects — implementation. Two are known and were reported in Section 4.15: a mistaken reversal cannot itself be rolled back, only compensated by a new unlinked entry, and a stale-revalidation window in the service worker is reasoned but unreproduced and is therefore a hypothesis rather than a defect. Neither touches the correctness of a figure the platform reports. Two further limitations of the same kind stand: per-operation equipment attribution is captured but costed nowhere, so no machine-hour, utilisation or cost-per-hour figure exists anywhere in this work, and the bioprocess summary endpoint has a typed client method and no caller. Three defects carried in earlier drafts are gone: the per-crop reversal-netting report was incorrect and is withdrawn, and unbounded monetary fields and the missing equipment edit path were both closed and verified. The residual set is materially smaller than the one this thesis previously carried, and the reason is worth stating plainly — the defects were found by the project’s own audit and closed before submission rather than being discovered by an examiner.

Verification boundaries — evaluation. The page and form layer has no component tests, including the drying-curve chart, whose visual correctness rests on inspection alone. The local-database migration is proven against an in-memory IndexedDB implementation rather than a browser. Backend tests run on SQLite while production runs PostgreSQL, and only the migration chain was executed against a real server; concurrency in particular is measured on SQLite and inferred for PostgreSQL. There is no end-to-end browser test, and the single driven-browser probe was taken at an earlier commit. The frontend suite is not timing-robust under machine contention. There is no independent security review, penetration test or threat model, and no load, stress or soak testing; the rate-limit counters, correspondingly, are in-process and neither span replicas

nor survive a restart, which is a residual worth naming rather than a control worth claiming.

Evaluation — the outstanding strand. The usability strand is outstanding, so there is no user evaluation, no task-completion measurement, no SUS score and no comparison against the paper-based practice this work is positioned against. Objective (iv) is partially achieved in consequence, and the claim the platform makes about cognitive load is the claim it has least evidence for.

5.2.4 Conclusion

This study set out to design, develop and evaluate AGRI-PROFIT, an offline-first farm record and decision-support platform for Nigerian smallholder and medium-scale enterprises, in response to a documented separation between operational and financial farm records and to the infrastructural and economic conditions that make existing farm management systems inaccessible to that user.

A working platform was built and evaluated at an identified implementation state, commit 78a68c2 of 29 August 2026. Its central design decision — that every operational record posts its financial transaction in the same commit, correctable only by a linked contra entry — was verified by a backend suite of 422 collected cases, 418 passing, 4 skipped and none failing, at 96% statement coverage over 1,594 statements, together with 148 frontend tests across 13 files at 43.18% statement coverage, and is visible in the live database as an exact one-to-one correspondence between 109 operational logs and 109 financial transactions with none unpaired. On that record the platform computes a deterministic decision-support layer that was shown to produce per-crop margin and ranking, unit cost on two bases, retrospective break-even yield, cost structure with an explicit coverage figure, a derived depreciation overlay, two

break-even prices, a sensitivity matrix, an operating-expense ratio, a partial budget and an Olympic-average yield baseline — and, as consistently, to withhold each figure where its inputs do not support it. A post-harvest drying module recovers known thin-layer parameters exactly, keeps water removed distinct from matter lost, and carries a moisture measurement through to a price per marketable kilogram, moving a maize enterprise from ₦35.00 to ₦41.67 per kilogram. The security boundary was exercised as well as asserted, across three roles and twelve permissions, expiring hashed share tokens, log scrubbing, throttling and input bounds. Three sets of negative-control runs establish that the tests behind these claims fail when the behaviour they guard is broken.

Measured against constrained conditions at the baseline, the application transfers 127 bytes of its shell on a repeat visit and renders with no connection at all, reaching first paint about eighty times faster warm than cold on a 0.4 Mbps link, though main content improves by a factor of about three and a half and that is the figure a user experiences. Its two exercised containers consume between 178.9 and 202.8 MiB, supporting an estimated ₦20,950 a month and a reasoned ₦105 per farm per month at a two-hundred-farm tenancy — a cost structure that is the opposite of the per-seat licensing identified in Chapter One as excluding this user.

What the work does not establish is equally definite. There has been no field trial, no real farmer data and no user evaluation; the usability strand of the specified evaluation is outstanding and Objective (iv) is therefore partially achieved. The forecast tier is trained on synthetic data and its reported accuracy is the recovery of a constructed relationship, not a claim about Nigerian yields. The data boundary is proven by tests and by probing a running stack, not by an independent security assessment, and the ledger is audit-trailed rather than cryptographically tamper-evident. The performance figures are modelled throttling against a local server rather than a field measurement. The feature-

phone channel that would reach the 68% is not implemented in any form. And while the platform has a production deployment path that has been built, started, probed, backed up and restored, it has never been deployed to serve a real user, on a real domain, on a departmental host — which is a smaller claim than an earlier draft of this chapter made, and it is the one the evidence supports.

The contribution, stated at its true size, is a verified prototype: an architecture in which the operational and financial record cannot drift apart, a decision-support layer that is arithmetically correct and honest about the limits of its inputs, and a demonstration that post-harvest process measurements can be carried through to a pricing decision automatically. To those this work adds a methodological result it did not set out to find — that a defect read out of code without being reproduced is not a defect — which is the reason its own correctness claims are tied to commands that were run. Whether that architecture changes what a Nigerian farmer earns is the question this work makes answerable and does not answer. The route to answering it is set out in Section 5.3, and it begins with putting the platform in front of its users.

5.3 Recommendations

5.3.1 Implications

For the smallholder and medium-scale farmer. The implication of the drying-to-price chain is specific and actionable: a farmer pricing against harvest mass is pricing against a mass that no longer exists. For the demonstration record the error is ₦6.67 on every kilogram, or 19%, and it is invisible without the moisture and mass figures the platform makes it trivial to record. More generally, the platform's outputs are of a kind a paper logbook cannot produce at all — not because the arithmetic is difficult, but because it

requires operational and financial records to be joined at the point of entry, which paper does not do.

For lenders, investors and cooperatives. A read-only, revocable, expiring, standardised report drawn from a ledger that refuses deletion is a materially different artefact from a farmer's own summary of their year. It is not cryptographically verifiable and should not be presented as such, but it changes the question a lender asks from “do I believe this figure?” to “do I believe this platform's boundary?” — and the latter is at least assessable. The cost structure compounds this: at ₦105 per farm per month at a two-hundred-farm tenancy, the natural adopter is an organisation serving many farms, which is also the party a lender would rather deal with.

For agricultural and bioresources engineering practice. The work is an instance of a general pattern worth naming: post-harvest process parameters that engineers already measure — moisture content, mass in and out, drying time — become financially decision-relevant the moment they are joined to a cost record. The engineering content of this platform is not the thin-layer models but that junction. The same pattern would extend to any process with a measurable yield loss: milling, threshing, storage shrinkage, or any conversion where the mass that leaves is not the mass that entered.

For low-resource FMIS design generally. Three design positions in this work appear to generalise. Pairing the operational and financial write removes a class of reconciliation problem rather than managing it. Withholding a figure whose inputs are insufficient is safer than defaulting it, and costs little to implement. And a service-worker precache converts a bandwidth problem into a first-visit problem, which is a much smaller problem — provided every write path explicitly invalidates the read cache, because the same layer will otherwise make the application lie about its own state.

For evaluation practice. A fourth position generalises from the retraction reported in Section 5.2.1: an audit that reads code and does not run it produces hypotheses, not findings, and a project that records those hypotheses as defects will carry them into its own thesis. The countermeasure is cheap — execute the path, and cite the command rather than the file.

5.3.2 Recommendations and Suggestions for Further Studies

The work below is prioritised in three tiers by what each is needed for. Tier A is required before any deployment; Tier B closes the evidence gaps this study leaves open; Tier C extends the contribution.

Tier A — required before any deployment.

- (a) Implement production observability — metrics, tracing and alerting. This is the one element of the deployment prerequisites that the hardening work did not close, and a single-instance service with no observability cannot be operated responsibly for a real farmer.
- (b) Obtain and install a TLS certificate for a real hostname and run the automatic-issuance path against it. The edge profile is configured and renders; it has never issued a certificate.
- (c) Externalise the model artefact so that the train-on-boot step no longer costs 8.57 s of unavailability on every restart and so that replicas become possible. The case for this rests on availability, not on hosting cost.
- (d) Move the rate-limit counters out of process, so that throttling spans replicas and survives a restart.

Tier B — required to close the evidence gaps.

- (a) Run the usability panel: three to five evaluators, the existing instrument pack, the completed instruments retained as scans. This closes Objective (iv) and is the highest-value remaining piece of evidence in the entire project.
- (b) Add component tests for the page and form layer, beginning with the drying-curve chart and the decision-support panels, which currently carry user-visible figures with no automated assertion.
- (c) Add an end-to-end browser test of the offline-to-online transition, the service-worker lifecycle and the installation path, and use it to settle the stale-revalidation window that is presently reasoned but unreproduced. Re-run the negative controls and the driven-browser probe at the submitted commit rather than attributing them by date.
- (d) Measure the concurrency behaviour on PostgreSQL rather than inferring it from the equivalent constraint, and run the backend suite against PostgreSQL rather than SQLite.
- (e) Commission an independent security review and conduct load testing sufficient to replace the reasoned tenancy figures with measured ones.
- (f) Record a continuous-integration run against the submitted commit, so that the verification figures carry a machine-attested provenance as well as a local one.
- (g) Capture the interface states named in Section 4.5.1 against the running system, following the project's capture run-sheet, and present them as figures in Chapter Four. Every capture must show a populated state that the system

actually produced, must carry a descriptive caption, must be referred to in the body text, and must not display a token or any other credential.

Tier C — to extend the contribution.

- (a) Retrain the forecast tier on accumulated real records once farms have a season of history, at which point the pre-processing pipeline described by Olisah et al. (2024) — imputation, unit harmonisation, interval reconciliation and outlier handling — becomes the substantive work rather than a design principle.
- (b) Report the forecast prediction interval prominently. A confidence band derived from the spread of the forest’s trees is a better answer to “why should a farmer trust a forecast?” than an in-distribution R^2 , because it communicates uncertainty rather than accuracy.
- (c) Consume the captured equipment identifier and machine hours to produce cost-per-hour and utilisation figures, which the data model already stores and no service currently reads. Any such figure must be bounded by the fact that machine hours are populated on three of eight seeded mechanisation logs, so a rate computed today would be a rate over an unknown fraction of use.
- (d) Give the bioprocess summary endpoint a consumer, so that per-crop drying aggregates reach the interface as the per-crop financial aggregates already do.
- (e) Extend the post-harvest module beyond drying, to any unit operation with a measurable mass loss, and conduct a real drying trial with instrumented equipment so that the fitted kinetics describe a process that was observed rather than seeded.

- (f) Implement the feature-phone entry channel against a live carrier gateway. Nothing of this channel exists today, so this is construction rather than extension, and it is the mechanism that would take the platform's reach beyond the 32% of rural Nigerians who own a smartphone.
- (g) Execute the field evaluation designed in Section 3.8.5 — a representative sample of smallholder and medium-scale farmers using the platform for their own records over a defined period, with task-completion rates, System Usability Scale scores across a larger sample, qualitative feedback, and real device and data-usage measurement. This is the study that would answer the question this work makes answerable and does not answer.

REFERENCES

- Abbasi, R., Martinez, P., & Ahmad, R. (2022). The digitization of agricultural industry — A systematic literature review on agriculture 4.0. *Smart Agricultural Technology*, 2, Article 100042. <https://doi.org/10.1016/j.atech.2022.100042>
- Abioye, D. O., Popoola, O., Akande, A., Fadare, D. A., Omitoyin, S. A., Yinusa, B., & Kolade, O. O. (2026). Farmers’ willingness to adopt digital application tools in Ogun State, Nigeria. *Journal of Strategy and Management*, 19(3), 602–622. <https://doi.org/10.1108/JSMA-06-2023-0135>
- Abiri, R., Rizan, N., Balasundram, S. K., Shahbazi, A. B., & Abdul-Hamid, H. (2023). Application of digital technologies for ensuring agricultural productivity. *Heliyon*, 9(12), Article e22601. <https://doi.org/10.1016/j.heliyon.2023.e22601>
- Baidhe, E., Clementson, C. L., Senyah, J., & Hammed, A. (2024). Appraisal of post-harvest drying and storage operations in Africa: Perspectives on enhancing grain quality. *AgriEngineering*, 6(3), 3030–3057. <https://doi.org/10.3390/agriengineering6030174>
- Basir, M. S., Buckmaster, D. R., Raturi, A., & Zhang, Y. (2024). From pen and paper to digital precision: A comprehensive review of on-farm recordkeeping. *Precision Agriculture*, 25(6), 2643–2682. <https://doi.org/10.1007/s11119-024-10172-7>
- Brooke, J. (1996). SUS: A “quick and dirty” usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.
- Carrer, M. J., de Souza Filho, H. M., & Batalha, M. O. (2017). Factors influencing the adoption of farm management information systems (FMIS) by Brazilian citrus farmers. *Computers and Electronics in Agriculture*, 138, 11–19. <https://doi.org/10.1016/j.compag.2017.04.004>

- Dayıoğlu, M. A., & Türker, U. (2021). Digital transformation for sustainable future — Agriculture 4.0: A review. *Journal of Agricultural Sciences*, 27(4), 373–399. <https://doi.org/10.15832/ankutbd.986431>
- Erbay, Z., & Icier, F. (2010). A review of thin layer drying of foods: Theory, modeling, and experimental results. *Critical Reviews in Food Science and Nutrition*, 50(5), 441–464. <https://doi.org/10.1080/10408390802437063>
- Food and Agriculture Organization of the United Nations. (n.d.). Maize: Post-harvest operations (Post-harvest Compendium). FAO. https://www.fao.org/fileadmin/user_upload/inpho/docs/Post_Harvest_Compendium_-_MAIZE.pdf
- Fountas, S., Carli, G., Sørensen, C. G., Tsiropoulos, Z., Cavalaris, C., Vatsanidou, A., Liakos, B., Canavari, M., Wiebensohn, J., & Tisserye, B. (2015). Farm management information systems: Current situation and future perspectives. *Computers and Electronics in Agriculture*, 115, 40–50. <https://doi.org/10.1016/j.compag.2015.05.011>
- Gebresenbet, G., Bosona, T., Patterson, D., Persson, H., Fischer, B., Mandaluniz, N., Chirici, G., Zacepins, A., Komasilovs, V., Pitulac, T., & Nasirahmadi, A. (2023). A concept for application of integrated digital technologies to enhance future smart agricultural systems. *Smart Agricultural Technology*, 5, Article 100255. <https://doi.org/10.1016/j.atech.2023.100255>
- Giua, C., Materia, V. C., & Camanzi, L. (2021). Management information system adoption at the farm level: Evidence from the literature. *British Food Journal*, 123(3), 884–909. <https://doi.org/10.1108/BFJ-05-2020-0420>
- Henderson, S. M., & Pabis, S. (1961). Grain drying theory I: Temperature effect on drying coefficient. *Journal of Agricultural Engineering Research*, 6(3), 169–174.

- Husemann, C., & Novković, N. (2012). The model of farm management information system: A case study of a diversified German farm. *DETUROPE — The Central European Journal of Regional Development and Tourism*, 4(1), 76–90.
- Jaiyeola, T. (2023). [AUTHOR TO SUPPLY: exact article title, publication date and URL of the BusinessDay Nigeria article reporting the GSMA figures that 32% of rural Nigerians owned a smartphone in 2022 against 58% of urban Nigerians]. *BusinessDay Nigeria*.
- Javaid, M., Haleem, A., Singh, R. P., & Suman, R. (2022). Enhancing smart farming through the applications of Agriculture 4.0 technologies. *International Journal of Intelligent Networks*, 3, 150–164. <https://doi.org/10.1016/j.ijin.2022.09.004>
- Kay, R. D., Edwards, W. M., & Duffy, P. A. (2016). *Farm management* (8th ed.). McGraw-Hill Education.
- Lewis, W. K. (1921). The rate of drying of solid materials. *Journal of Industrial and Engineering Chemistry*, 13(5), 427–432. <https://doi.org/10.1021/ie50137a021>
- Mhlanga, D., & Ndhlovu, E. (2023). Digital technology adoption in the agriculture sector: Challenges and complexities in Africa. *Human Behavior and Emerging Technologies*, 2023, Article 6951879. <https://doi.org/10.1155/2023/6951879>
- Nielsen, J. (1994). Heuristic evaluation. In J. Nielsen & R. L. Mack (Eds.), *Usability inspection methods* (pp. 25–62). John Wiley & Sons.
- Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 249–256). Association for Computing Machinery. <https://doi.org/10.1145/97243.97281>
- Olisah, C. C., Smith, L., Smith, M., Lawrence, M. O., & Ojukwu, O. (2024). Corn yield prediction model with deep neural networks for smallholder farmer decision

support system (arXiv:2401.03768) [Preprint]. arXiv.
<https://doi.org/10.48550/arXiv.2401.03768>

- Onwude, D. I., Hashim, N., Janius, R. B., Nawi, N. M., & Abdan, K. (2016). Modeling the thin-layer drying of fruits and vegetables: A review. *Comprehensive Reviews in Food Science and Food Safety*, 15(3), 599–618.
<https://doi.org/10.1111/1541-4337.12196>
- Page, G. E. (1949). Factors influencing the maximum rates of air drying shelled corn in thin layers [Master's thesis, Purdue University].
- Poppe, K., Vrolijk, H., & Bosloper, I. (2023). Integration of farm financial accounting and farm management information systems for better sustainability reporting. *Electronics*, 12(6), Article 1485. <https://doi.org/10.3390/electronics12061485>
- Tummers, J., Kassahun, A., & Tekinerdogan, B. (2019). Obstacles and features of farm management information systems: A systematic literature review. *Computers and Electronics in Agriculture*, 157, 189–204.
<https://doi.org/10.1016/j.compag.2018.12.044>
- van Klompenburg, T., Kassahun, A., & Catal, C. (2020). Crop yield prediction using machine learning: A systematic literature review. *Computers and Electronics in Agriculture*, 177, Article 105709. <https://doi.org/10.1016/j.compag.2020.105709>
- Xiong, H., Dalhaus, T., Wang, P., & Huang, J. (2020). Blockchain technology for agriculture: Applications and rationale. *Frontiers in Blockchain*, 3, Article 7.
<https://doi.org/10.3389/fbloc.2020.00007>